The Islamic University of Gaza

Faculty of Information Technology

بغـزة الجـامعـــــــة الإســـلاميـة

كلية تكنولوجيا المعلومات

On-Demand Home Services Platform for Connecting Craftsmen with Service Seekers

منصة خدمات منزلية عند الطلب لربط الحرفيين بأصحاب الحاجة

By

Student Name

Yara Hatem Jawad Mahdi 220221483

Sozan Waleed Khaled Ashour 220220178

Roba Adnan Ahmed Hamada 220222192

Nora Ayman Shareef AlBatsh 220222713

Supervised by

Dr. Ahmed Qazzaz

A graduation project report submitted in partial fulfilment of the requirements for the degree of Bachelor of Information Technology

July 2026

Abstract

In light of the exceptional conditions currently affecting Gaza, which have resulted in widespread destruction of infrastructure and residential buildings, a large number of residents have been forced to live in damaged or partially destroyed homes due to the lack of alternative housing options. This reality has led to a significant increase in the demand for home services, particularly services provided by skilled craftsmen such as carpenters, metalworkers, and maintenance workers. These services have become a pressing necessity rather than a luxury.

On the other hand, people in need face considerable difficulty in accessing reliable craftsmen who are capable of performing work inside damaged homes while adhering to appropriate schedules and reasonable pricing, especially given the reliance on traditional and unorganized methods for finding such services. At the same time, craftsmen themselves struggle to obtain consistent job opportunities despite possessing the required skills and experience, particularly under challenging economic conditions.

This project aims to design and develop an on-demand home services platform that directly and systematically connects craftsmen with people in need, while taking into account the unique context of Gaza and the increased demand for maintenance and temporary repair services within damaged homes. The platform provides a user-friendly interface that allows craftsmen to present their services and enables users to quickly access suitable service providers without the need for intermediaries.

The Agile methodology is adopted in the development of the system due to its flexibility in handling changing requirements and its support for continuous improvement throughout the development phases. This project contributes to addressing a real need arising from current circumstances, supporting the craftsmen sector, and improving access to home services, with the potential for future development into a practical and scalable real-world system.

ملخص الدراسة

في ظل الظروف الاستثنائية التي تمر بها غزة، وما نتج عنها من تدمير واسع في البنية التحتية والمنازل السكنية، اضطر عدد كبير من السكان إلى السكن في بيوت متضررة أو شبه مدمرة، لعدم توفر بدائل أخرى. هذا الواقع أدى إلى زيادة ملحوظة في الطلب على الخدمات المنزلية، لا سيما خدمات الحرفيين مثل النجارة، والحدادة، وأعمال الصيانة والترميم المؤقت، حيث أصبحت هذه الخدمات ضرورة ملحّة وليست رفاهية.

في المقابل، يواجه أصحاب الحاجة صعوبة كبيرة في الوصول إلى حرفيين موثوقين، قادرين على تلبية متطلبات العمل داخل المنازل المتضررة.

في ظل الاعتماد على وسائل تقليدية وأساليب غير رسمية او معارف شخصية او وسطاء غير موثوقين للبحث عن هذه الخدمات. كما يعاني الحرفيون أنفسهم من صعوبة الوصول إلى فرص عمل منتظمة، رغم امتلاكهم للخبرة والمهارة، خاصة في ظل الظروف الاقتصادية الصعبة.

يهدف هذا المشروع إلى تصميم وتطوير منصة خدمات منزلية عند الطلب تعمل على ربط الحرفيين بأصحاب الحاجة بشكل مباشر ومنظم، مع الأخذ بعين الاعتبار طبيعة الواقع في غزة، وزيادة الطلب على أعمال الصيانة والترميم داخل المنازل المتضررة. توفر المنصة واجهة سهلة الاستخدام تتيح للحرفيين عرض خدماتهم، وتمكّن المستخدمين من الوصول السريع إلى الحرفيين المناسبين دون الحاجة إلى وسطاء.

تم اعتماد منهجية Agile في تطوير النظام لما توفره من مرونة في التعامل مع المتطلبات المتغيرة، وتحسين مستمر خلال مراحل التطوير. يساهم هذا المشروع في تلبية حاجة حقيقية ناتجة عن الظروف الراهنة، ودعم فئة الحرفيين، وتحسين الوصول إلى الخدمات المنزلية، مع إمكانية تطوير المنصة مستقبلًا لتكون نظامًا عمليًا قابلًا للتطبيق على أرض الواقع

Dedication

To our beloved parents,

whose endless love, patience, and prayers have lighted our path and given us the strength to reach this moment.

To our families and friends, who stood beside us with constant encouragement and unwavering belief in us throughout this journey.

And to every craftsman and every hardworking hand in Gaza, whose resilience and skill inspired the very idea behind this work.

We proudly dedicate this project.

Acknowledgment

First and foremost, we thank God Almighty for granting us the strength, patience, and determination to complete this project.

We would like to express our deepest gratitude to our supervisor, [Supervisor Name], for the valuable guidance, continuous support, and insightful feedback that shaped this work from its earliest idea to its final form.

Our sincere thanks also extend to the Faculty of Information Technology at the Islamic University of Gaza, and to all our instructors, whose knowledge and dedication provided us with the skills that made this project possible.

Finally, we are profoundly grateful to our families and friends for their unconditional love, encouragement, and patience throughout our academic journey. To all of you, we say thank you.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Meaning
AI	Auto Increment (Auto-Generated)
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DDoS	Distributed Denial of Service
ER	Entity–Relationship
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDX	Index
IUG	Islamic University of Gaza
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express, React, Node.js
MIME	Multipurpose Internet Mail Extensions
NN	Not Null
NoSQL	Not Only SQL
ODM	Object Document Mapper
OTP	One-Time Password
PK	Primary Key
REF	Reference (Foreign Key)
REST	Representational State Transfer
SMS	Short Message Service
SQL	Structured Query Language
UC	Use Case
UI	User Interface
UN	Unique
URL	Uniform Resource Locator
UX	User Experience
XSS	Cross-Site Scripting

Chapter 1
Introduction

Chapter 1
Introduction

In recent years, Gaza has experienced exceptional and unprecedented circumstances due to a prolonged and destructive war that has caused catastrophic damage to residential buildings and infrastructure. According to the final Rapid Damage and Needs Assessment jointly conducted by the European Union, the United Nations, and the World Bank in April 2026, more than 371,888 housing units in the Gaza Strip have been destroyed or damaged, more than 60% of the population has lost their homes, and 1.9 million people have been displaced [1]. Satellite-based assessments by UNOSAT similarly found that approximately 81% of all structures in Gaza sustained some level of damage [2].

Reconstruction needs are estimated at USD 71.4 billion over the coming decade, with housing identified as the hardest-hit sector [1].

The war also generated an estimated 61 million tonnes of rubble, and debris clearance alone is projected to take up to seven years even under ideal conditions [3]. Field assessments have found that residents whose homes sustained only partial damage have already begun repairing them independently and moving back in, often salvaging materials from the rubble around them [3], and shelter/housing has been ranked the single highest-priority need by humanitarian coordination bodies operating in Gaza, ahead of food and hygiene items [4]. Together, this evidence shows a massive, urgent, and sustained demand for craftsmen's services such as carpentry, blacksmithing, and general home maintenance and repair, driven by residents trying to restore a minimum level of safety and privacy in their homes.

Globally, on-demand service platforms have proven to be an effective way to meet exactly this kind of demand by connecting skilled workers with people who need their services through simple, accessible digital tools rather than heavy infrastructure. The global home-improvement services market was valued at USD 785.49 billion in 2024 and is projected to nearly double within a decade [5], while the broader global gig economy reached an estimated USD 556.7 billion in the same year [6]. Established platforms such as TaskRabbit and Thumbtack have demonstrated the viability of this model at scale, together connecting several hundred thousand independent workers with clients across thousands of service categories [7]. Regionally, platforms such as Mrsool in Saudi Arabia and Justmop in the UAE show that this model adapts successfully to Arabic-speaking markets, with Mrsool alone reaching four million registered users [8]. On a more conceptual level, the International Labour Organization notes that in developing economies, where a large share of economic activity takes place outside formal channels, digital platforms increasingly serve as intermediaries connecting informal workers to customers [9], and the World Bank has explicitly recommended developing digital job-matching platforms as a means of connecting self-employed Palestinians and freelancers to clients and improving employment outcomes [10]. This body of evidence supports the choice to build Forsa as a digital matching platform rather than relying on informal, unorganized word-of-mouth channels.

To build such a platform under Gaza's unstable and unpredictable working conditions, this project adopted the Agile Scrum methodology. Scrum is widely recognized in the software engineering literature as particularly suited to projects with high uncertainty and volatile requirements, since long-term planning and forecasting tend to be ineffective in such

environments; instead, work is organized into short, incremental sprints that allow the team to adapt continuously as circumstances change [11].
This made it a natural fit for a team operating under recurring internet outages, power interruptions, and shifting priorities. On the technical side, the project was implemented using the MERN stack: MongoDB [12] as a flexible, JSON-like NoSQL database; Express.js [13] as a lightweight backend framework; React [14] for building the user interface; and Node.js [15] as the server-side runtime. The MERN stack is a modern, open-source, and cost-effective technology stack that unifies development around a single language, JavaScript, across both the client and the server, which simplifies development, reduces context-switching, and lowers overhead for a small student team with limited resources [16].
Despite the clear demand and the proven viability of the on-demand platform model elsewhere, a significant gap remains in Gaza's local market that existing solutions do not address. Unemployment in the Gaza Strip reached 78% in 2025, and the local economy contracted by 84% compared to pre-war levels [17], leaving a large pool of skilled tradespeople with severely reduced and irregular income opportunities. At the same time, informal employment accounts for over half of all Palestinian employment outside agriculture, and roughly two-thirds of that informal employment is concentrated in construction and related trades, representing well over 150,000 workers who operate with no contracts, no organized visibility, and no structured channel connecting them to clients [18]. Existing international platforms such as TaskRabbit and Thumbtack are not adapted to this context: they assume stable, high-bandwidth internet connectivity, integrated card or online payment systems, and formal registration and background-check processes. None of these assumptions hold in Gaza. Telecommunications infrastructure has been repeatedly and severely disrupted, with recurring multi-day blackouts affecting large parts of the network throughout the war [19]; the banking system faces a severe liquidity crisis that makes integrated digital payment impractical for most residents; and approximately 90% of the Palestinian population already relies on WhatsApp as its primary communication tool [20]. This combination of factors is what the proposed platform, Forsa, is specifically designed to address: a lightweight, web-based application that allows craftsmen to be found and contacted directly via WhatsApp, without requiring integrated payments, continuous high-speed connectivity, or complex registration — offering a practical, locally-adapted solution rather than simply replicating a model built for a fundamentally different market.

1.1 Statement of the problem

Despite the significant increase in demand for skilled tradespeople's services in Gaza, access to these services remains hampered by several challenges that negatively impact both those in need and the tradespeople themselves. Residents often rely on informal methods to find tradespeople, such as personal recommendations or social media groups, making it difficult to locate reliable .tradespeople capable of meeting the demands of work within damaged homes

The core problem lies in the absence of a unified and organized online platform that facilitates direct connections between tradespeople and those in need. This platform should provide clear information about the types of services offered, approximate prices, tradespeople's availability, and service quality assessments. This lack of a platform leads to numerous problems, most notably delays in project completion, failure to meet deadlines, price discrepancies, and a decline .in trust between the two parties

Furthermore, tradespeople struggle to find regular employment opportunities, despite possessing the necessary skills and experience, particularly given the difficult economic conditions and limited income sources. This situation results in a waste of time and effort for both parties and .undermines the efficiency of home-based services within the community

Based on the above, there is a need for an effective technological solution that organizes the process of providing and requesting home services, and contributes to improving communication between craftsmen and those in need, taking into account the nature of the reality in Gaza and the increasing needs for maintenance and restoration work inside damaged homes

1.2 Objectives

1.2.1 Main Objective

This project aims to design and develop an online platform for on-demand home services. This platform will connect skilled tradespeople with those in need in an organized and reliable manner, thereby facilitating access to home services and improving their efficiency under the current circumstances in Gaza

.

1.2.2 Specific Objectives

1-Design a user-friendly and simple interface that enables users to request home services quickly and .clearly
.2-Enable tradespeople to create accounts and showcase their services and expertise in an organized way .
3-Develop an order management system that facilitates communication and coordination between the . two parties
4-Test and evaluate the system's performance to ensure service quality and ease of use .

1.3 Scope and Limitations

1.3.1 Scope

1- The platform is designed exclusively to serve users and tradespeople within the Gaza Strip and will not be available or functional outside it.
2- Home services only: The platform focuses solely on tradespeople specializing in home-related services such as carpentry, blacksmithing, and maintenance, and will not expand to freelancers or general employment opportunities.

3- Web-based system: The platform will be delivered as a web application consisting of a frontend, a backend, and a database.
4- Three system components: The system covers user management, tradesperson management, service listings, and service requests.
5- No registration required for browsing: Users can search for tradespeople and view service information without creating an account.
6- There is no direct communication on the platform between the craftsman and the homeowner.

1.3.2 Limitations

1- The platform will not include a smartphone application in its current phase due to time and resource constraints; this may be considered in future development.
2- No electronic payment system: The platform does not support online payment due to the absence of reliable and accessible payment infrastructure in Gaza.
3- No real-time service tracking: Real-time tracking of service requests or tradesperson location is not implemented, as it requires additional infrastructure and integration that exceeds the current project scope.
4- Use of dummy data: Real user and tradesperson data is not available at this stage, so development and testing will rely on dummy data.
5- Infrastructure and connectivity limitations: The Gaza Strip's unstable internet infrastructure poses technical challenges that limit certain features requiring stable or high-speed connectivity.

1.4 Significations

This project carries significance on several levels. On the social and humanitarian level, it directly addresses a pressing need created by the exceptional circumstances in Gaza, where widespread damage to residential buildings has sharply increased the demand for craftsmen's services such as carpentry, blacksmithing, and general maintenance. By providing an organized channel that connects craftsmen with people in need, the platform helps ease the daily struggle of residents trying to restore a minimum level of safety and privacy in their homes.
On the economic level, the platform supports craftsmen, many of whom face reduced and irregular income opportunities under the current conditions, by giving them a wider and more reliable channel to reach potential clients. This helps sustain a vital segment of the local workforce and preserve traditional craft skills that might otherwise be lost due to the lack of steady work.
On the technical and academic level, the project demonstrates the practical application of modern web development practices, including the MERN stack, RESTful API design, and secure authentication mechanisms, within a real and constrained environment. It also reflects the value of adopting Agile practices to manage a graduation project under unpredictable circumstances, such as recurring internet outages and limited resources.
Finally, the project lays a foundation that can be extended in future work into a fully-fledged, real-world platform, with the potential to scale beyond Gaza to other regions facing similar challenges in accessing organized home services.

Chapter 2
Related Works

Chapter 2
Related Works

2.1 Task rabbit

TaskRabbit is a global on-demand service platform founded in the United States. It connects individuals with local service providers (referred to as “Taskers”) who offer services such as carpentry, plumbing, electrical repairs, furniture assembly, and home maintenance. The platform relies heavily on geolocation technology to match users with nearby service providers and facilitates online booking and electronic payment **[21]**

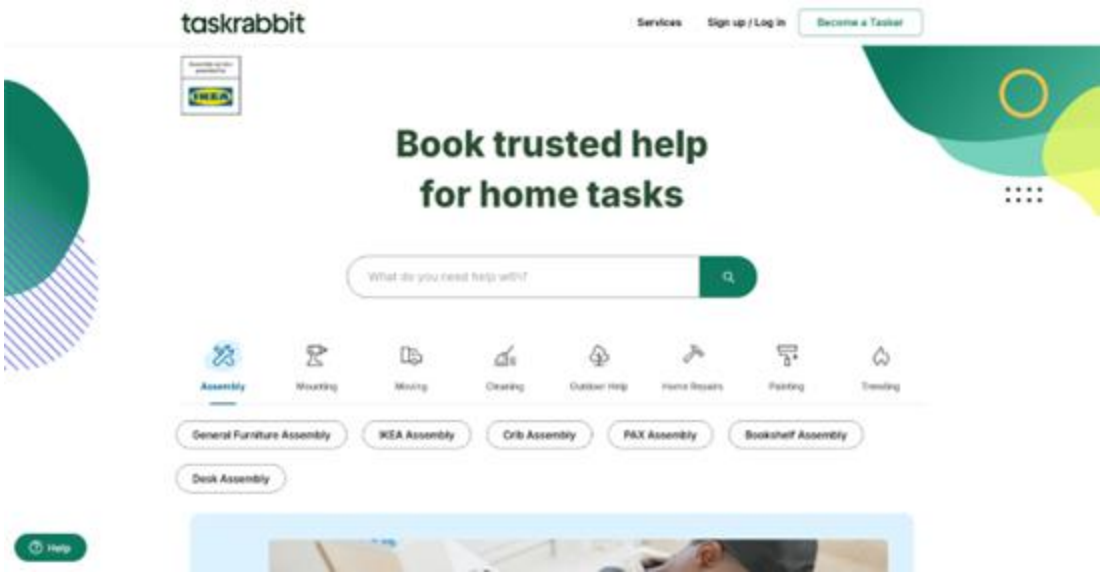


Figure 2.1 Taskrabbit website Screenshots [21]

Strength

- Advanced location-based matching system.
- Integrated electronic payment system.
- Verified service providers and structured rating system.
- Organized and categorized service listings

Weakneese:

- Full reliance on electronic payment methods (credit/debit cards).
- Requires stable internet connectivity for real-time operation.
- Designed primarily for economically stable and digitally advanced markets.

May not be suitable for crisis-affected or infrastructure-limited regions

2.2 Thumbtack

Thumbtack is a U.S.-based service marketplace that enables customers to request services and receive price quotes from multiple professionals. It covers a wide range of services including home repair, event services, cleaning, and personal services. The system operates through a request-and-bid model, where professionals respond to customer requests with customized offers.**[22]**

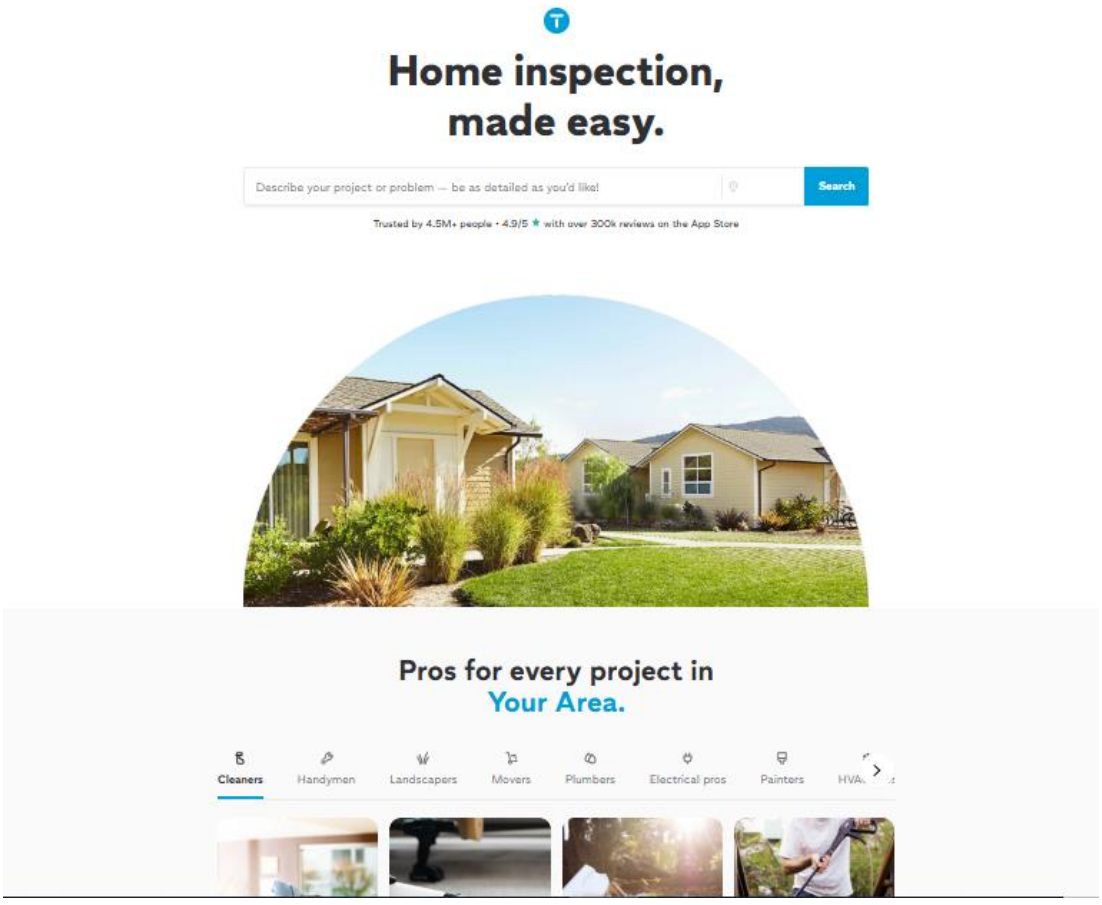


Figure 2.2 Thumbtack website Screenshots [22]

Strengths

- Competitive quotation system that allows price comparison.
- Detailed professional profiles with reviews and ratings.
- Wide service coverage across multiple sectors.
- Strong digital infrastructure and automated workflow management.

Weaknesses

- Requires continuous digital interaction from service providers.
- Primarily designed for structured and formally registered professionals.
- Depends on electronic payment and advanced digital literacy.
- More complex interface compared to lightweight service platforms

2.3 Souq Al-Harafyeen

Souq Al-Harafyeen is an Arabic online platform that aims to connect craftsmen with customers seeking manual and technical services. The platform operates primarily as a digital directory, allowing craftsmen to create profiles and display their services across categorized fields such as plumbing, carpentry, electrical work, and maintenance. Users can browse available categories and contact service providers directly through the information provided on the platform.

Unlike fully automated global platforms, Souq Al-Harafyeen focuses on facilitating visibility and direct communication rather than managing the entire service transaction process [23]

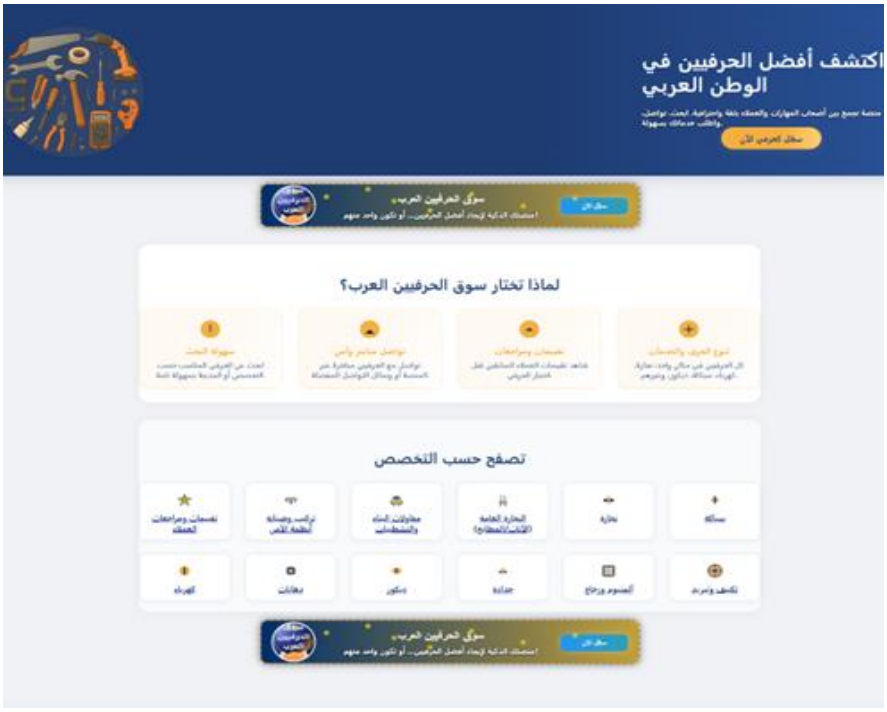


Figure 2.3 Souq Al-Harafyeen website Screenshots [23]

Advantages:

- Focuses on tradespeople and field services.
- Arabic interface.
- Diverse range of trades (carpentry, plumbing, electrical work, interior decorating, etc.).
- Easy to search by trade.

Disadvantages:

- Lacks a clear internal payment system.
- Limited reach compared to global platforms.
- • Limited or incomplete rating system.

2.4 Comparing features

Table 2.1: Comparison of Platform Features

Feature	Thumbtack	TaskRabbit	Souq Al-Harafyeen	Our App
Specialized in Home Services	No	Yes	Yes	Yes
Craftsmen-Focused (not freelancers)	No	No	Yes	Yes
Electronic Payment Required	Yes	Yes	No	No
Algorithmic Matching	Yes	Yes	No	No
Verified Craftsmen Profiles	Yes	Yes	No	Yes
Designed for Crisis/Conflict Context	No	No	No	Yes
Works with Low Internet Infrastructure	No	No	No	Yes
Direct WhatsApp Contact	No	No	No	Yes

Chapter 3
Methodology

Chapter 3
Methodology

3.1 Methodology

The Agile Development Methodology [24] was adopted using the Scrum framework [25] to implement the Forsa project. This methodology was selected due to its iterative nature, which allows the project to be divided into short development cycles known as Sprints. This approach provides flexibility in handling changing requirements and technical challenges that may arise during development, especially considering the nature of the project and the local environment.

The following reasons justified the selection of Agile:

1. **Flexibility:** The ability to modify system features based on feedback from the academic supervisor or insights gained from initial system usage.
2. **Continuous Improvement:** Conducting regular testing after each sprint helps improve system quality and gradually reduce software errors.
3. **Risk Management:** Identifying technical issues at early stages of the project instead of discovering them at later stages, which reduces their impact on overall project progress.
4. **Suitability for the Working Environment:** Since the team works partially remotely and under unstable internet conditions, Agile allows tasks to be divided into small, independent units that can be completed separately.

Although the core principles of Agile methodology were followed, several adjustments were made to suit the nature of an academic graduation project:

- **Client Role:** In standard Scrum, there is a role known as the Product Owner. In this project, the academic supervisor assumes this role through periodic review sessions, where feedback and suggested modifications are reviewed and approved.
- **Sprint Duration:** Each sprint was set to last two weeks, in order to align with the university's approved timeline and milestone submissions.
- **Documentation:** Unlike traditional Agile approaches that minimize documentation, this project emphasized preparing essential academic documents in parallel with development, such as the Software Requirements Specification (SRS) and the System Design Specification (SDS)

3.1.1 Requirement Stage

To ensure that the system meets real user needs, several data collection methods suitable for the project were used:

- **Competitive Analysis:** Studying and analyzing similar home-service platforms to identify strengths and weaknesses and benefit from best practices in system design.
- **Brainstorming and Observation:** Conducting brainstorming sessions among team members based on an analysis of the local market and community needs, especially under current conditions, which helped define the core system features.
- **Academic Supervisor Review Sessions:** Feedback from the academic supervisor during periodic meetings was adopted as a primary reference for finalizing system requirements and guiding the development process.

3.1.2 Development Phases

According to the adopted Agile methodology, the project was implemented through a set of iterative phases within each sprint, as follows:

1. **Planning and Analysis:** Identifying system requirements and formulating them as User Stories, such as: *"As a craftsman, I want to create an account to display my services."*
2. **Design:** Converting requirements into prototypes (Wireframes and Mockups) using UI design tools.
3. **Development:** Implementing the frontend and backend components of the system and integrating them with the server and database.
4. **Testing:** Conducting Unit Testing and Integration Testing to ensure the correct functionality of system features.
5. **Review and Deployment:** Presenting the completed system version to the academic supervisor, collecting feedback, and applying improvements in the next sprint.
6. **System Evaluation and Validation:** Final validation of the system is conducted through User Acceptance Testing (UAT) and structured review sessions with the academic supervisor, ensuring that all functional requirements defined in the SRS have been successfully implemented.

The project followed an incremental development approach, where each sprint focused on a specific system feature, which was analyzed, designed, developed, and tested within the same sprint.

3.2 Time Table

The project duration was divided into several short development cycles (**Sprints**), each with a predefined duration and a set of planned tasks.
The project timeline is illustrated in **Table (3.1)**, which outlines the implementation phases, sprint durations, and main tasks accomplished during each sprint. This timeline demonstrates structured project planning, gradual progress, and adherence to the approved academic schedule.

Table 3.1: Sprint Schedule

Sprint	Mission	Start date	End date	Estimated duration
Sprint 1	Project Setup	18 Dec 2025	24 Dec 2025	6 Days
	Planning	21 Dec 2025	27 Dec 2025	6 Days
	Distribute tasks	23 Dec 2025	30 Dec 2025	7 Days
Sprint 2	User Interface (UI/UX)	30 Jan 2026	15 Feb 2026	15 Days
	Front End Implementation	20 Feb 2026	3 Mar 2026	8 Days
	Database Design & Backend Development	15 Mar 2026	19 Mar 2026	4 Days
Sprint 3	Design & implement craftsman registration	20 Mar 2026	25 Mar 2026	5 Days
	Connect page to DB & test registration	27 Mar 2026	30 Mar 2026	3 Days
Sprint 4	Design craftsman's details page	1 April 2026	5 April 2026	4 Days
	Integrate WhatsApp	7 April 2026	9 April 2026	2 Days

3.3 Tools and Equipment

3.3.1 Equipment

Table 3.2: Equipment Specifications

Device	Specifications
Laptops	Moderate specifications, minimum 8GB RAM, capable of running local development environments

3.3.2 Tools

Table 3.3: Tools and Technologies

Tool / Technology	Description	Category
Visual Studio Code	Main source code editor	Development Environment
Postman	API testing and validation	Development Environment
Figma	UI/UX design and wireframing	Design
Git & GitHub	Version control and collaborative development	Collaboration
Notion	Task organization and progress tracking	Project Management
React.js	Frontend library for building interactive user interfaces	Frontend
Node.js	JavaScript runtime for server-side logic	Backend
Express.js	Web framework for handling server requests and routing	Backend
MongoDB (Atlas)	Cloud-based NoSQL database for flexible data storage	Database
Google Meet	Weekly meetings to review progress	**Communication**

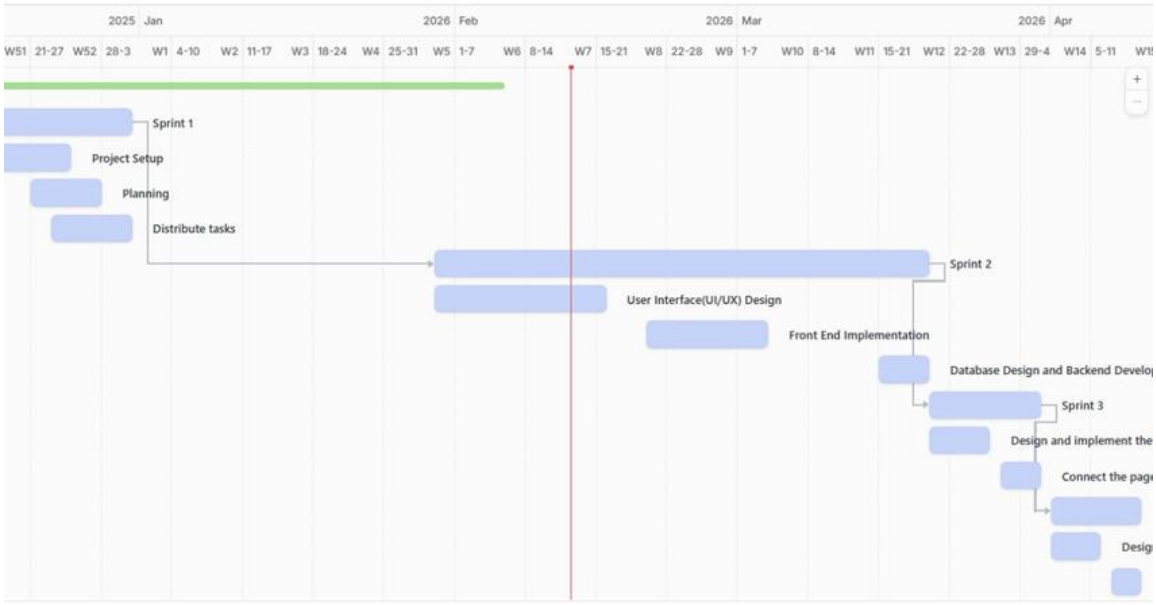


Figure 3.1 Time line

Chapter 4
Requirement Analysis

Chapter 4
Requirement Analysis

4.1 Requirement Gathering

The idea behind the Forsa platform was formed and shaped based on a direct observation of a clear local need. Given the widespread structural damage affecting homes across the Gaza Strip, a large number of homeowners are in urgent need of skilled craftsmen for repair and reconstruction work, while no reliable digital platform exists to connect both parties.

The team relied on three main sources to gather the system requirements:

First: Direct Problem Observation The team observed that the only channel currently available is informal Facebook groups. While these groups serve a community purpose, they suffer from several practical shortcomings: craftsmen are not categorized by service type or location, inquiries are frequently left unanswered, and no mechanism exists to verify the credibility or quality of the listed craftsmen.

Second: Competitor Analysis and Related Works A structured review was conducted of existing platforms in the local, regional, and global market, including TaskRabbit, Khamsat, and Mostaqel. The analysis revealed that none of these platforms adequately addresses the local needs of Gaza — whether in terms of Arabic-first interface design, availability of relevant trade categories, or compatibility with limited infrastructure conditions.

Third: Internal Team Discussions and Supervisor Review The team conducted internal brainstorming sessions to translate the observed problems into actionable system requirements. These sessions covered defining user types, core platform functionalities, and system boundaries. The outcomes were periodically reviewed with the academic supervisor to ensure methodological soundness and technical feasibility.

4.2 Actors

The Forsa platform involves three types of actors, each interacting with the system in a different way:

- **Visitor:** Any individual who accesses the platform without creating an account. Visitors can browse the platform, search for craftsmen by service type, and view craftsman profiles. To initiate contact with a craftsman via WhatsApp, the visitor is required to fill a simple form providing their name and phone number. This information is stored as a service request in the craftsman's requests list, allowing the craftsman to track and manage incoming inquiries.

- **Craftsman:** A registered service provider who has created a verified account on the platform. Craftsmen can log in, manage their profile information, view and track service requests submitted by visitors, and update their availability and service details.
- **Admin:** A privileged system operator who manages the platform through a dedicated administration dashboard. The admin is responsible for reviewing and approving craftsman registration requests, managing active accounts, and overseeing the overall content and integrity of the platform.

4.3 Functional Requirements

4.3.1 Visitor Functionalities

- The system shall allow visitors to browse all listed craftsmen without requiring account registration.
- The system shall allow visitors to search for craftsmen by name or keyword..
- The system shall allow visitors to filter craftsmen by service type (e.g., carpentry, blacksmithing, maintenance)
- The system shall allow visitors to view a craftsman's full profile, including name, trade type, location, contact details, service description, and work portfolio images
- The system shall allow visitors to submit a service request form containing their name and phone number before being redirected to contact the craftsman via WhatsApp
- The system shall store the submitted request in the craftsman's requests list upon form submission.

4.3.2 Craftsman Functionalities

- The system shall allow craftsmen to register a new account by providing their personal and professional information, along with up to 3 portfolio images at the time of registration.
- The system shall allow craftsmen to log in and log out of their accounts securely.…
- The system shall allow craftsmen to edit their profile information, including name, trade type, location, service description, and contact details.
- The system shall allow craftsmen to upload additional work portfolio images after registration.
- The system shall allow craftsmen to view a list of all service requests submitted by visitors
- The system shall allow craftsmen to update the status of each request to either Accepted or Rejected

- The system shall allow craftsmen to delete their profile image.

4.3.3 Admin Functionalities

- The system shall allow the admin to log in to a dedicated administration dashboardThe control panel should allow admin to add new category.
- The system shall allow the admin to deactivate or permanently delete craftsman accounts
- The system shall allow the admin to assign or remove the "Featured Craftsman" designation for any craftsman listed on the platform.

4.4 Non-Functional Requirements

1. Usability
The system should ensure usability by providing a simple and intuitive user interface that requires no technical background to navigate through the platform pages.
The system should ensure usability by displaying a clear loading state to the user while data is being fetched from the server.

2. Compatibility
The system should ensure compatibility by remaining fully functional and properly displayed on both desktop and mobile screens, achieved through CSS Media Queries with breakpoints at 480px, 768px, and 1024px.

3. Security
The system should ensure security by encrypting craftsman passwords using the bcryptjs library [26] with 10 salt rounds, ensuring passwords are never stored in plain text.
The system should ensure security by protecting all craftsman-related routes through JWT token authentication [27].
The system should ensure security by validating all user inputs on two levels: client-side validation in the browser, and server-side validation at the database model layer.

4. Reliability
The system should ensure reliability by handling server communication errors gracefully without crashing, implemented through try/catch blocks on all API requests.

5. Performance
The system should ensure performance by displaying craftsman listings in a paginated format, reducing the amount of data loaded at once per request.

6. Maintainability
The system should ensure maintainability by following a component-based architecture that supports future scalability and the addition of new features without major restructuring.

4.5 Use Case Diagram

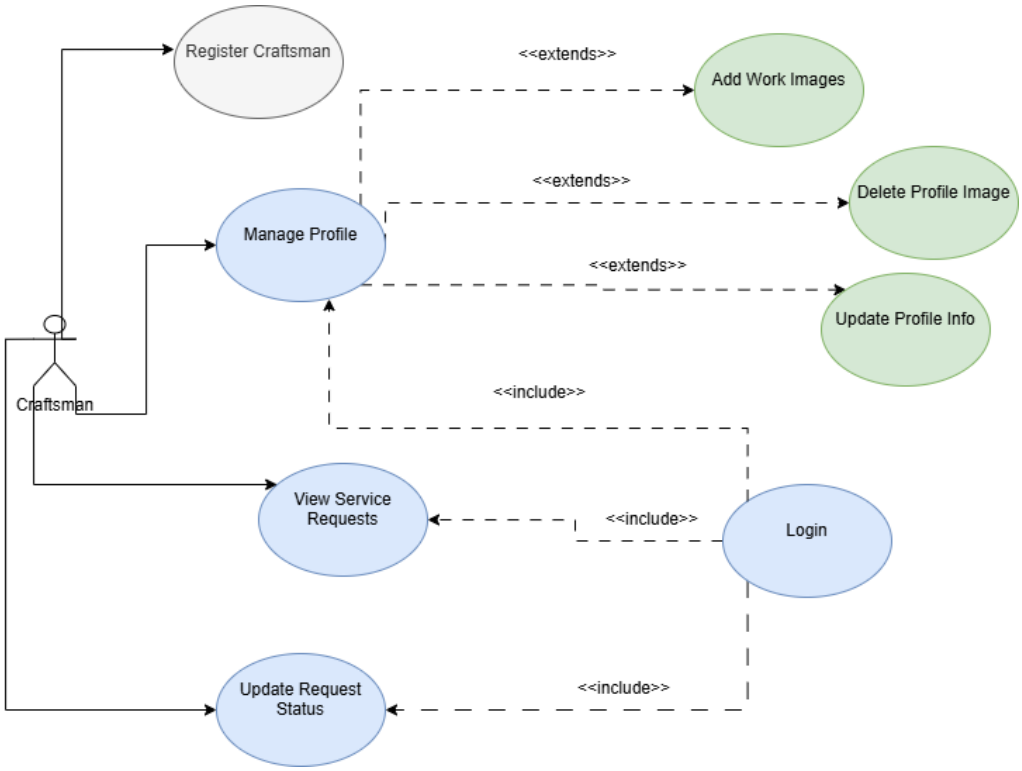


Figure 4.1 Craftsman use cases diagram

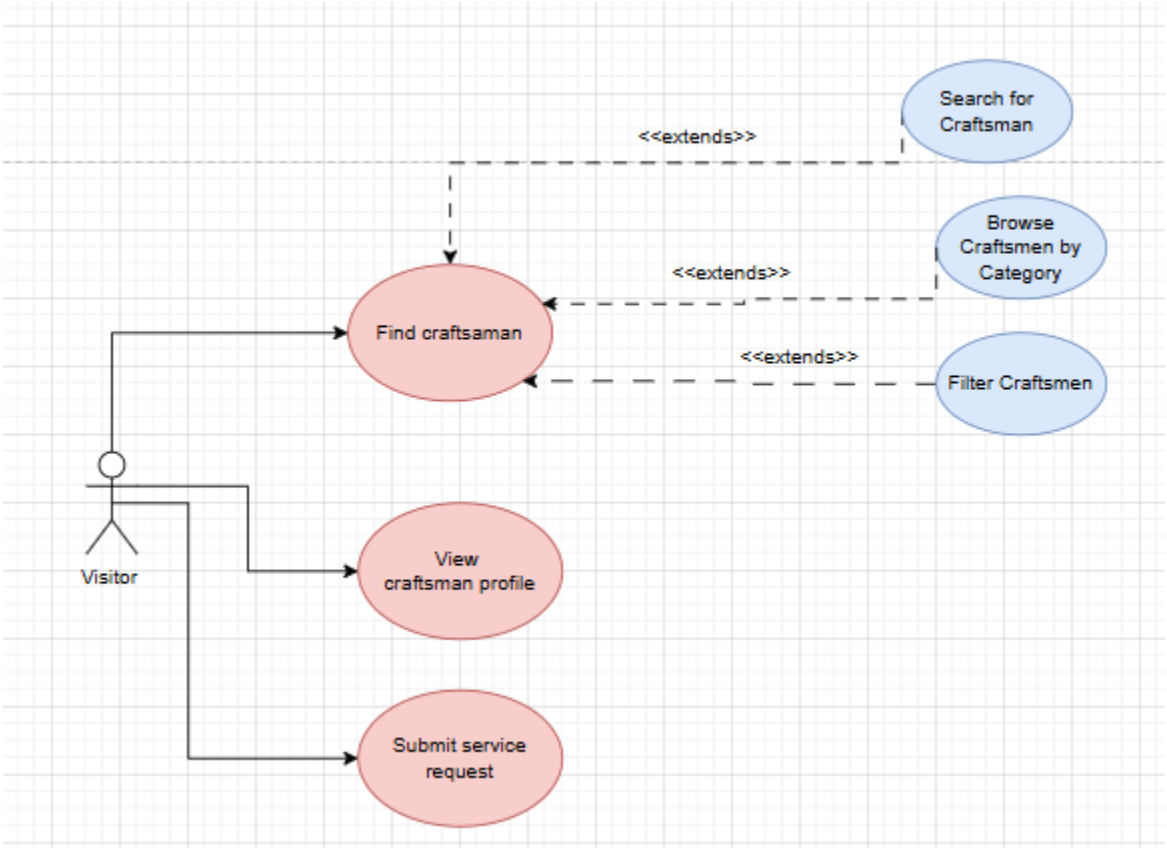


Figure 4.2 visitor Use Cases diagram

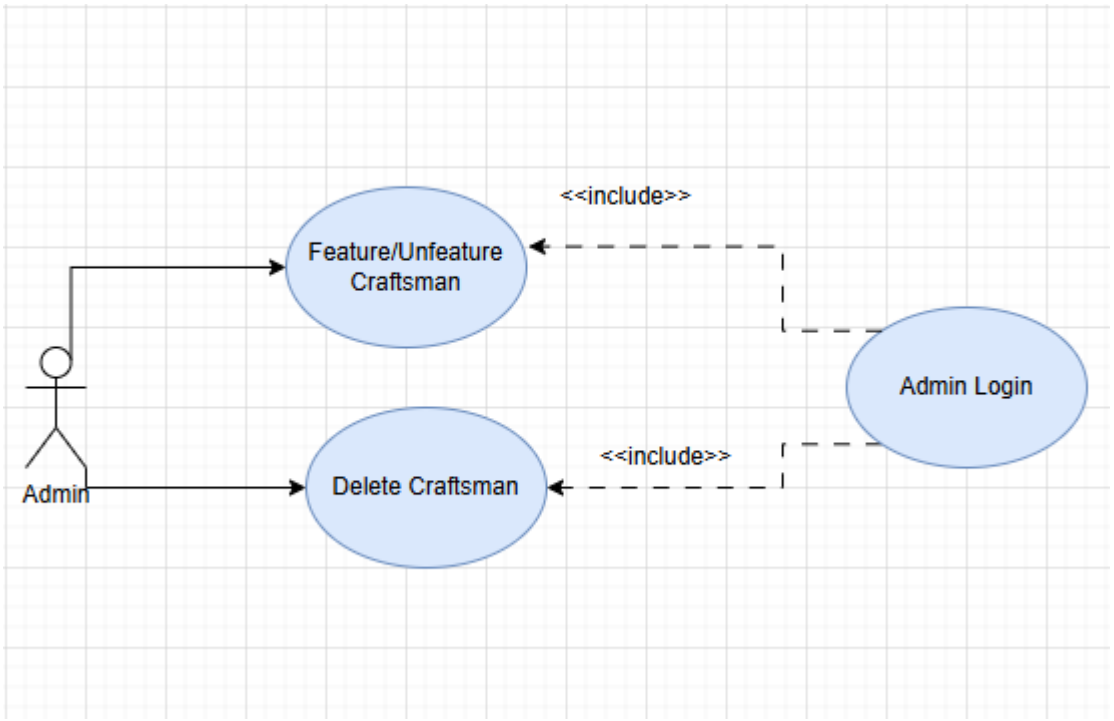


Figure 4.3 Admin Use Cases diagram

4.6 Use Cases

Table 4.1: Use Cases List

ID	Name	Description
UC 01	Register Craftsman	A new craftsman creates an account through a 3-step form including personal info, professional details, and work images.
UC 02	Craftsman Login	A registered craftsman logs in using email and password and is redirected to the homepage.
UC 03	Admin Login	The admin logs in using a dedicated email and password and is redirected to the admin dashboard.
UC 04	Search for Craftsman	A visitor searches for a craftsman by name or trade type using the search bar with auto-suggestions.
UC 05	View Craftsman Profile	A visitor views a craftsman's full profile including contact details, location map, and work images.
UC 06	Submit Service Request	A visitor fills a form with their name and phone number and is then redirected to contact the craftsman via WhatsApp.
UC 07	Manage Profile	An authenticated craftsman updates their personal and professional information.
UC 08	Add Work Images	An authenticated craftsman uploads additional work portfolio images after registration.
UC 09	View Service Requests	An authenticated craftsman views the list of all incoming service requests.
UC 10	Update Request Status	An authenticated craftsman confirms a service request.
UC 11	Feature/Unfeature Craftsman	The admin assigns or removes the featured designation for a craftsman.
UC 12	Delete Craftsman	The admin permanently deletes a craftsman account from the platform.
UC 13	Browse Craftsmen by Category	A visitor selects a trade category from the homepage (e.g., Engineer, Carpenter) and is redirected to a page listing all craftsmen registered under that category.
UC 14	Filter Craftsmen	A visitor filters the craftsmen listing page by trade type, region, or price range to narrow down the search results.
UC 15	Delete Profile Image	The craftsman removes their current profile picture from their account

4.7 Use Cases Descriptions

Table 4.2: Use Case UC-01 Register Craftsman

UC 01 Register Craftsman	
Actor	Craftsman
Environment	Web Browser
Priority	High
Pre-Conditions	The craftsman does not have an existing account on the platform
Flow	1. The craftsman navigates to the registration page. 2. The craftsman fills in Step 1: first name, last name, email, password, confirm password, and phone number, then clicks "Next". 3. The craftsman fills in Step 2: trade type, years of experience, region, price range, detailed address, and an optional bio, then clicks "Next". 4. The craftsman uploads exactly 3 work portfolio images in Step 3, then clicks "Create Account". 5. The system validates all inputs and creates the account. 6. The craftsman is automatically logged in and redirected to the homepage
Post-Conditions	- A new craftsman account is created and the craftsman is logged in.
Alternative Flow (1)	If any required field is missing or invalid, the system displays an inline error message and prevents progression to the next step.
Alternative Flow (2)	If the email is already registered, the system displays an error message indicating that the account already exists
Alternative Flow (3)	If fewer than 3 work images are uploaded, the system displays an error and prevents account creation.

Table 4.3: Use Case UC-02 Craftsman Login

UC-02: Craftsman Login	
Actor	Craftsman
Environment	Web Browser
Priority	High
Pre-Conditions	The craftsman has a registered account on the platform
Flow	1. The craftsman navigates to the login page. 2. The craftsman enters their email and password. 3. The craftsman clicks the "Login" button. 4. The system validates the credentials. 5. The craftsman is redirected to the homepage and their session is stored via JWT token.

Post-Conditions	- The craftsman is successfully authenticated and logged in.
Alternative Flow (1)	If the email or password is incorrect, the system displays an error message: " خطأ في كلمة المرور أو الإيميل...."

Table 4.4: Use Case UC-03 Admin Login

UC-03 Admin Login	
Actor	Admin
Environment	Web Browser
Priority	High
Pre-Conditions	The admin has a dedicated email and password configured in the system.
Flow	1. The admin navigates to the login page. 2. The admin enters the designated admin email and password. 3. The admin clicks the "Login" button. 4. The system recognizes the admin credentials. 5. The admin is redirected to the admin dashboard.
Post-Conditions	- The admin is successfully authenticated and has access to the admin dashboard.
Alternative Flow (1)	If the email or password is incorrect, the system displays an error message and prevents access to the dashboard.

Table 4.5: Use Case UC-04 Search for Craftsman

UC-04 Search for Craftsman	
Actor	Visitor
Environment	Web Browser
Priority	High
Pre-Conditions	-
Flow	1. The visitor navigates to the homepage. 2. The visitor types a craftsman name or trade type in the search bar. 3. The system displays auto-suggestions based on the input. 4. The visitor selects a suggestion or clicks "Search". 5. The system displays a results page with all matching craftsmen
Post-Conditions	- A list of craftsmen matching the search query is displayed.

Alternative Flow (1)	If no craftsmen match the query, the system displays a message indicating no results were found.

Table 4.6: Use Case UC-05 View Craftsman Profile

UC-05 View Craftsman Profile	
Actor	Craftsman
Environment	Web Browser
Priority	High
Pre-Conditions	-
Flow	1. The visitor clicks on a craftsman card from any listing or search results page. 2. The system displays the craftsman's full profile including name, trade type, region, years of experience, price range, phone number, email, address, location map, and work portfolio images.
Post-Conditions	- The craftsman's full profile is displayed to the visitor
Alternative Flow (1)	If the craftsman's profile fails to load due to a connection error, the system displays an appropriate error message.

Table 4.7: Use Case UC-06 Submit Service Request

UC-06 Submit Service Request	
Actor	Visitor
Environment	Web Browser
Priority	High
Pre-Conditions	The visitor is viewing a craftsman's profile page.
Flow	1. The visitor clicks the "Request Service" button on the craftsman's profile. 2. A form popup appears requesting the visitor's full name and phone number. 3. The visitor fills in the form and clicks "Send Request". 4. The system saves the request to the craftsman's requests list. 5. A success message is displayed with two WhatsApp contact options (+972 and +970). 6. The visitor selects a WhatsApp option and is redirected to a WhatsApp chat with the craftsman.

Post-Conditions	- The service request is saved in the craftsman's requests list and the visitor is connected to the craftsman via WhatsApp.
Alternative Flow (1)	If the visitor leaves the name or phone number field empty, the system displays a validation error and prevents submission.

Table 4.8: Use Case UC-07 Manage Profile

UC-07 Manage Profile	
Actor	Craftsman
Environment	Web Browser
Priority	High
Pre-Conditions	The craftsman is logged in.
Flow	1. The craftsman navigates to their profile page. 2. The craftsman edits any of their information including name, trade type, region, price range, address, or bio. 3. The craftsman clicks the save button. 4. The system validates and saves the updated information. 5. The profile is updated and the changes are reflected immediately.
Post-Conditions	- The craftsman's profile information is updated successfully.
Alternative Flow (1)	If any required field is left empty or contains invalid data, the system displays an error and prevents saving.
Alternative Flow (2)	If the session has expired, the system redirects the craftsman to the login page.

Table 4.9: Use Case UC-08 Add Work Images

UC-08 Add Work Images	
Actor	Craftsman
Environment	Web Browser
Priority	Medium
Pre-Conditions	The craftsman is logged in and has an existing account with at least 3 work images.
Flow	1. The craftsman navigates to their profile page. 2. The craftsman uploads one or more additional work images. 3. The system validates the file type (JPEG, PNG, WebP) and size (max 2MB per image). 4. The images are saved and displayed in the craftsman's portfolio.

Post-Conditions	- The new images are added to the craftsman's work portfolio and visible on their public profile.
Alternative Flow (1)	If the uploaded file exceeds 2MB or is an unsupported format, the system displays an error and rejects the upload.
Alternative Flow (2)	If the total number of images exceeds 12, the system prevents further uploads.

Table 4.10: Use Case UC-09 View Service Requests

UC-09 View Service Requests	
Actor	Craftsman
Environment	Web Browser
Priority	High
Pre-Conditions	The craftsman is logged in.
Flow	1. The craftsman navigates to their profile page. 2. The system retrieves and displays all incoming service requests associated with the craftsman's account. 3. Each request shows the visitor's name, phone number, and current status.
Post-Conditions	- The craftsman can view all service requests submitted by visitors.
Alternative Flow (1)	If no requests have been submitted yet, the system displays an empty state message.

Table 4.11: Use Case UC-10 Update Request Status

UC-10 Update Request Status	
Actor	Craftsman
Environment	Web Browser
Priority	Medium
Pre-Conditions	The craftsman is logged in and has at least one incoming service request.
Flow	1. The craftsman views the list of incoming service requests. 2. The craftsman selects a request and changes its status to confirmed 3. The system saves the updated status.
Post-Conditions	- The service request status is updated and reflected in the requests list.

Alternative Flow (1)	If the update fails due to a connection error, the system displays an error message and retains the previous status.

Table 4.12: Use Case UC-11 Feature / Unfeature Craftsman

UC-11 Feature / Unfeature Craftsman	
Actor	Admin
Environment	Web Browser
Priority	Medium
Pre-Conditions	The admin is logged in and viewing the admin dashboard.
Flow	1. The admin views the craftsmen list on the dashboard. 2. The admin clicks the "Feature" button next to a craftsman to assign the featured designation, or clicks "Unfeature" to remove it. 3. The system updates the craftsman's featured status. 4. The craftsman appears or is removed from the featured section on the platform.
Post-Conditions	- The craftsman's featured status is updated and reflected on the platform.
Alternative Flow (1)	If the update fails, the system displays an error message and retains the previous status.

Table 4.13: Use Case UC-12 Delete Craftsman

UC-12: Delete Craftsman	
Actor	Admin
Environment	Web Browser
Priority	High
Pre-Conditions	The admin is logged in and viewing the admin dashboard.
Flow	1. The admin views the craftsmen list on the dashboard. 2. The admin clicks the "Delete" button next to a craftsman. 3. The system permanently removes the craftsman's account and all associated data.
Post-Conditions	- The craftsman's account is permanently deleted and no longer appears on the platform.

Alternative Flow (1)	If the deletion fails due to a server error, the system displays an error message and the account remains active.

Table 4.14: Use Case UC-13 Browse Craftsmen by Category

UC-13 Browse Craftsmen by Category	
Actor	Visitor
Environment	Web Browser
Priority	High
Pre-Conditions	-
Flow	1. The visitor views the trade category section on the homepage. 2. The visitor clicks on a category icon (e.g., Engineer, Carpenter, Electrician). 3. The system redirects the visitor to a dedicated page listing all craftsmen registered under that category.
Post-Conditions	- A filtered list of craftsmen belonging to the selected trade category is displayed.
Alternative Flow (1)	If no craftsmen are registered under the selected category, the system displays an appropriate empty state message.

Table 4.15: Use Case UC-14 Filter Craftsmen

UC-14: Filter Craftsmen	
Actor	Visitor
Environment	Web Browser
Priority	Medium
Pre-Conditions	The visitor is on the craftsmen listing page.
Flow	1. The visitor opens the filter panel on the craftsmen listing page 2. The visitor selects one or more filters: trade type, region, or price range. 3. The system dynamically updates the displayed craftsmen list based on the selected filters.
Post-Conditions	- The craftsmen list is filtered and displays only craftsmen matching the selected criteria.
Alternative Flow (1)	If no craftsmen match the selected filters, the system displays a message indicating no results were found.

Table 4.16: Use Case UC-15 Delete Profile Image

UC-15: Delete Profile Image	
Actor	Craftsman
Environment	Web Browser
Priority	Low
Pre-Conditions	The craftsman is logged in and currently has a profile image set.
Flow	1. The craftsman opens their profile dashboard page. 2. The craftsman clicks the "The X mark above the profile image " 3. The system deletes the image file from the server and clears the image field in the database. 4. The system displays a confirmation message
Post-Conditions	- The profile image is removed, and the profile is displayed without a picture.

Chapter 5
System Design

Chapter 5
System Design

5.1 System Architecture

The Forsa platform follows a three-tier architecture separating the presentation, business logic, and data layers. The frontend is built with React.js and Vite [28], hosted on Vercel [29]. The backend is built with Node.js and Express, hosted on Render [30]. The database is MongoDB Atlas [31], a cloud-hosted NoSQL database. The frontend communicates with the backend through HTTP REST API requests, and the backend communicates with the database through Mongoose [32] queries.

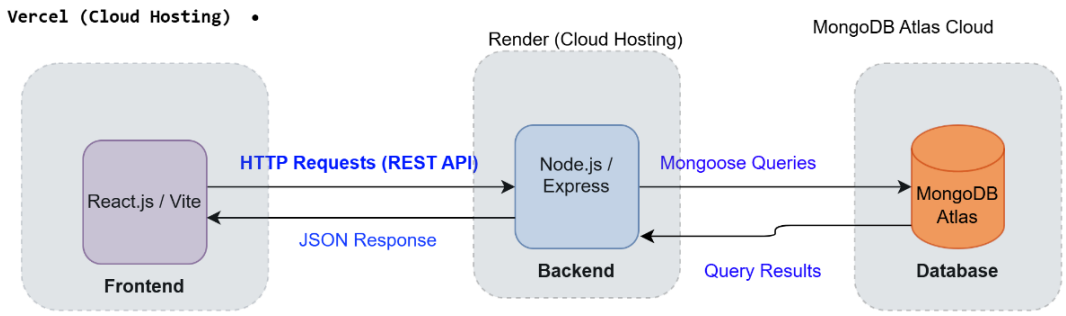


Figure 5.1: System Architecture Diagram

5.2 Data Flow

5.2.1 Core Authentication Flows

The following sequence diagram illustrates the login flow for craftsmen. The user submits their credentials through the frontend, which forwards the request to the backend. The backend validates the credentials against the database using bcrypt password comparison, and returns a JWT token upon success.

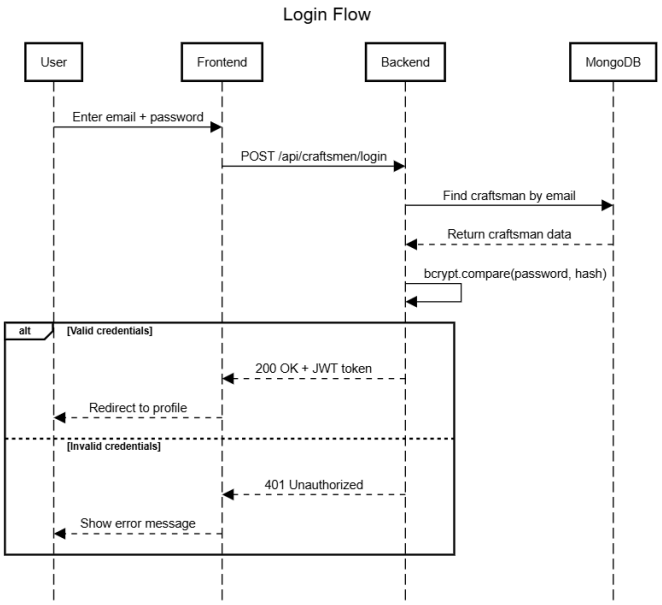


Figure 5.2: Login Sequence Diagram

5.2.2 Service Discovery Flows

The following sequence diagram illustrates the search flow. The user applies filters such as profession, city, or name through the frontend. The backend sanitizes the input to prevent NoSQL injection, queries MongoDB with the filter object and pagination parameters, and returns the matching craftsmen list.

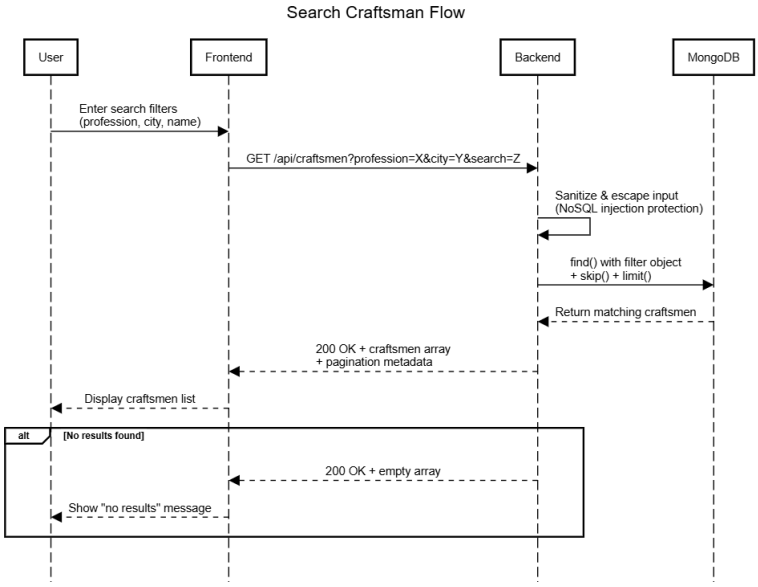


Figure 5.3: Search Craftsman Sequence Diagram

5.2.3 Transaction Management Flows

The following sequence diagram illustrates the service request submission flow. The user fills in their name, phone number, and job details. The backend validates the input, verifies the

craftsman exists, saves the request with a pending status, and returns the craftsman's phone number to the user.

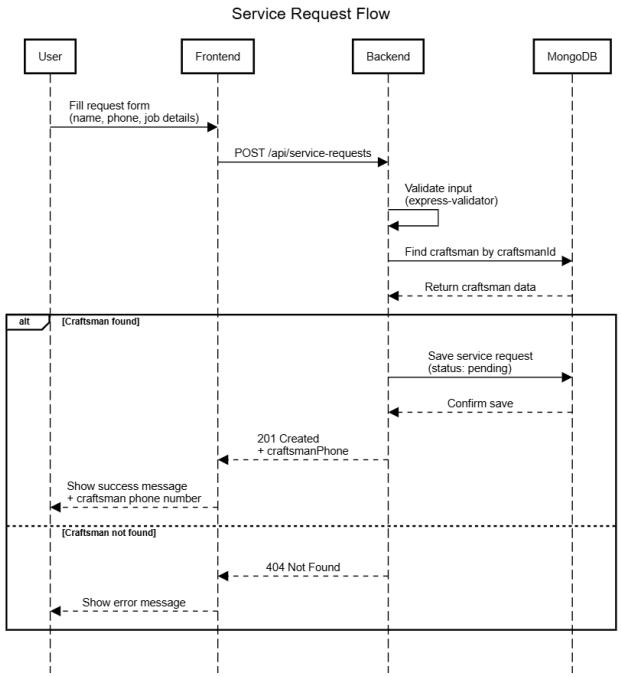


Figure 5.4: Service Request Sequence Diagram

5.3 Database Design

5.3.1 Overview

The Forsa platform uses MongoDB, a document-oriented NoSQL database. MongoDB was selected for several reasons: the development team gained practical experience with it during their academic studies, which accelerated the development process and reduced the learning curve. Additionally, MongoDB's flexible schema structure makes it well-suited for storing craftsman profiles that contain multiple fields and arrays, such as portfolio work images, which would require more complex handling in a traditional relational database.

To interact with MongoDB from the Node.js environment, Mongoose was used as the ODM (Object Document Mapper). Mongoose provides an abstraction layer over the MongoDB driver, enabling the definition of clear schemas with built-in validation rules, which ensures the integrity of stored data.

For hosting, the database was deployed on MongoDB Atlas, a cloud-based database service, for several reasons: the service provides a free cluster sufficient for the project's requirements; Atlas clusters operate natively as a Replica Set, which enables the use of MongoDB Transactions required for the cascade delete operations implemented in the system; and Atlas provides automatic backups and a visual interface for performance monitoring.

The database connection string (MONGO_URI) is loaded from environment variables (.env) using the dotenv library. Any connection failure is handled by an error handler that immediately

terminates the server process (process.exit) rather than allowing the application to run in an unstable state.

5.3.2 Entities and Relationships

The Forsa platform database consists of two main collections: the craftsmen collection and the servicerequests collection. The following sections detail each collection and its fields.

Craftsman Collection:

This collection is the core of the system, storing all information related to craftsmen registered on the platform. Mongoose automatically assigns the collection name in plural form based on the model name (Craftsman → craftsmen). Regarding image storage, a design decision was made to save portfolio images and profile pictures as relative file paths on local disk rather than relying on external cloud storage services such as Cloudinary or AWS S3, in order to maintain simplicity within the project scope.

Table 5.1: Craftsman Collection Schema

Field Name	Data Type	Attributes	Validation / Constraints
_id	ObjectId	PK, AI	Auto-generated by MongoDB
firstName	String	NN	trim
lastName	String	NN	trim
email	String	NN, UN, IDX	Gmail only — regex: /^[^\s@]+@gmail\.com$/i, lowercase, trim
password	String	NN	minlength: 8 — stored as bcrypt hash (10 salt rounds)
securityQuestion	String	NN	must be one of 3 predefined values (SECURITY_QUESTIONS list)
securityAnswer	String	NN (hashed)	normalized: lowercase + trim before hashing — stored as bcrypt hash (10 salt rounds)
profession	String	NN, IDX	trim
city	String	NN, IDX	trim — enum: ["Gaza", "North Gaza", "Middle Area", "South"]
neighborhood	String	NN	trim
phone	String	NN	regex: /^059\d{7}$/ (Palestinian mobile format)
yearsOfExperience	Number	NN	min: 0
price	Number	NN	min: 1
bio	String	--	trim — default: ""

Field Name	Data Type	Attributes	Validation / Constraints
profileImage	String	--	File path — default: ""
workImages	[String]	--	Array of file paths — min: 3, max: 12 (custom validator)
averageRating	Number	--	min: 0, max: 5 — default: 0
ratingsCount	Number	--	min: 0 — default: 0
featured	Boolean	IDX	Admin-controlled — default: false
createdAt	Date	Auto	Auto-generated by Mongoose (timestamps: true)
updatedAt	Date	Auto	Auto-generated by Mongoose (timestamps: true)

Attributes Legend: PK = Primary Key, NN = Not Null (required: true), UN = Unique, IDX = Indexed, REF = Reference (Foreign Key), AI = Auto Generated.

ServiceRequests Collection:

This collection stores all service requests submitted by clients to craftsmen through the platform. User registration is not required to submit a request; the client only needs to provide their name and phone number. Mongoose automatically assigns the collection name in plural form based on the model name (ServiceRequest → servicerequests). Each service request is linked to a single craftsman through the craftsmanId reference field, establishing a one-to-many relationship where one craftsman can receive many service requests.

Table 5.2: ServiceRequests Collection Schema

Field Name	Data Type	Attributes	Validation / Constraints
_id	ObjectId	PK, AI	Auto-generated by MongoDB
clientName	String	NN	trim
clientPhone	String	NN	10–13 digits — Arabic-Eastern digits normalized to Western automatically
craftsmanId	ObjectId	NN, IDX, REF	ref: "Craftsman" — Foreign key to craftsmen collection
status	String	--	enum: ["pending", "confirmed", "contacted", "completed", "cancelled"] — default: "pending"
jobDetails	String	--	trim — default: "" — Work description provided by the client
createdAt	Date	Auto	Auto-generated by Mongoose (timestamps: true)

Field Name	Data Type	Attributes	Validation / Constraints
updatedAt	Date	Auto	Auto-generated by Mongoose (timestamps: true)

The following indexes are explicitly defined on this collection to improve query performance:

Table 5.3: ServiceRequests Indexes

Index	Fields	Type	Purpose
Index 1	{ craftsmanId: 1 }	Single field	Fast retrieval of all requests for a specific craftsman
Index 2	{ craftsmanId: 1, status: 1, createdAt: -1 }	Compound	Efficient filtering by craftsman and status, sorted by newest first

Relationships Between Collections

The platform uses Referenced Relationships rather than Embedded Documents, meaning that documents live in separate collections and are linked through ObjectId references.

Table 5.4: Relationships Between Collections

Implementation	Type	Relationship
servicerequests.craftsmanId holds an ObjectId pointing to craftsmen._id	One-to-Many	Craftsman → ServiceRequests
Not yet implemented — listed under future improvements	One-to-Many (planned)	Craftsman → Reviews

Rationale for Referenced over Embedded:

Referenced Relationships were chosen because service requests are independent entities with their own lifecycle (pending → confirmed → completed). The craftsman needs to view and update them independently from their profile data. Embedding them inside the craftsman document would cause the document to grow excessively over time and make updates more complex.

Important notes on relationships:

There is no automatic cascade delete in MongoDB. Therefore, cascade delete logic was implemented manually in the backend: when a craftsman is deleted by the admin, all associated service requests are deleted within a single MongoDB Transaction to guarantee data consistency.

The averageRating and ratingsCount fields on the craftsman document are denormalized summary fields — they store a pre-computed statistical summary rather than being calculated on the fly from a reviews collection. This is a deliberate design decision to improve query performance.

5.3.3 Constraints and Validation Rules

The platform applies a two-layer validation approach to ensure the integrity of stored data:

First Layer — Request Level (express-validator):

Applied at the request level before it reaches the database, defined in separate validator files independent from the controllers. This layer covers checking for required fields, data formatting, and enum values.

Second Layer — Schema Level (Mongoose):

Applied at the schema level as a last line of defense before saving to the database. The following table outlines the key validation rules applied:

Table 5.5: Schema Validation Rules

Field	Rule	Layer
email	Must match Gmail format (@gmail.com) — regex	Both layers
phone	Must start with 059 and consist of 10 digits — regex	Both layers
city	Must be one of four predefined cities — enum	Both layers
status	Must be one of five predefined values — enum	Both layers
workImages	Minimum 3 and maximum 12 images — custom validator	Schema level
price	Must be greater than or equal to 1 — min	Both layers
yearsOfExperience	Must be greater than or equal to 0 — min	Both layers
averageRating	Must be between 0 and 5 — min/max	Schema level
clientPhone	Arabic-Eastern digits automatically converted to Western before validation — custom sanitizer	Request level
status transition	Governed by a state machine — reverting from completed or cancelled is not permitted	Controller level

5.3.4 Indexes

The following indexes are defined in the database to improve the performance of the most frequently executed queries:

Craftsmen Collection:

Table 5.6: Craftsmen Collection Indexes

Index	Field(s)	Type	Purpose
Default	_id	Single field	Auto-generated by MongoDB
Unique	email	Single field	Auto-created due to unique: true — prevents duplicate emails and speeds up lookups
	profession	Single field	Filtering craftsmen by profession
	city	Single field	Filtering craftsmen by city
	featured	Single field	Retrieving featured craftsmen for the homepage
Compound	{ profession: 1, city: 1 }	Compound	Combined filtering by profession and city — the most common search pattern

ServiceRequests Collection:

Table 5.7: ServiceRequests Collection Indexes

Index	Field(s)	Type	Purpose
Default	_id	Single field	Auto-generated by MongoDB
	{ craftsmanId: 1 }	Single field	Retrieving all requests for a specific craftsman
Compound	{ craftsmanId: 1, status: 1, createdAt: -1 }	Compound	Efficient filtering by craftsman and status, sorted from newest to oldest

Note on Trade-offs:

Indexes improve read performance but increase storage consumption and slightly slow down write operations, since MongoDB updates the index on every insert or modification. This trade-off is acceptable in the Forsa platform because read operations (browsing craftsmen) significantly outnumber write operations (registering a new craftsman).

5.3.5 Query Optimization

Several strategies are applied to optimize database query performance in the Forsa platform:

1. Selective Field Loading

All queries that return craftsman data use .select("-password") to exclude the password field from the response. This reduces the size of returned documents and prevents accidental exposure of sensitive data.

2. Filtering Before Fetching

The getAllCraftsmen endpoint applies profession, city, and search filters directly in the MongoDB query using the filter object, rather than fetching all documents and filtering in memory. This ensures only matching documents are transferred from the database to the application.

3. Pagination

All list endpoints (GET /api/craftsmen, GET /api/admin/craftsmen, GET /api/service-requests/me) implement pagination using .skip() and .limit() combined with a parallel countDocuments() call. This prevents loading thousands of records in a single request as the database grows.

4. Regex Escaping for Search

User input passed to MongoDB regex queries is escaped using a custom escapeRegex utility before use. This prevents malicious inputs from causing catastrophic backtracking (ReDoS) that could block the MongoDB thread.

5. Compound Indexes

A compound index on { profession: 1, city: 1 } supports the most common search pattern on the platform — filtering by both profession and city simultaneously — without requiring two separate index lookups.

5.3.6 Views and Stored Procedures

MongoDB, as a NoSQL document-oriented database, does not support traditional SQL Views or Stored Procedures in the same manner as relational databases such as MySQL or PostgreSQL. As an alternative, the Forsa platform handles the equivalent functionality at the application level:

Views equivalent:

Complex data retrieval logic is encapsulated within dedicated controller functions. For example, the getAllCraftsmen controller combines filtering, searching, and pagination in a single reusable function, similar to how a SQL View would simplify a complex query.

Stored Procedures equivalent:

Repeated database operations such as cascade delete (removing a craftsman along with all associated service requests and image files) are encapsulated within dedicated controller functions wrapped in MongoDB Transactions, ensuring atomicity and data consistency — the same goals that Stored Procedures serve in relational databases.

5.3.7 Security Considerations

The platform relies on multiple security layers to protect user data:

1. Password Hashing

Passwords are never stored in plain text. Upon registering a new craftsman, the password is hashed using the bcryptjs library with 10 salt rounds before being saved to the database. During login, bcrypt.compare() is used to compare the entered password against the stored hash. Additionally, the password field is excluded from all API responses using .select("-password").

2. Authentication (JWT)

Upon successful login, a JSON Web Token (JWT) is issued, signed with a secret key stored in environment variables, and valid for 7 days. The craftsman must send this token with every request that requires authorization via the Authorization header in the format Bearer <token>. The verifyToken middleware verifies the token's validity and expiry before allowing any protected operation to proceed.

3. Admin Authentication

The admin relies on an authentication system completely independent from the craftsman system. Admin credentials (email + bcrypt hash of the password) are stored in environment variables only — there is no admin record in the database. Upon successful admin login, a JWT carrying role: "admin" in the payload is issued, and the verifyAdmin middleware checks for this role before granting access to any admin route.

4. Input Sanitization

To protect the database from NoSQL Injection attacks, the express-mongo-sanitize library is applied at the app level before processing any request. This library removes any key starting with $ or containing . from req.body, req.query, and req.params, preventing attackers from injecting MongoDB operators.

5. File Upload Security

Image files are handled through the multer library with the following restrictions: allowed MIME types are image/jpeg, image/png, and image/webp only; maximum file size is 2MB; and maximum number of files per request is 10. To prevent XSS attacks via files, the original filename is not used to determine the extension; instead, the extension is derived from the validated MIME type through a fixed mapping (image/jpeg → .jpg), preventing the upload of malicious .html files.

6. Security Headers (Helmet [33])

The helmet library is applied at the app level to add HTTP security headers to every response:

Table 5.8: Security Headers (Helmet)

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevents the browser from guessing the file type
X-Frame-Options	SAMEORIGIN	Protection against Clickjacking
Strict-Transport-Security	max-age=15552000	Enforces HTTPS in production
X-Powered-By	(removed)	Hides information about the tech stack

7. Rate Limiting

To protect the platform from Brute Force and DDoS attacks, rate limiting is applied at multiple levels:

Table 5.9: Rate Limiting Configuration

Endpoint	Max Requests	Time Window
All APIs	100 requests	Per 15 minutes
Login	5 attempts	Per 15 minutes
Registration	3 attempts	Per hour
Service request submission	5 requests	Per 10 minutes

8. Cascade Delete

When a craftsman is deleted by the admin, the deletion is executed inside a MongoDB Transaction to guarantee data consistency — the craftsman and all associated service requests are deleted atomically. After the transaction succeeds, image files are deleted from disk. If any step fails, the entire operation is rolled back.

5.3.8 Scalability and Future Considerations

The Forsa platform database was designed with future scalability in mind across several dimensions:

1. MongoDB Atlas — Infrastructure

By using MongoDB Atlas, scaling is managed at the infrastructure level. Atlas enables vertical scaling by upgrading the cluster size, and horizontal scaling through Sharding to distribute data across multiple servers when needed, without requiring any changes to the application code.

2. Replica Set

Atlas clusters operate natively as a Replica Set — a group of servers maintaining identical copies of the data. This provides High Availability: if one server fails, another automatically takes over. The Replica Set is also a prerequisite for using MongoDB Transactions, which the platform's cascade delete system relies on.

3. Schema Flexibility

Thanks to MongoDB's nature as a NoSQL database, new fields can be added to the craftsmen collection in the future (such as identity verification or payment options) without requiring a migration that affects existing records.

4. Suggested Future Improvements

Table 5.10: Suggested Future Improvements

Improvement	Reason
Implement the reviews system	The schema is ready (averageRating, ratingsCount) but the collection has not been implemented yet
Migrate images to a cloud service (Cloudinary / AWS S3)	Local storage is not suitable for distributed production environments
Add Redis as a cache layer	To reduce database load as the number of users grows
Replace in-memory rate limit store with Redis store	To ensure rate limiting effectiveness when deployed across multiple servers
Add OTP system for password reset	To enhance account security

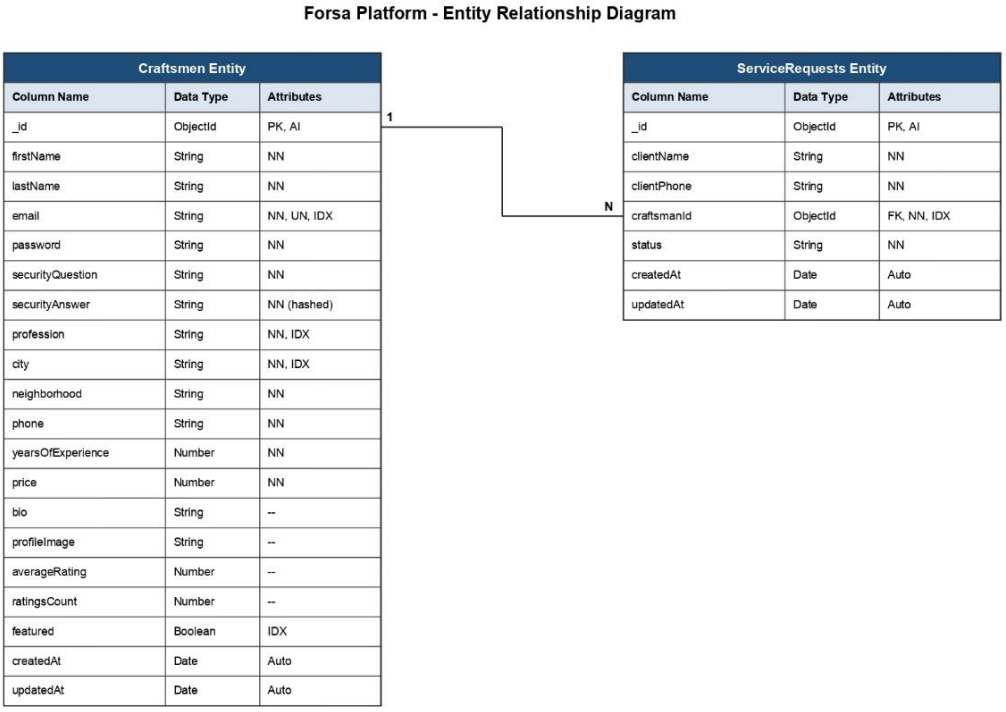


Figure 5.5: Entity-Relationship (ER) Diagram

5.4 Backend API Design

5.4.1 Craftsman Endpoints

Table 5.11: Register Craftsman Endpoint

Register Craftsman	
Purpose	Register a new craftsman on the platform along with portfolio images
Endpoint	/api/craftsmen/register
Method	POST
Header	Content-Type: multipart/form-data
Authentication	Not required
Body	firstName, lastName: STRING — required; email: STRING (Gmail only) — required; password: STRING (min 8) — required; securityQuestion, securityAnswer — required; profession, neighborhood: STRING — required; city: STRING (enum) — required; phone: STRING (059XXXXXXX) — required; yearsOfExperience: NUMBER (≥0); price: NUMBER (≥1); bio: STRING — optional; workImages: FILE[] — required, exactly 3 files, max 2MB each

Register Craftsman	
Response	{ status: { status: true, message: "Craftsman registered successfully" }, data: { token: "<JWT — 7 days>", craftsman: { ...all fields except password } } }
Error	400 — Missing fields / invalid format / not exactly 3 images; 400 — Email already registered; 500 — Server error

Table 5.12: Login Craftsman Endpoint

Login Craftsman	
Purpose	Authenticate a registered craftsman and return a JWT token
Endpoint	/api/craftsmen/login
Method	POST
Header	Content-Type: application/json
Authentication	Not required
Rate Limit	5 attempts / 15 minutes
Body	email: STRING — required; password: STRING — required
Response	{ status: { status: true, message: "Login successful" }, data: { token: "<JWT — 7 days>", craftsman: { ...all fields except password } } }
Error	400 — Wrong email or password (intentionally vague to prevent email enumeration); 500 — Server error

Table 5.13: Get My Profile Endpoint

Get My Profile	
Purpose	Retrieve the authenticated craftsman's own profile data
Endpoint	/api/craftsmen/me
Method	GET
Header	Authorization: Bearer <token>
Authentication	Required
Response	{ status: { status: true, message: "..." }, data: { craftsman: { ...all fields except password } } }
Error	401 — Missing or invalid token; 404 — Craftsman not found; 500 — Server error

Table 5.14: Update My Profile Endpoint

Update My Profile	
Purpose	Update the authenticated craftsman's profile data and/or images
Endpoint	/api/craftsmen/me
Method	PATCH
Header	Authorization: Bearer <token>; Content-Type: multipart/form-data
Authentication	Required
Body	All fields optional: firstName, lastName, profession, city, neighborhood, phone, bio: STRING; yearsOfExperience, price: NUMBER; profileImage: FILE (max 1) — replaces old image; workImages: FILE[] (max 9) — appended to existing, total must not exceed 12
Response	{ status: { status: true, message: "..." }, data: { craftsman: { ...updated fields except password } } }
Error	401 — No valid token; 400 — Work images would exceed 12 total; 404 — Craftsman not found; 500 — Server error

Table 5.15: Get Featured Craftsmen Endpoint

Get Featured Craftsmen	
Purpose	Retrieve craftsmen flagged as featured for display on the homepage
Endpoint	/api/craftsmen/featured
Method	GET
Authentication	Not required
Response	{ status: { status: true, message: "..." }, data: { craftsmen: [...sorted by createdAt descending] } }
Error	500 — Server error

Table 5.16: Get All Craftsmen Endpoint

Get All Craftsmen	
Purpose	Retrieve a list of craftsmen with optional filtering, search, and pagination
Endpoint	/api/craftsmen
Method	GET
Authentication	Not required

Get All Craftsmen	
Query	profession: STRING — filter by profession; city: STRING — filter by city; search: STRING — search across name, profession, city, neighborhood; page: NUMBER (default: 1); limit: NUMBER (default: 20, max: 100)
Response	{ status: { status: true, message: "..." }, data: { craftsmen: [...], pagination: { page, limit, total, totalPages } } }
Error	500 — Server error

Table 5.17: Get Craftsman By ID Endpoint

Get Craftsman By ID	
Purpose	Retrieve a single craftsman's full profile by MongoDB ObjectId
Endpoint	/api/craftsmen/:id
Method	GET
Authentication	Not required
Parameters	id: MongoDB ObjectId — required
Response	{ status: { status: true, message: "..." }, data: { craftsman: { ...all fields except password } } }
Error	404 — Craftsman not found; 500 — Server error

5.4.2 Service Request Endpoints

Table 5.18: Create Service Request Endpoint

Create Service Request	
Purpose	Submit a service request to a specific craftsman — available to guests without registration
Endpoint	/api/service-requests
Method	POST
Header	Content-Type: application/json
Authentication	Not required
Rate Limit	5 requests / 10 minutes
Body	clientName: STRING — required; clientPhone: STRING — required, 10–13 digits, Arabic-Eastern digits normalized automatically; craftsmanId: ObjectId — required, must reference an existing craftsman; jobDetails: STRING — optional

Create Service Request	
Response	{ status: { status: true, message: "Service request created successfully" }, data: { serviceRequest: {...}, craftsmanPhone: "string" } }
Error	400 — Missing clientName or invalid clientPhone; 400 — Invalid craftsmanId format; 400 — Craftsman cannot request their own service; 404 — Craftsman not found; 500 — Server error

Table 5.19: Get My Service Requests Endpoint

Get My Service Requests	
Purpose	Retrieve all service requests received by the authenticated craftsman
Endpoint	/api/service-requests/me
Method	GET
Header	Authorization: Bearer <token>
Authentication	Required
Query	page: NUMBER (default: 1); limit: NUMBER (default: 20, max: 100)
Response	{ status: { status: true, message: "..." }, data: { serviceRequests: [...sorted by createdAt descending], pagination: {...} } }
Error	401 — Unauthorized; 500 — Server error

Table 5.20: Update Service Request Status Endpoint

Update Service Request Status	
Purpose	Update the status of a service request — craftsman can only update their own requests
Endpoint	/api/service-requests/:requestId/status
Method	PATCH
Header	Authorization: Bearer <token>; Content-Type: application/json
Authentication	Required
Parameters	requestId: MongoDB ObjectId — required
Body	status: STRING — required; allowed: pending, confirmed, contacted, completed, cancelled
Response	{ status: { status: true, message: "..." }, data: { serviceRequest: { ...updated request } } }
Error	401 — Unauthorized; 400 — Invalid status transition; 400 — Invalid requestId format; 404 — Request not found or does not belong to this craftsman; 500 — Server error

Status transitions are governed by a state machine to ensure data integrity. The following table defines the allowed transitions:

Table 5.21: Service Request Status Transitions

From	Allowed Transitions
pending	confirmed, contacted, cancelled
confirmed	contacted, completed, cancelled
contacted	completed, cancelled
completed	— (final state)
cancelled	— (final state)

Table 5.22: Delete Profile Image Endpoint

Delete Profile Image	
Purpose	Delete the logged-in craftsman's profile image from the server and database
Endpoint	/api/craftsmen/me/profile-image
Method	DELETE
Header	Authorization: Bearer <JWT>
Authentication	Required (JWT token)
Response	{ status: { status: true, message: "Profile image deleted successfully" }, data: { ...craftsmanObject } }
Error	401 — Invalid or missing token; 400 — No profile image to delete; 404 — Craftsman not found; 500 — Server error

5.4.3 Admin Endpoints

Table 5.23: Admin Login Endpoint

Admin Login	
Purpose	Authenticate the admin and return a JWT token
Endpoint	/api/admin/login
Method	POST
Header	Content-Type: application/json
Authentication	Not required
Rate Limit	5 attempts / 15 minutes

Admin Login	
Body	email: STRING — required; password: STRING — required
Response	{ status: { status: true, message: "Login successful" }, data: { token: "<JWT — 1 day>" } }
Error	401 — Invalid credentials (intentionally vague); 500 — Server error

Table 5.24: Get All Craftsmen (Admin) Endpoint

Get All Craftsmen (Admin)	
Purpose	Retrieve a complete list of all craftsmen for the admin dashboard
Endpoint	/api/admin/craftsmen/
Method	GET
Header	Authorization: Bearer <adminToken>
Authentication	Required — Admin JWT
Query	page: NUMBER (default: 1); limit: NUMBER (default: 20, max: 100)
Response	{ status: { status: true, message: "..." }, data: { craftsmen: [...without passwords], pagination: {...} } }
Error	401 — Missing or invalid token; 403 — Valid token but not admin; 500 — Server error

Table 5.25: Toggle Featured Status Endpoint

Toggle Featured Status	
Purpose	Enable or disable a craftsman's featured status on the homepage
Endpoint	/api/admin/craftsmen/:id/featured
Method	PATCH
Header	Authorization: Bearer <adminToken>; Content-Type: application/json
Authentication	Required — Admin JWT
Parameters	id: MongoDB ObjectId — required
Body	featured: BOOLEAN — required, must be strict boolean (true/false)
Response	{ status: { status: true, message: "..." }, data: { id: "...", featured: true } }
Error	401 — Missing or invalid token; 403 — Not admin; 400 — featured is not a boolean; 404 — Craftsman not found; 500 — Server error

Table 5.26: Delete Craftsman (Admin) Endpoint

Delete Craftsman	
Purpose	Delete a craftsman along with all associated data (Cascade Delete)
Endpoint	/api/admin/craftsmen/:id
Method	DELETE
Header	Authorization: Bearer <adminToken>
Authentication	Required — Admin JWT
Parameters	id: MongoDB ObjectId — required
Response	{ status: { status: true, message: "Craftsman and related data deleted successfully" }, data: { deletedCraftsmanId, deletedServiceRequestsCount, deletedFilesCount } }
Error	401 — Missing or invalid token; 403 — Not admin; 404 — Craftsman not found; 500 — Server error

5.5 UX/UI Design

5.5.1 Persona Creation

The Forsa platform targets two primary user types: the client, who seeks a reliable local craftsman, and the craftsman, who wants to showcase their skills and receive service requests. Both personas share a common frustration — the lack of a single trusted source for local craft services in Gaza. Forsa directly addresses this gap by providing a centralized, profile-based platform for discovery and direct contact

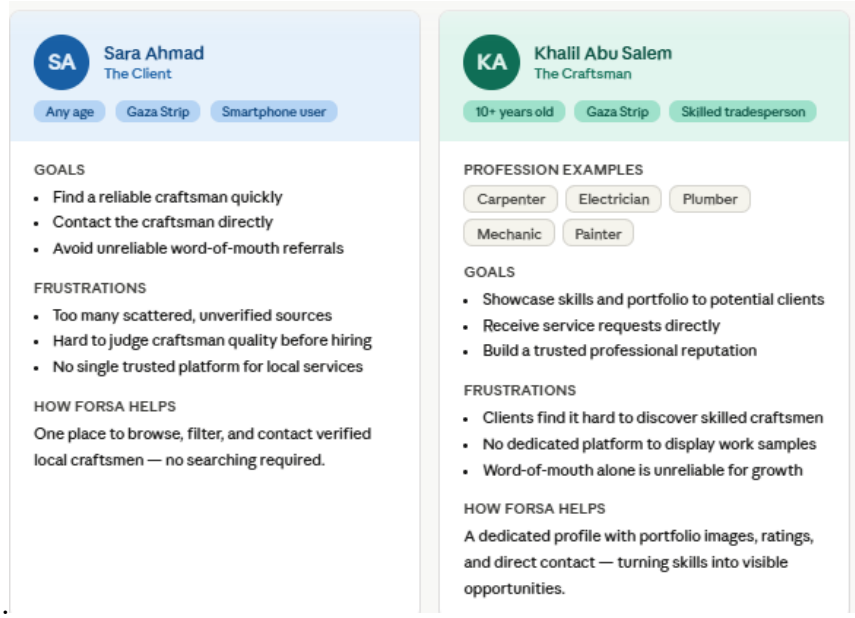


Figure 5.6 Persona Creation

5.5.2 Competitor Analysis

A comparative analysis was conducted against four existing platforms operating in similar domains: Taskty, Fiverr, Thumbtack, and Khidmah. The analysis reveals that no existing platform is specifically designed for the Gaza context. Forsa differentiates itself through Gaza-specific location filtering, direct WhatsApp contact without in-platform payment requirements, primary Arabic language support, and lightweight registration requiring only a name and phone number — features tailored to the local infrastructure and user behavior.

Table 5.27 Competitor Analysis

Feature / Criterion	Forsa (Us)	Taskty	Fiverr	Thumbtack	Khidmah
Target market	**Gaza, Palestine**	Arab region	Global	USA / Canada	Gulf region
Browse without login	✓ **Full browsing**	◦ Limited	✓	✗ Login required	◦ Limited
Filter by location	✓ **Within Gaza**	✓	✗	✓	✓
Filter by craft / skill	✓	✓	✓	✓	✓
Filter by budget	✓	◦ Partial	✓	✓	◦ Partial
Craftsman portfolio	✓	✓	✓	◦ Limited	✓
Contact via WhatsApp	✓ **Direct**	✗	✗	✗	✗
In-platform payment	✗ **Not needed**	✓	✓	✓	✓
Craftsman self-manage profile	✓	✓	✓	✓	✓
Arabic language support	✓ **Primary**	✓	◦ Partial	✗	✓
Lightweight registration	✓ **Name + phone only**	◦ Full form	✗ Full account	✗ Full account	◦ Full form
Designed for low-resource context	✓ **Gaza-specific**	✗	✗	✗	✗

5.5.3 User Journey / Flow

The user journey maps all possible paths through the Forsa platform for three actor types: the guest client, the registered craftsman, and the admin. A guest can browse craftsmen by search or trade category, view craftsman details, and submit a service request — all without registration. A craftsman can create an account, log in, manage their profile, and track incoming service requests. The admin accesses a dedicated dashboard to manage craftsmen and platform content.

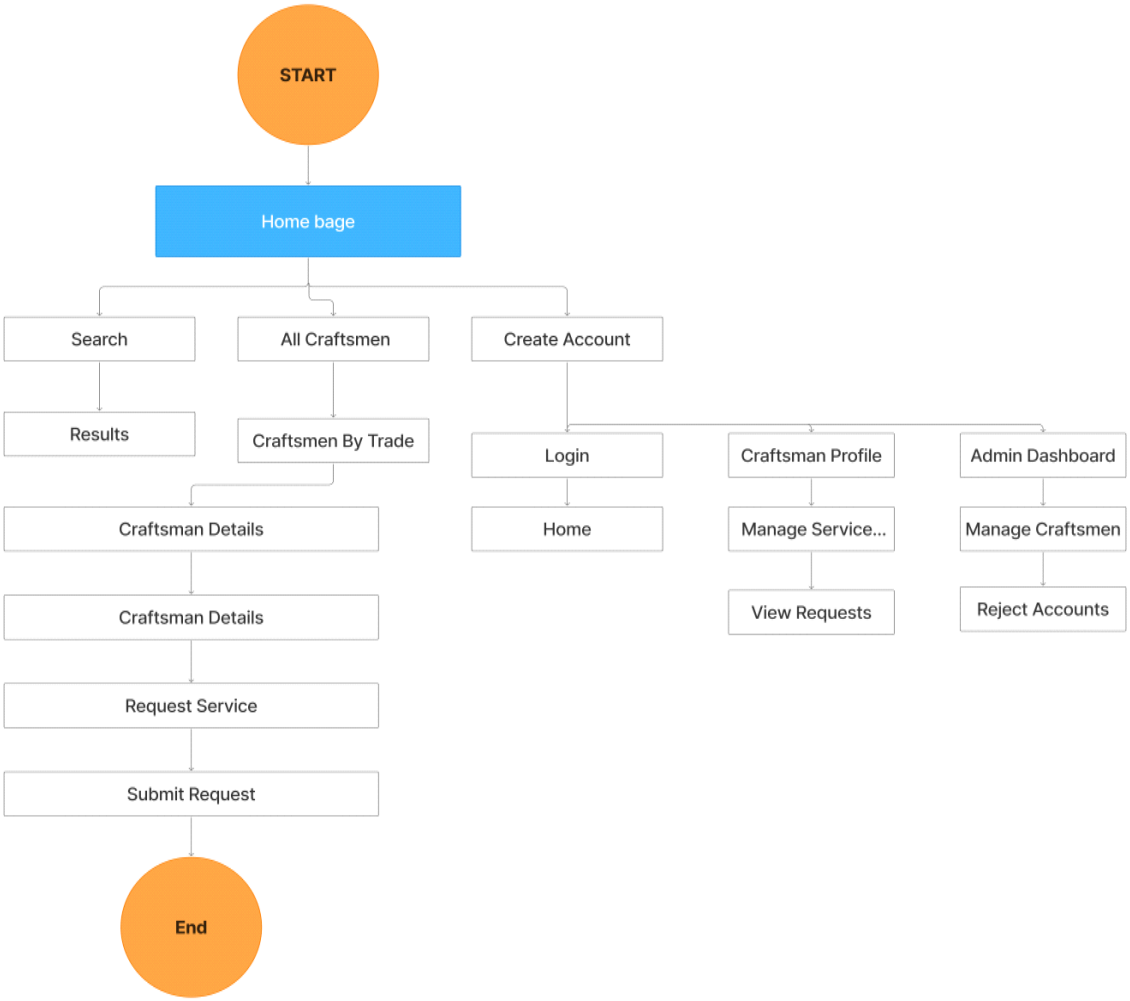


Figure 5.8: Comprehensive End-to-End User Flow

5.5.4 Sketching (Low-Fidelity Design)

Initial low-fidelity sketches were drawn by hand to explore the layout and structure of the main screens before moving to digital design. The sketches cover the core user flows including the home page, craftsman listing, craftsman profile, service request form, and craftsman dashboard. This stage focused purely on content hierarchy and navigation structure, intentionally ignoring colors, typography, and visual details.

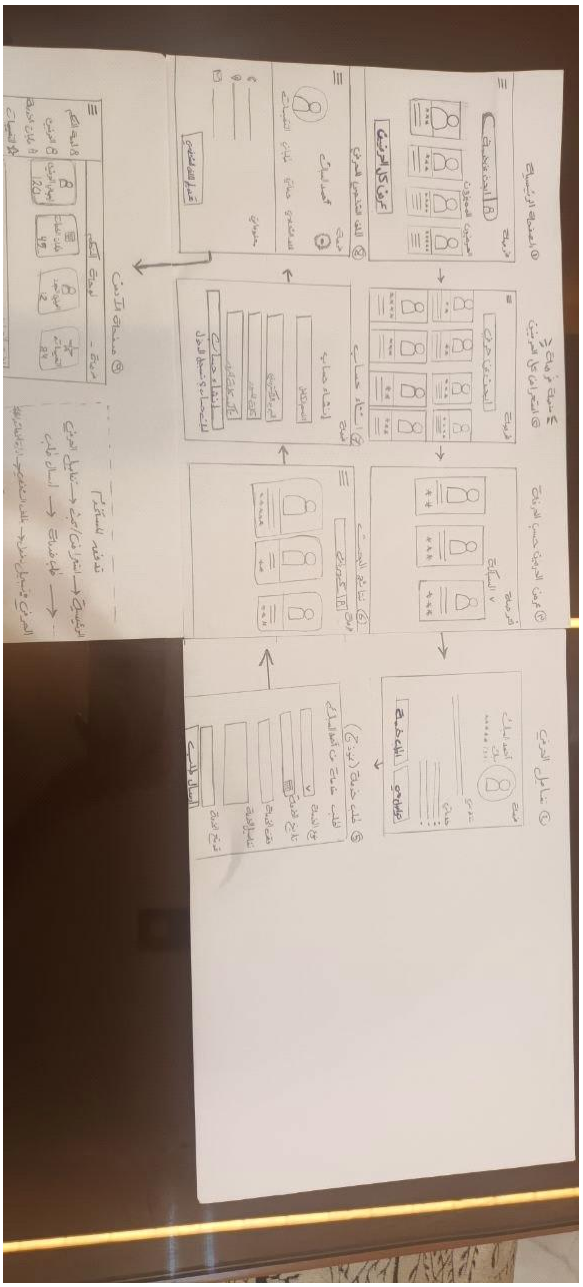


Figure 5.9: Low-Fidelity Layout Sketches

5.5.5 Wireframes

Low-fidelity wireframe structures establish strict, un-stylized UI grids to ensure digital layouts align properly across mobile viewports.

5.5.5.1 Home Page

The home page wireframe defines the layout of the hero section, featured craftsmen cards, trade category filters, and the craftsman registration call-to-action banner.

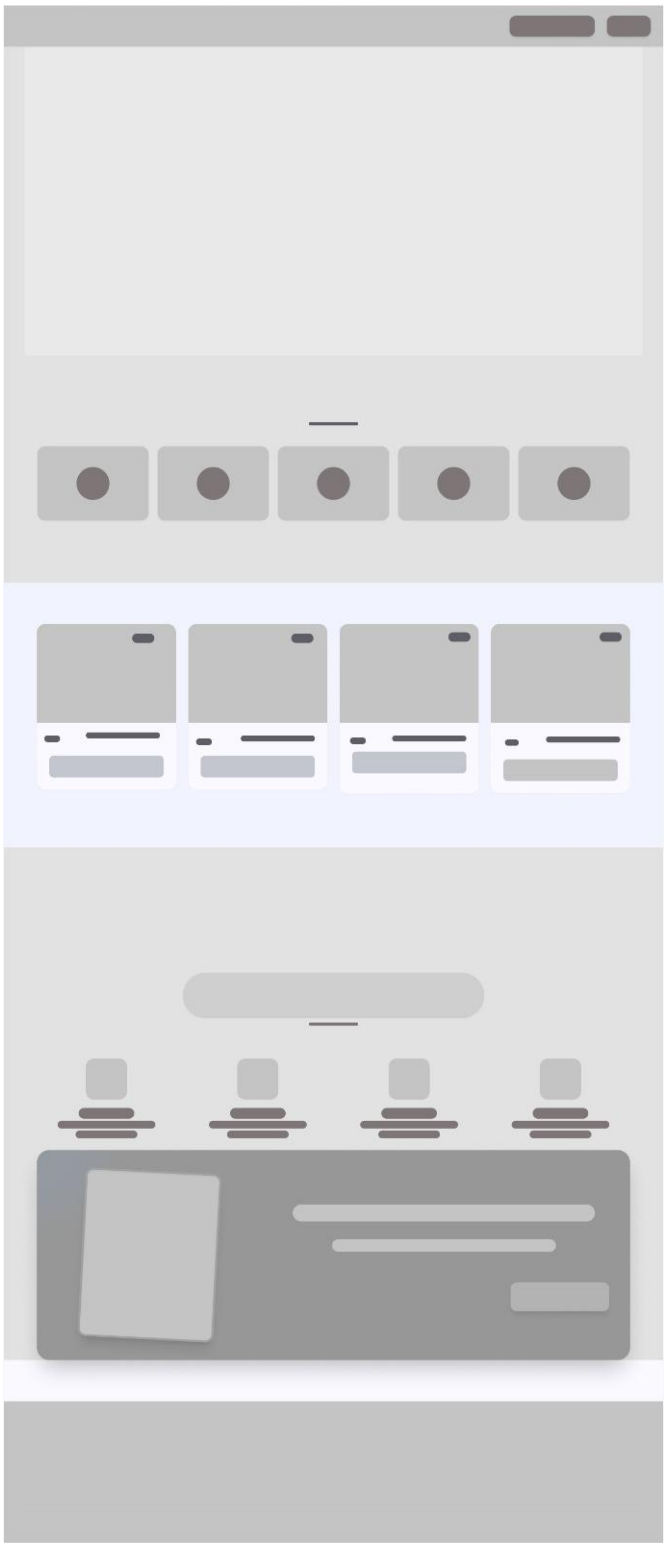


Figure 5.10: home page wireframe

5.5.5.2 Log in

The login wireframe defines the placement of the email and password input fields, the login button, and the social login options.

Figure 5.11: login page wireframe

5.5.5.3 Craftsman details

The craftsman details wireframe defines the layout of the profile header, skills and bio section, portfolio image gallery, and service request button.

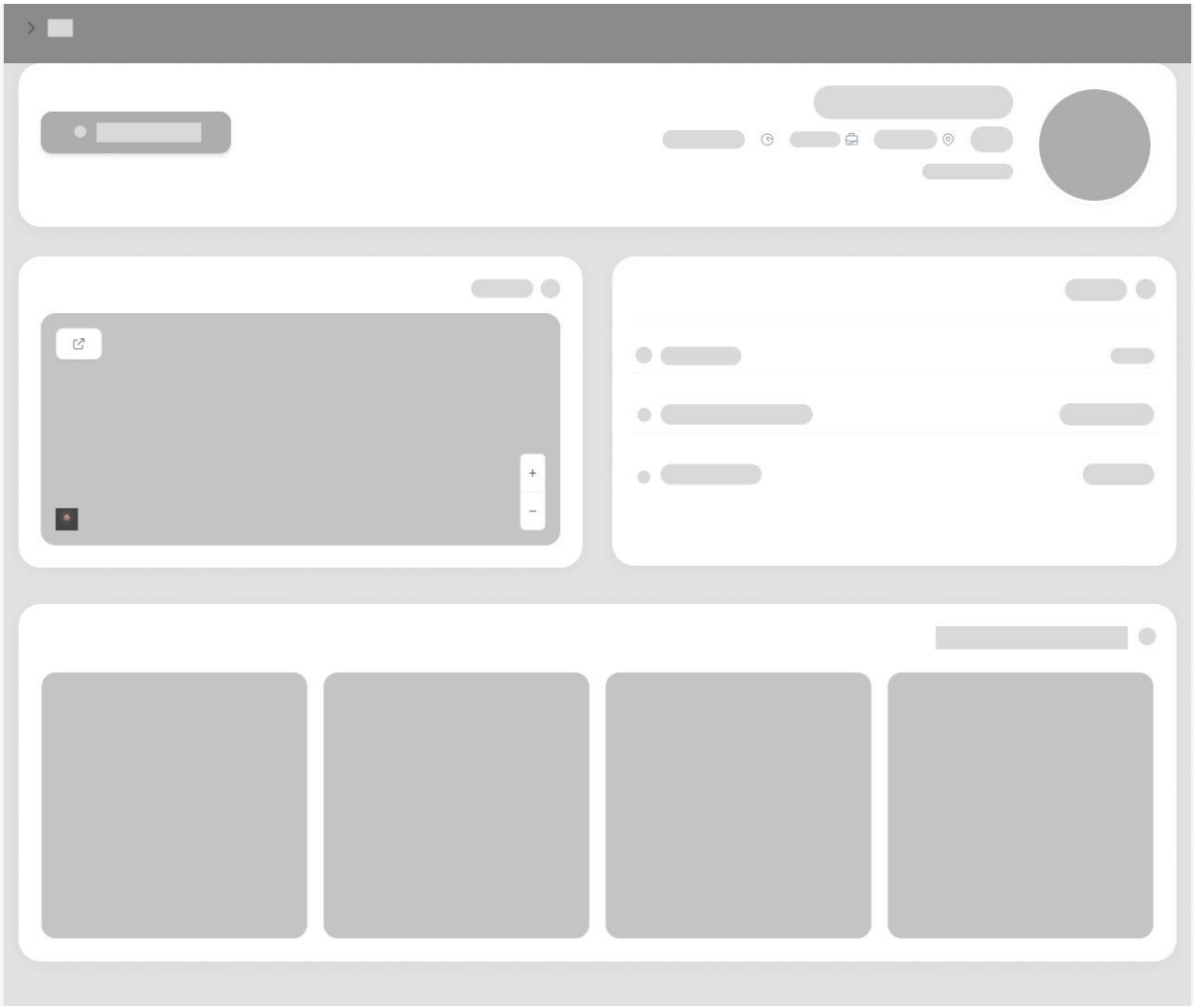


Figure 5.12: craftman details page wireframe

5.5.5.4 Service Request Modal

The service request wireframe defines the modal layout containing the client name, phone number, job details input fields, and the submit button.

Figure 5.13 service request modal wireframe

5.5.6 UI Visual Design

The final user interface design applies accessible typography and consistent visual cues to ensure a clear, modern presentation.

5.5.6.1 Home Page

The home page introduces the platform with a hero section, a list of featured craftsmen, and a call-to-action for craftsmen to join the platform.

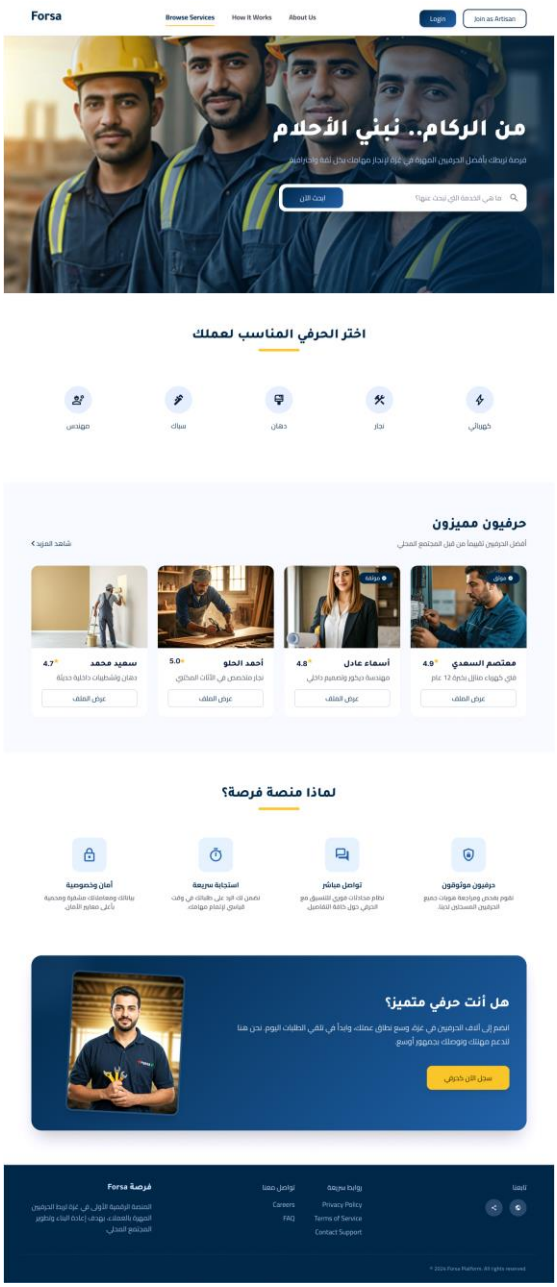


Figure 5.14 Home Page

5.5.6.2 Who are we?

This section explains the platform's mission and the value it provides to both clients and craftsmen in Gaza.

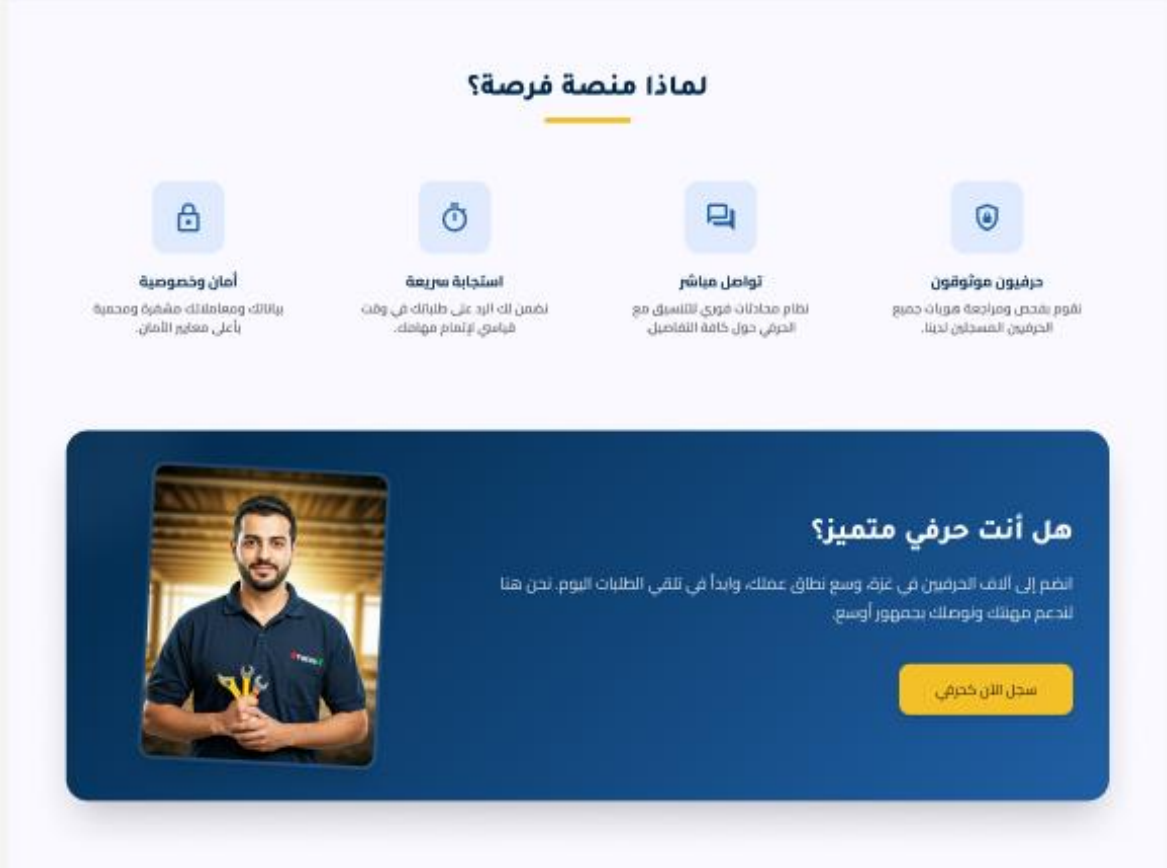


Figure 5.15 who are we

5.5.6.3 Create an account

The registration page allows new craftsmen to create an account by entering their personal and professional information.

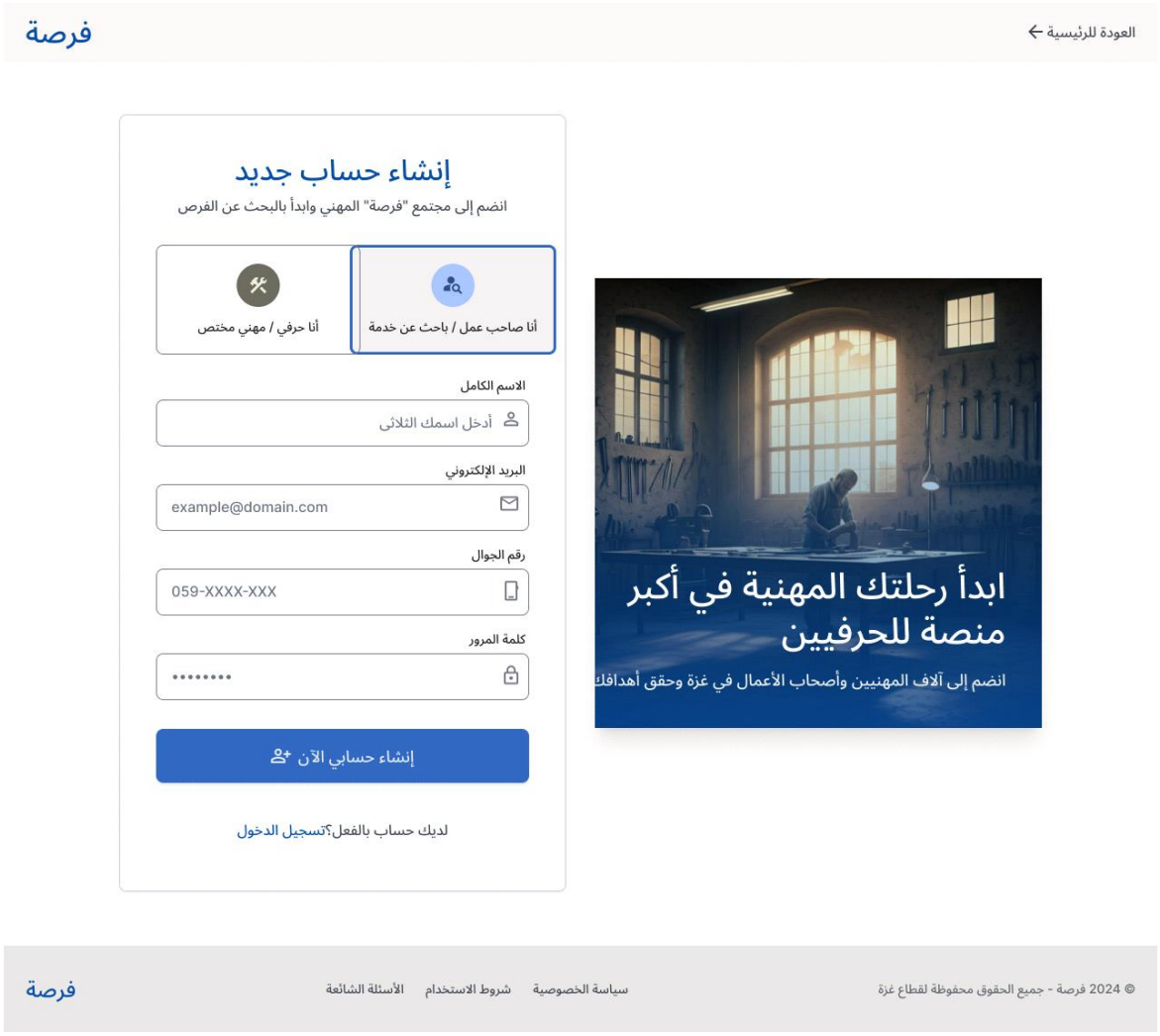


Figure 5.16 create account

5.5.6.4 Log in

The login page allows registered craftsmen to access their accounts using their email and password.

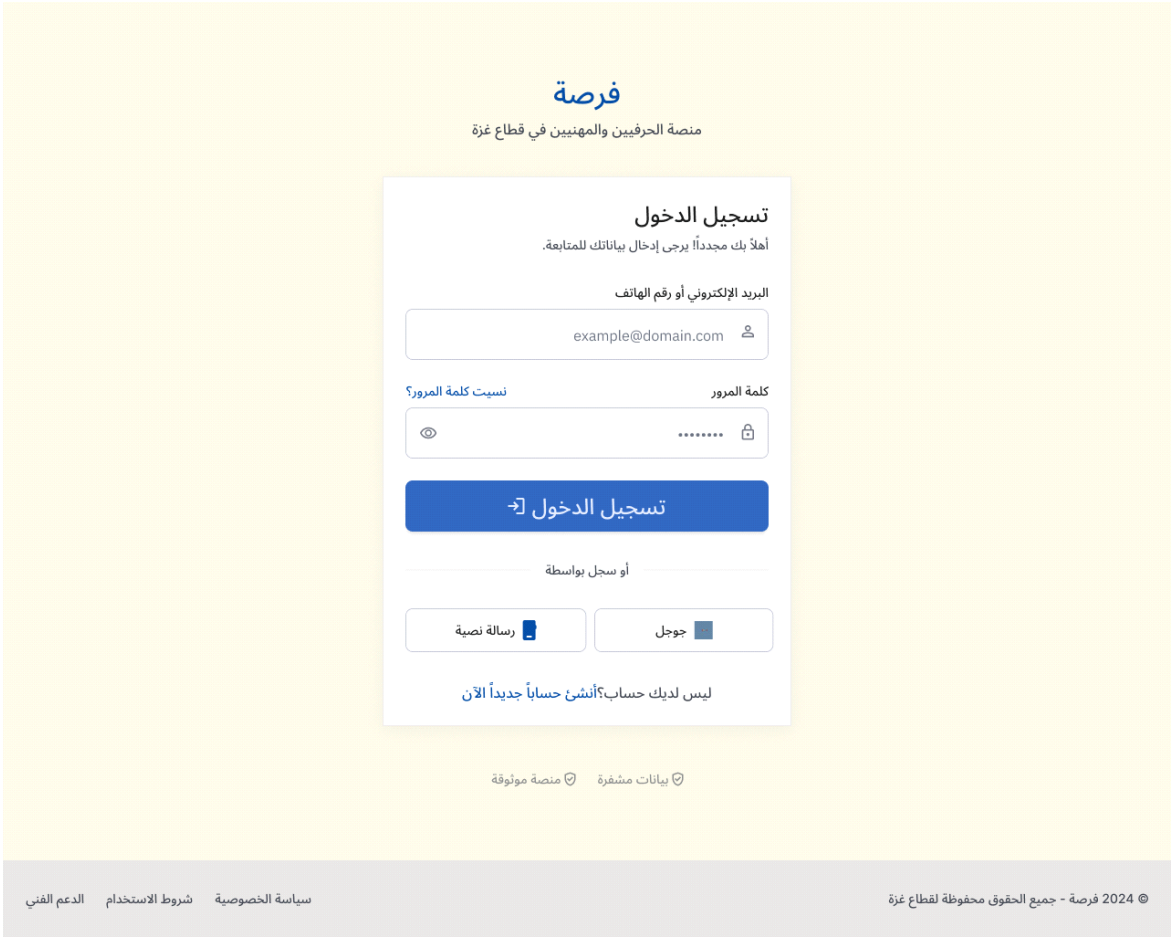


Figure 5.17 log in

5.5.6.5 Craftsman's File

The craftsman profile page displays full details including bio, skills, portfolio images, ratings, and a direct contact button.

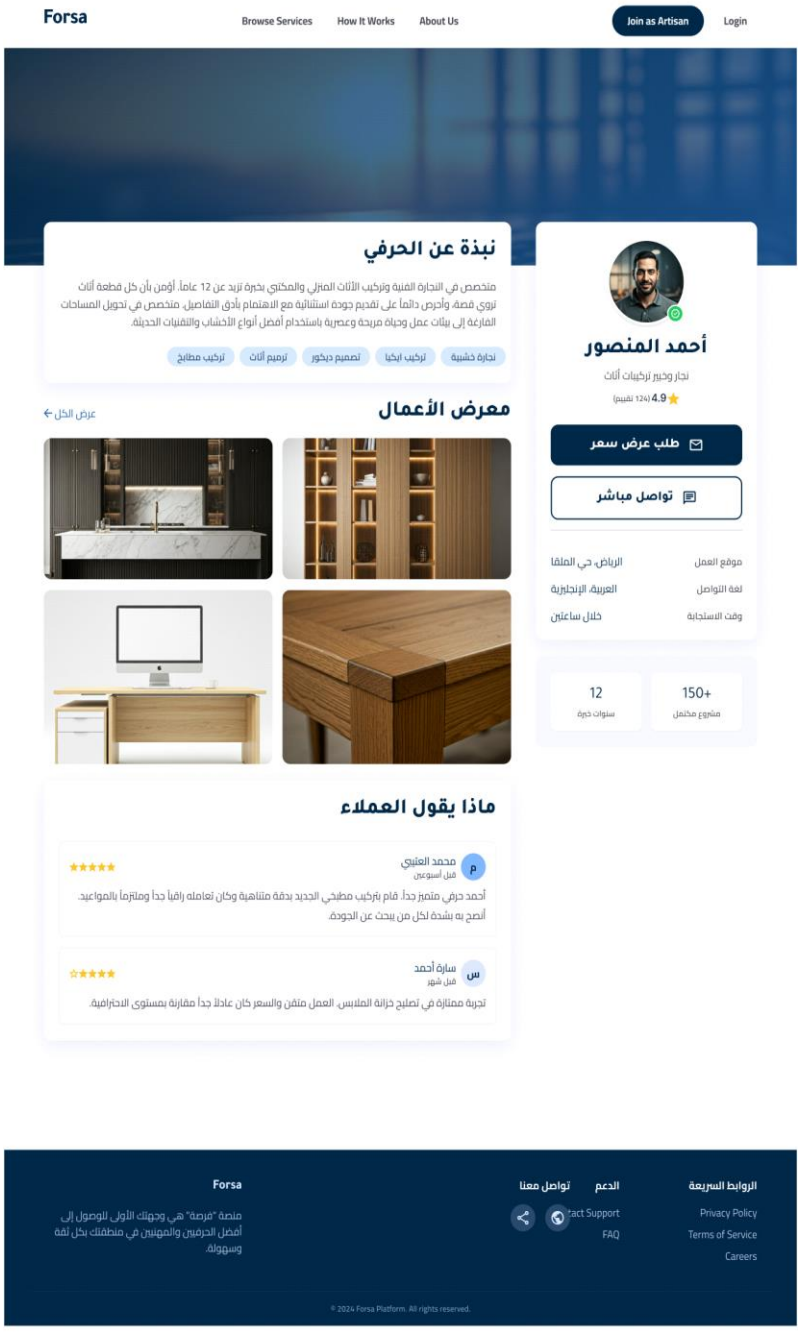


Figure 5.18 Craftsman's File

5.5.6.6 Browse the craftsmen

The browse page allows clients to discover craftsmen with filtering options by profession, city, and rating.

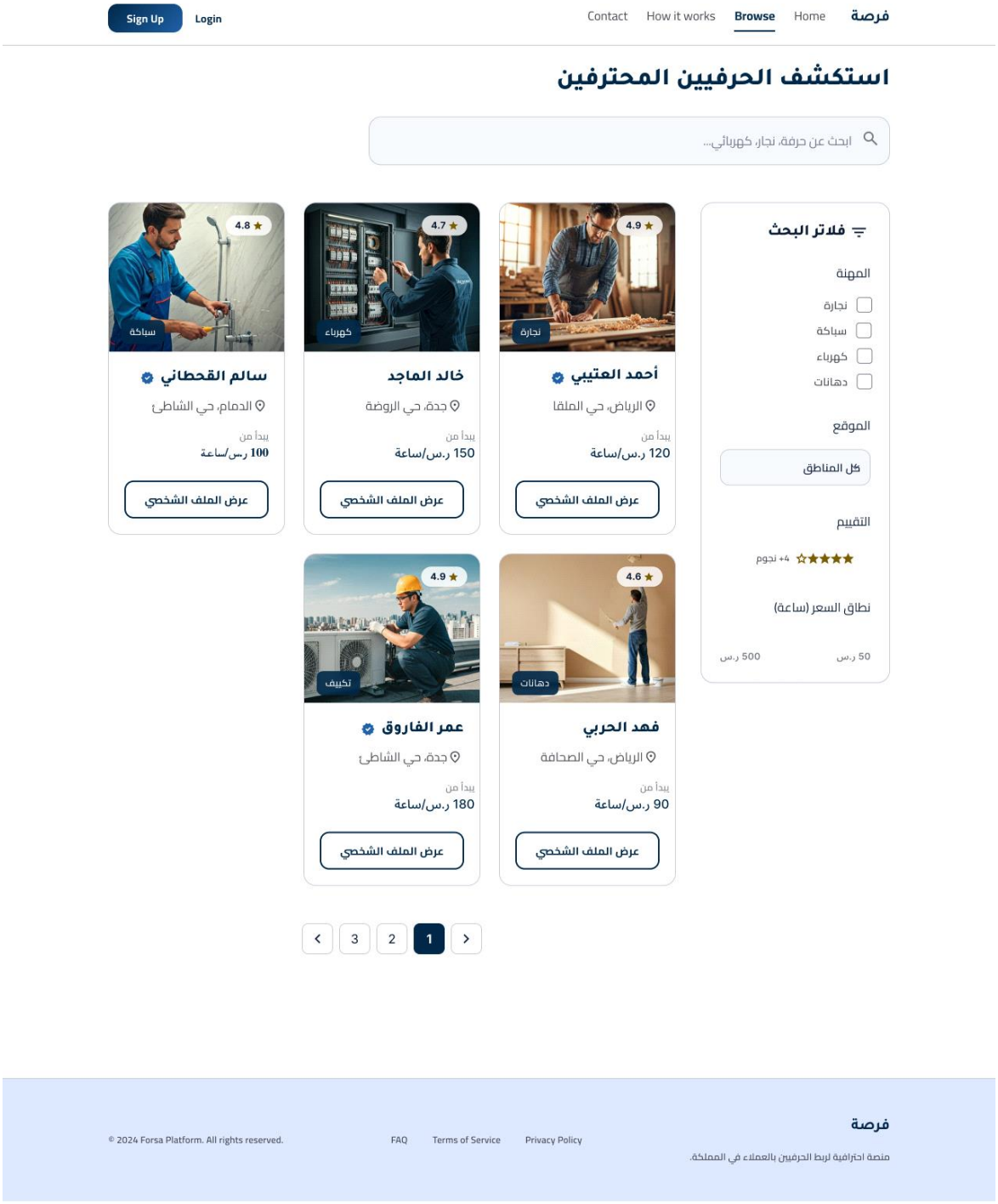


Figure 5.19 browse the craftmen

5.5.6.7 Admin Management Panel

The admin dashboard provides a complete list of registered craftsmen with options to manage their featured status and delete accounts.

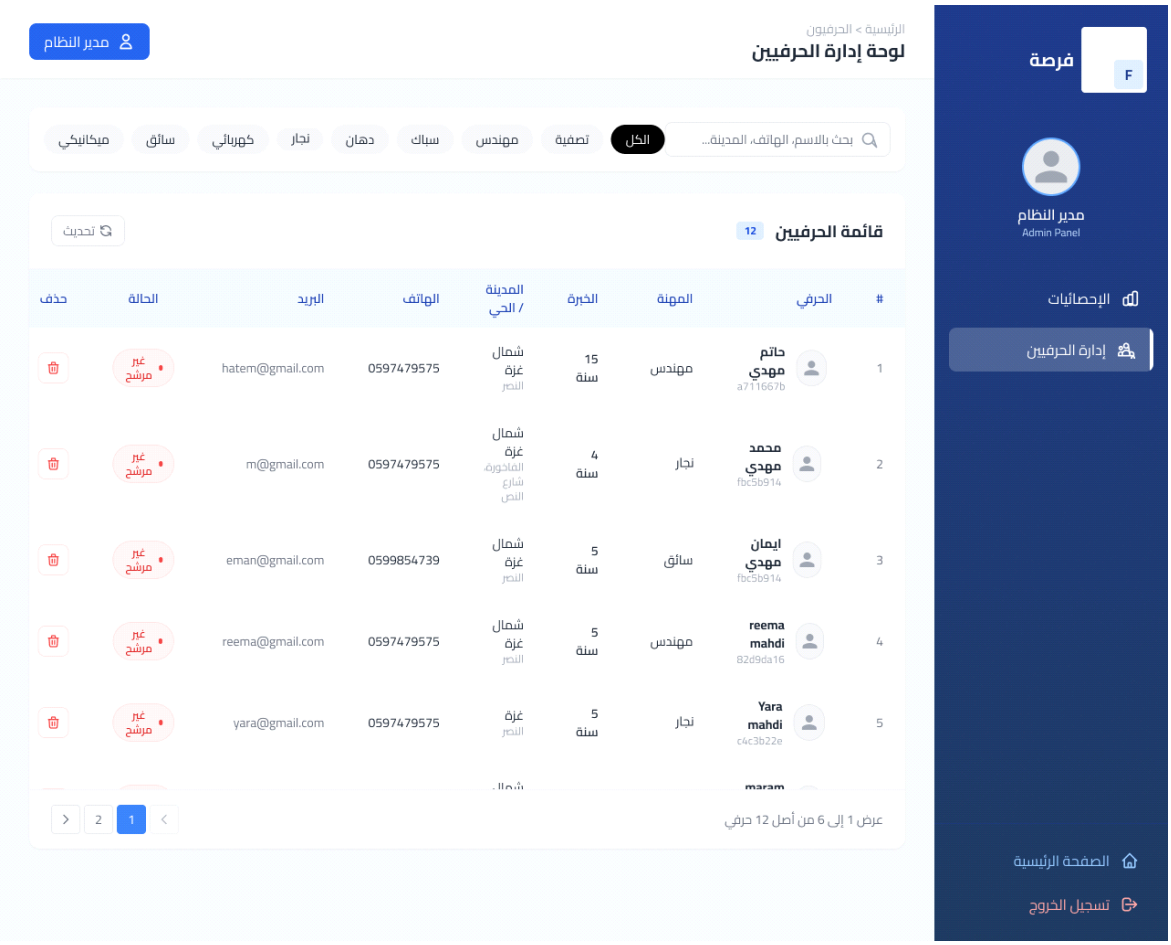


Figure 5.20 Admin Management Panel

5.5.6.8 Management Statistics and Reports

The statistics page displays key platform metrics including total craftsmen, profession distribution, and city distribution.

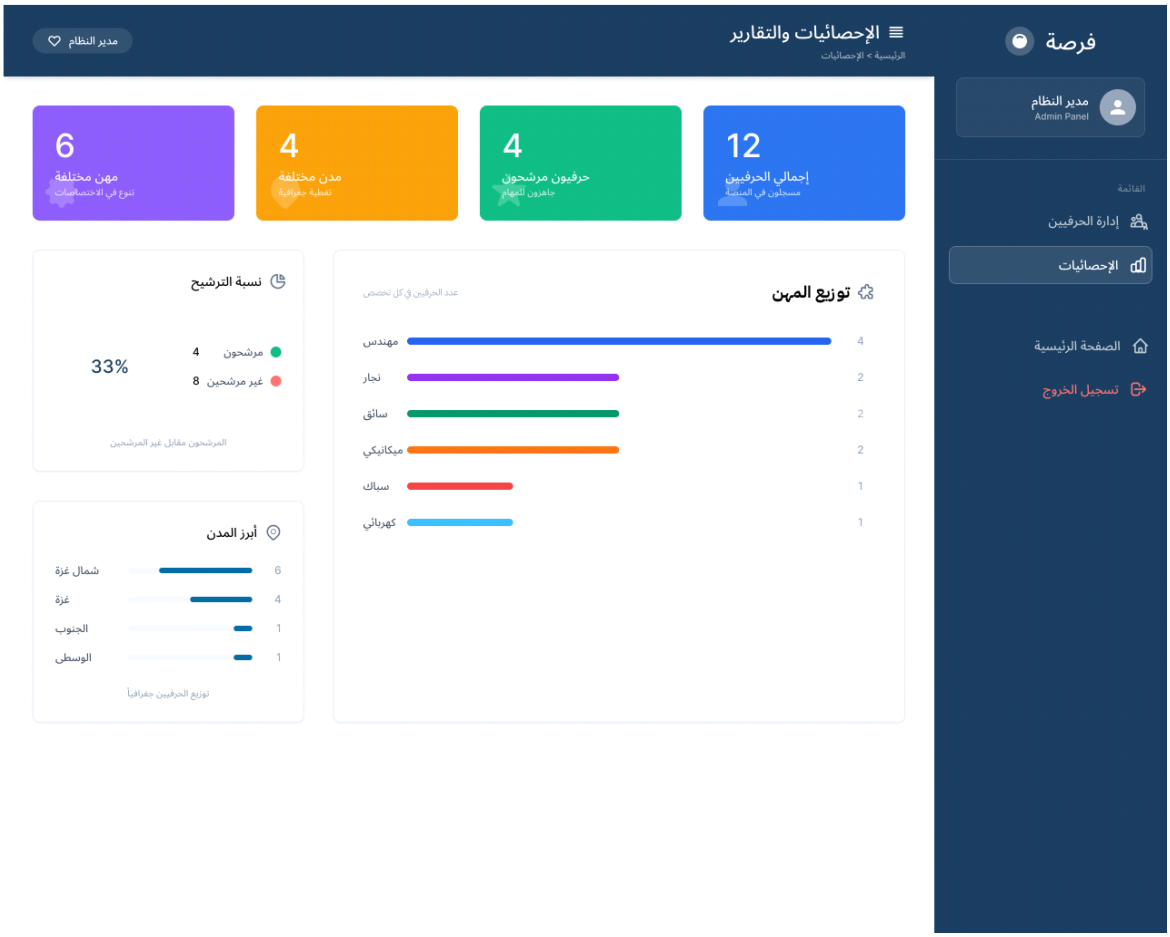


Figure 5.21 Management Statistics and Reports

5.5.6.9 Change Password

The change password page allows craftsmen to update their password by entering their email and new password.

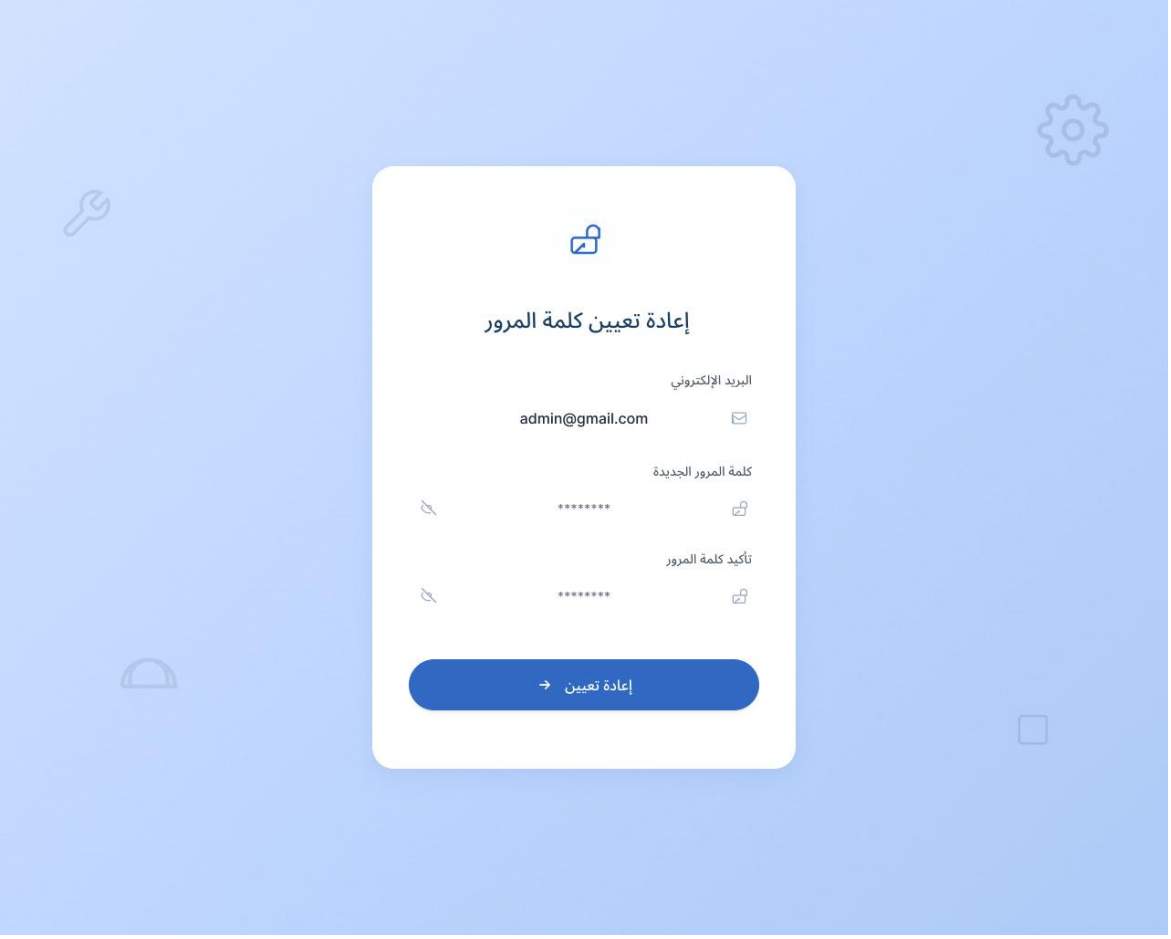


Figure 5.22 change password

5.5.7 Prototyping

An interactive prototype was developed using Figma to simulate the complete user flows across all platform screens before development began. As shown in Figure 5.X, the prototype maps the navigation connections between all screens including the home page, craftsman registration, login, craftsman profile, browse all craftsmen, craftsman dashboard, and statistics page.
The prototype demonstrates screen transitions and user interactions, allowing the team to validate the user experience and navigation logic before implementation. The full interactive prototype is accessible at:
https://www.figma.com/proto/XpacY33Jqy9V12FX4QT0RM/Untitled?node-id=120-2&t=pt4utN5V8Mzcp8P0-1

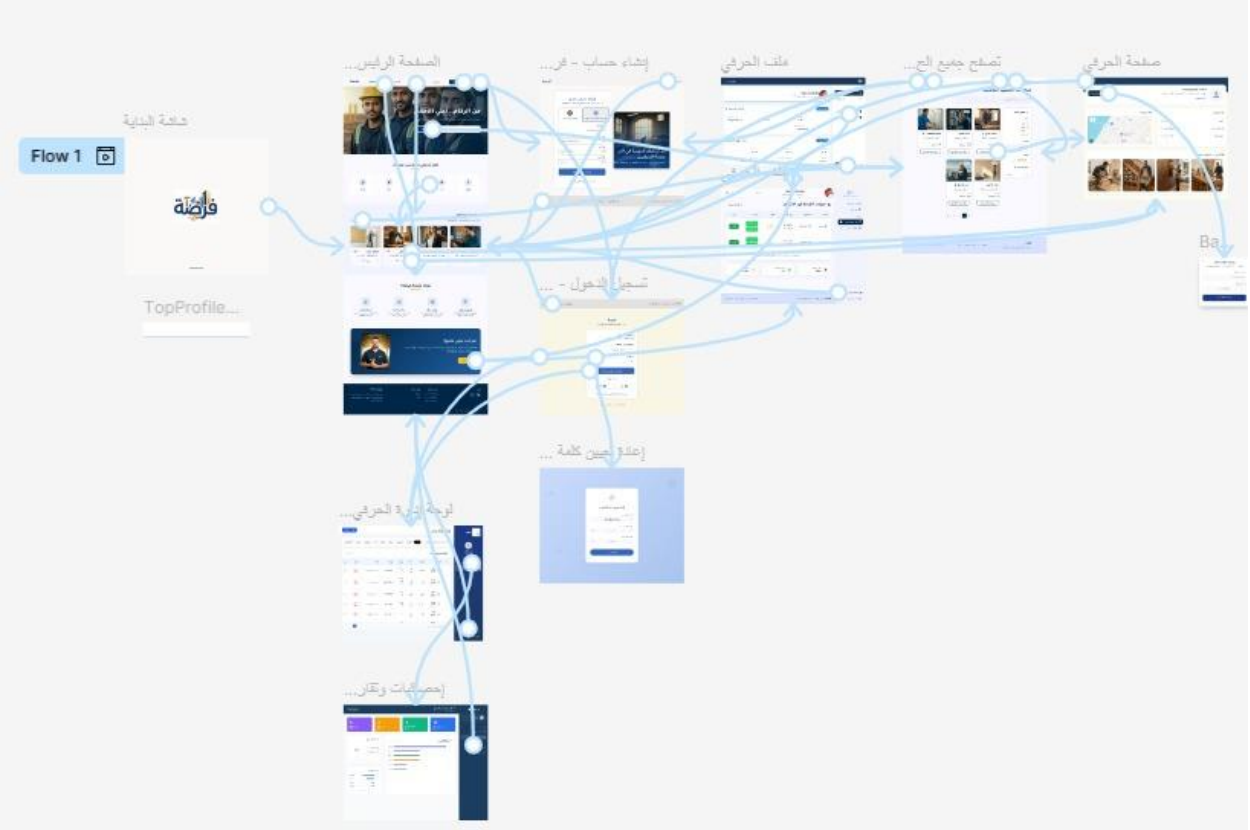


Figure 5.23 prototype

5.5.8 Usability Considerations

The Forsa platform was designed with usability as a core priority, guided by the needs of users in the Gaza context. Navigation is kept simple and intuitive, with clear category filters and a prominent search bar accessible from every page. The interface uses Arabic as the primary language to ensure accessibility for all local users. Buttons and call-to-action elements are clearly labeled and consistently placed across all screens. The design is fully responsive, adapting to both desktop and mobile viewports to accommodate users accessing the platform from different devices. Color contrast and font sizes were chosen to ensure readability, and the overall layout minimizes the number of steps required to find a craftsman and submit a service request.

Chapter 6
Implementation

Chapter 6
System Implementation

This chapter includes the implementation results based on the system description in the previous chapter.

6.1 Architecture

The Forsa platform consists of two main components: a React-based frontend and a Node.js/Express backend API. Each component follows a distinct architectural pattern suited to its role in the system.

6.1.1 Frontend Architecture

The frontend is built using React and follows a Component-Based Architecture, which is the natural pattern for React applications. Each part of the user interface is broken down into independent, reusable components, making the codebase easier to maintain and extend.

The project is organized into two main layers: pages, which represent full screens such as the homepage, craftsman profile, and dashboard, and components, which are shared UI elements reused across multiple pages such as the Navbar, Hero section, and Footer.

For state management, React Context API was used instead of heavier libraries such as Redux. This decision was justified by the nature of the shared state in the application, which is limited to the interface language (Arabic/English) and the user session data. Using Redux would have introduced unnecessary complexity for this scope, whereas Context API is specifically designed for this type of lightweight global state.

6.1.2 Backend Architecture

The backend is built with Node.js and Express.js and follows the MVC (Model–View–Controller) pattern. This pattern was chosen to enforce a clear separation of concerns between data, logic, and routing. Since the backend functions purely as a REST API, the View layer is replaced by structured JSON responses returned directly from the controllers.

The three layers are applied as follows:

Model — Defined using Mongoose, the models represent the core data entities of the system: Craftsman, ServiceRequest, and Review. Each model includes built-in validation rules such as email format

restrictions, phone number patterns, and enumerated city values to ensure data consistency at the database level.

Controller — The controllers contain all business logic of the application. The system includes three main controllers: craftsman.controller.js, which handles registration, authentication, search, and profile management; serviceRequest.controller.js, which manages the creation, retrieval, and status updates of service requests; and admin.controller.js, which handles admin authentication and craftsman moderation.

Routes — The routes layer defines all API endpoints and maps each one to its corresponding controller function. All routes are prefixed under /api/ and are mounted in the main app.js file.

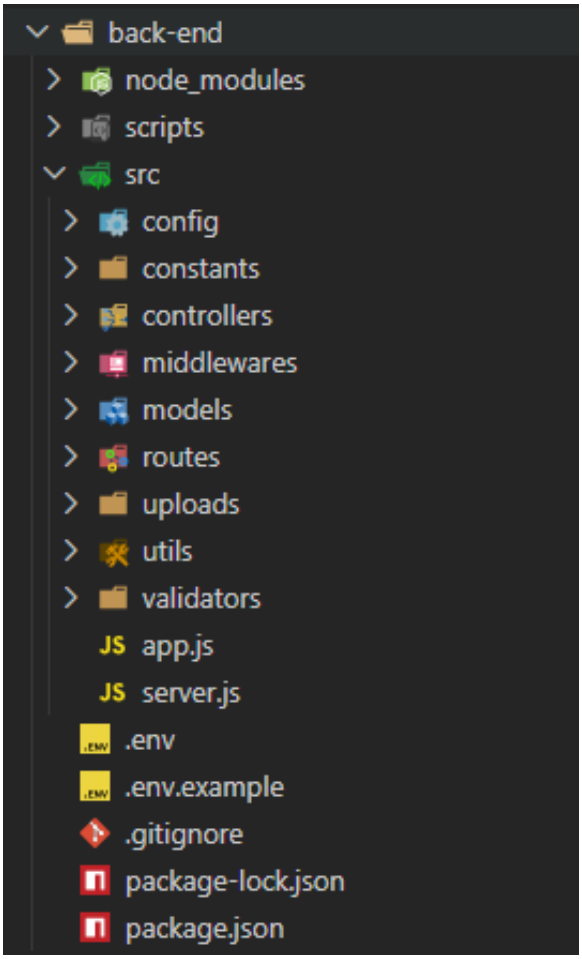


Figure 6.1 Backend Folder Structure

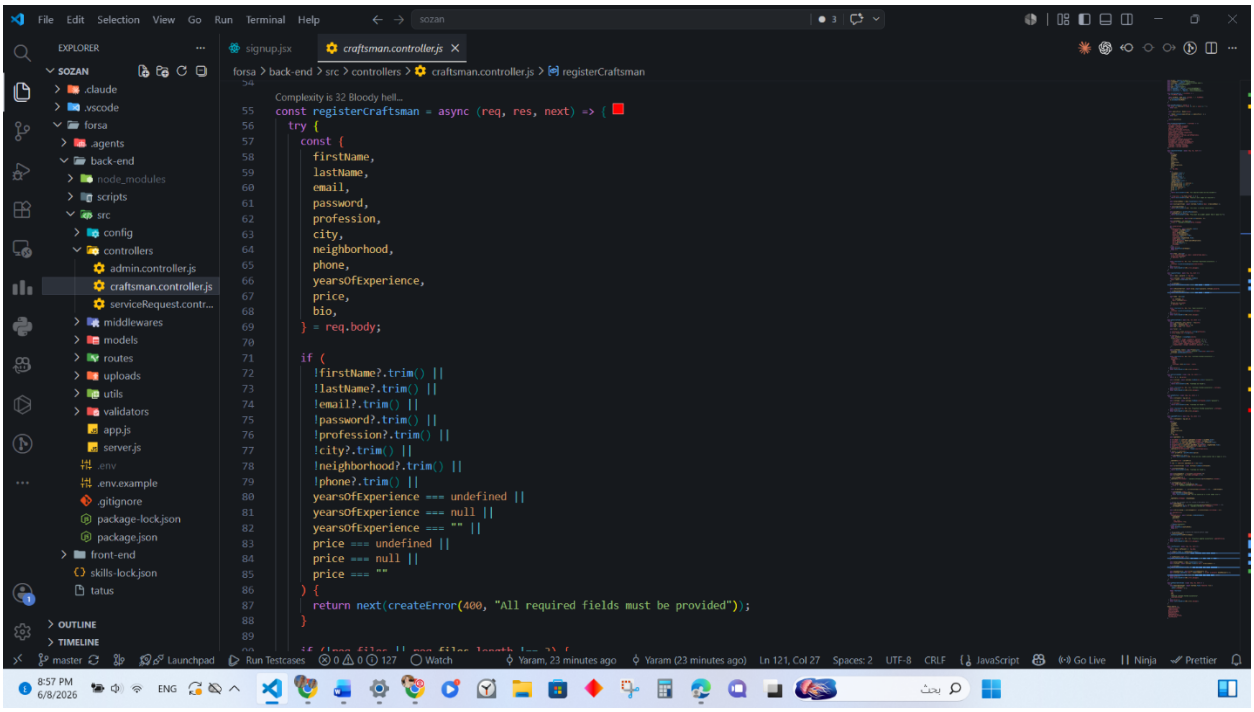
In addition to the three MVC layers, the backend includes a Middlewares layer that handles cross-cutting concerns: auth.middleware.js verifies JWT tokens on protected routes, admin.middleware.js checks for admin role privileges, and upload.middleware.js manages file uploads using Multer with type and size validation.

6.2 Implementation Code

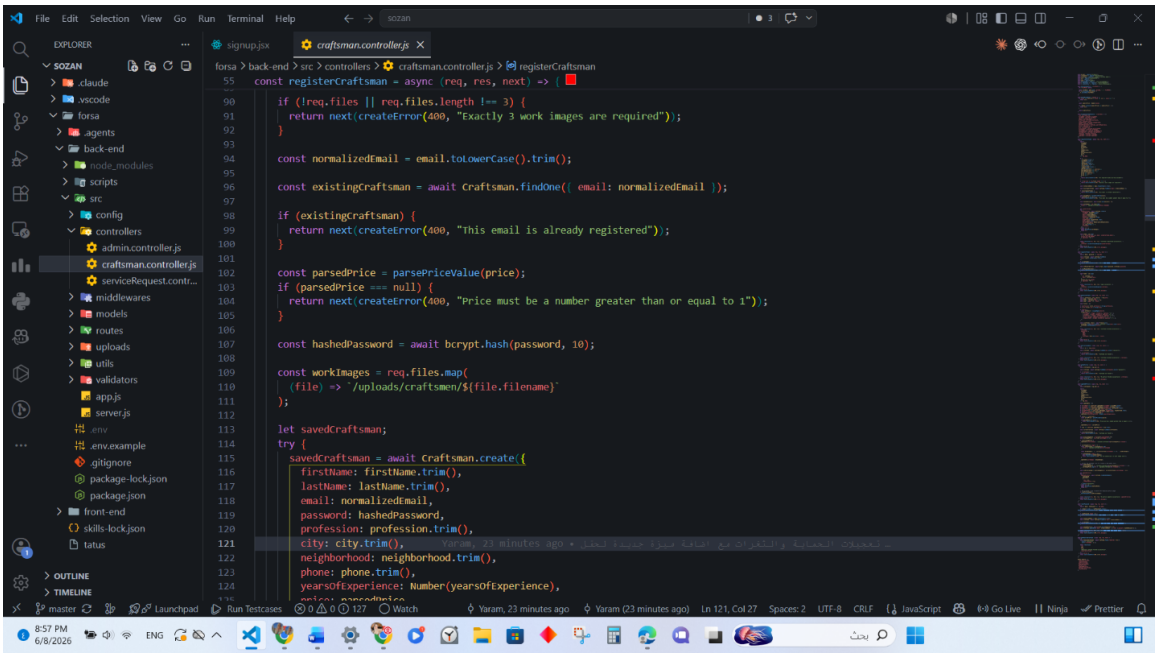
6.2.1 Backend Implementation

Register Craftsman Function

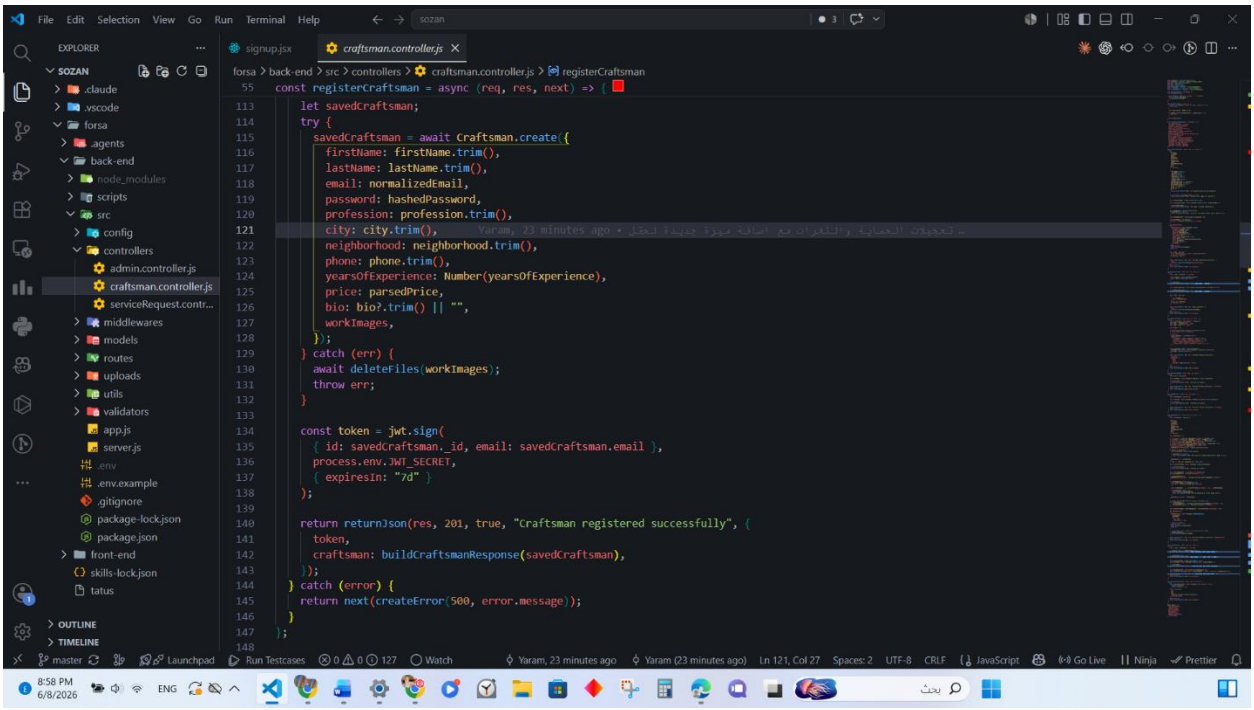
The **registerCraftsman** function handles the creation of new craftsman accounts. It first validates that all required fields are present, then checks that exactly three work images are uploaded. It normalizes the email to lowercase and verifies it is not already registered in the database. The submitted price is validated to ensure it is a valid number greater than or equal to 1. The password is hashed using bcrypt with a salt round of 10 before being stored, ensuring that plain-text passwords are never persisted. The craftsman record is created inside a try/catch block — if the database operation fails, all uploaded image files are deleted from the server to prevent orphaned files. Upon successful registration, a JSON Web Token (JWT) is generated with a 7-day expiration and returned alongside the craftsman's profile data.



(Figure 6.2 — registerCraftsman: input validation)



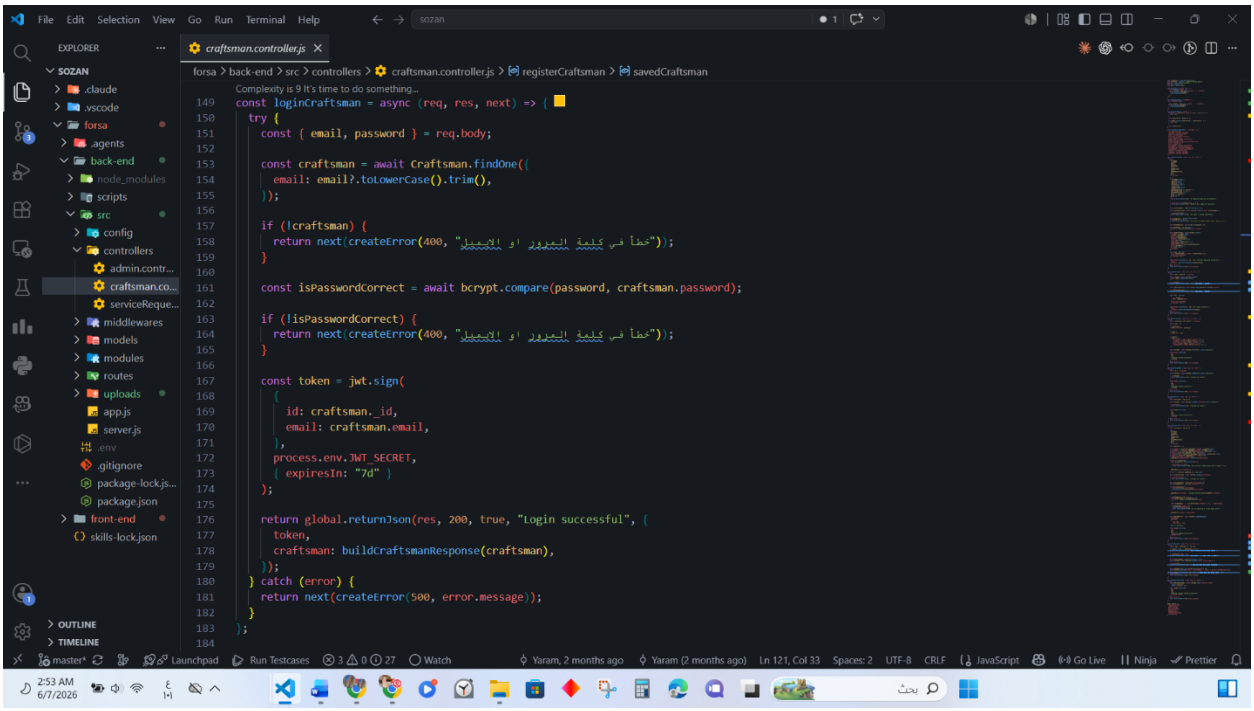
(Figure 6.3 — registerCraftsman: email check and password hashing)



(Figure 6.4 — registerCraftsman: JWT generation and response)

Login Craftsman Function

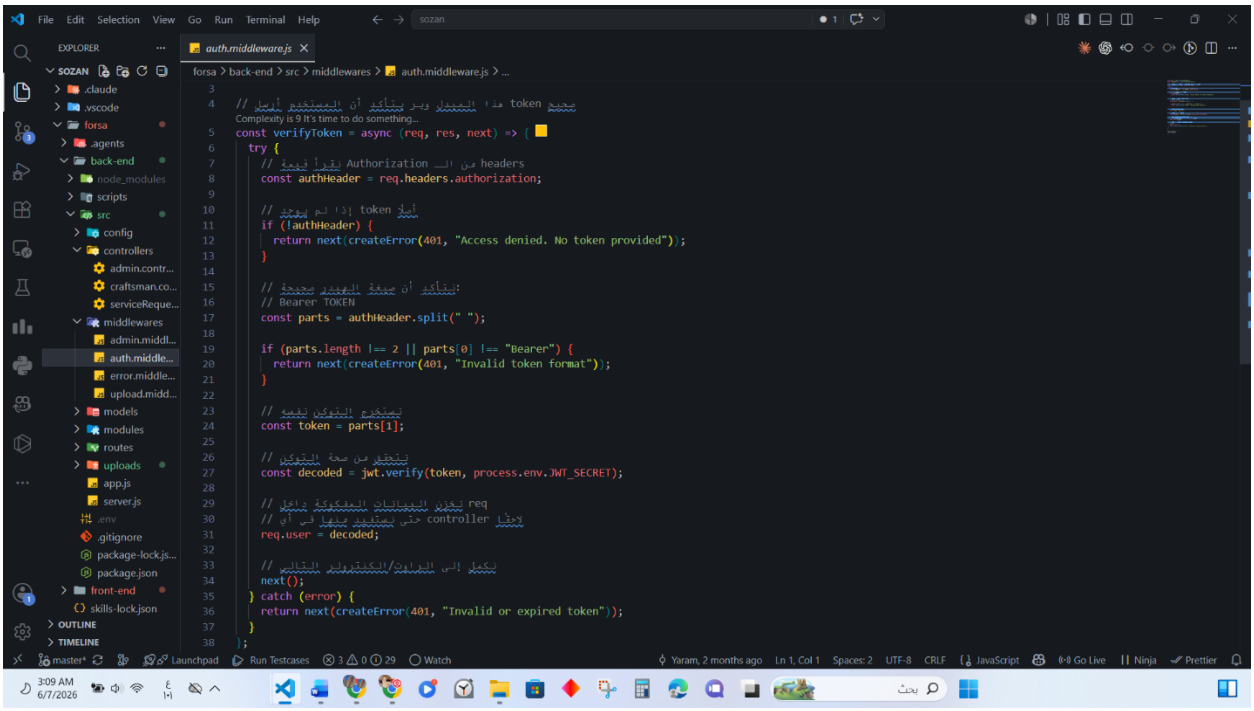
The loginCraftsman function authenticates an existing craftsman. It queries the database using the normalized email and returns a generic error message if no account is found — deliberately avoiding disclosure of whether the email or password is incorrect, which is a standard security practice. The submitted password is compared against the stored hash using bcrypt.compare. On successful authentication, a new JWT is issued and returned with the craftsman's profile, enabling the client to maintain a stateless authenticated session.



(Figure 6.5 — loginCraftsman function)

Verify Token Middleware

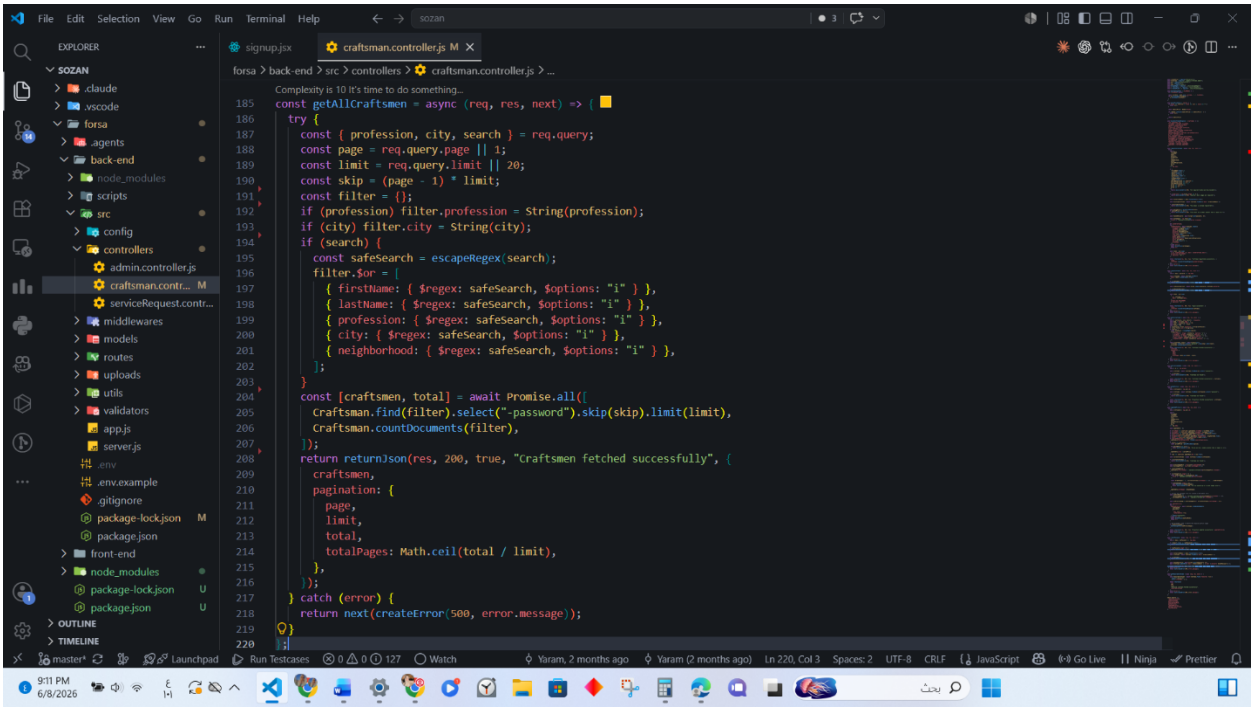
The verifyToken middleware protects routes that require authentication. It reads the Authorization header and validates that it follows the Bearer <token> format. The token is then verified against the application's JWT_SECRET environment variable using the jsonwebtoken library. If the token is valid, the decoded payload containing the craftsman's id and email is attached to the req.user object, making it available to downstream controllers. Any missing, malformed, or expired token results in a 401 Unauthorized response.



(Figure 6.6 — verifyToken middleware)

Get All Craftsmen Function

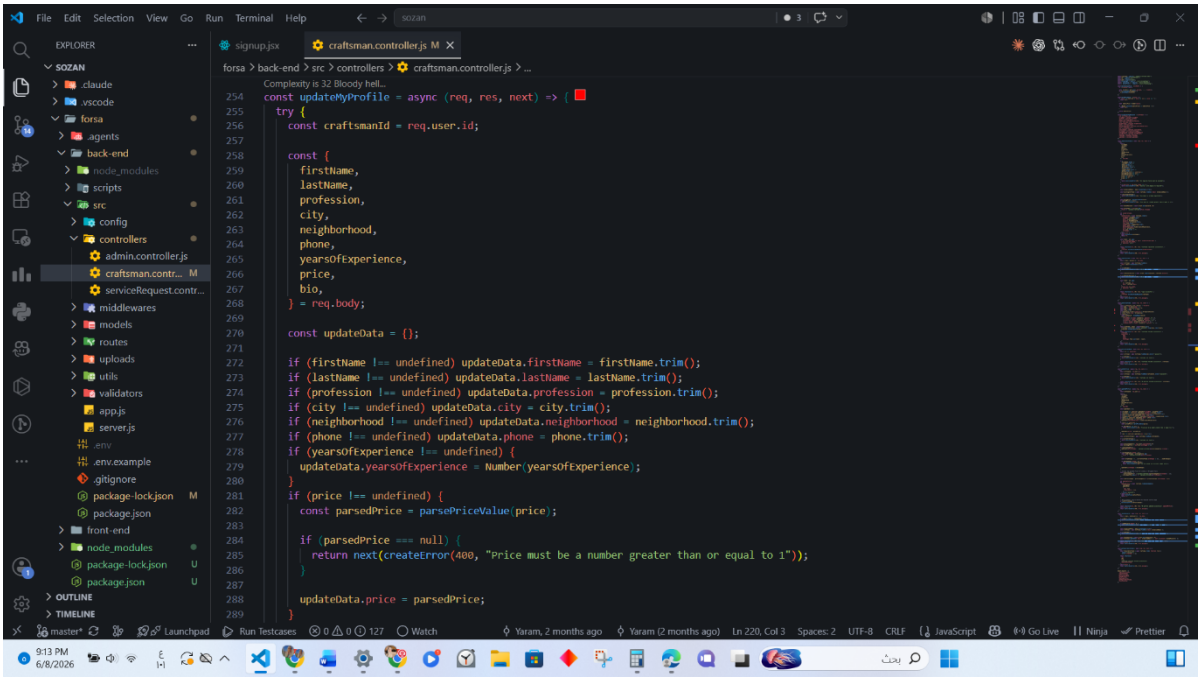
The **getAllCraftsmen** function retrieves craftsmen from the database with support for dynamic filtering and pagination. It accepts three optional filtering parameters: profession and city for exact-match filtering, and search for a case-insensitive text search across the craftsman's first name, last name, profession, city, and neighborhood fields using MongoDB's $regex operator. The search input is sanitized through an escape utility before being used in the query to prevent regular expression injection attacks. The function also accepts page and limit parameters for pagination, defaulting to page 1 and 20 results per page. The query and the total document count are executed in parallel for efficiency, and the password field is explicitly excluded from all results. The response returns the list of craftsmen along with pagination metadata including the current page, limit, total count, and total number of pages.



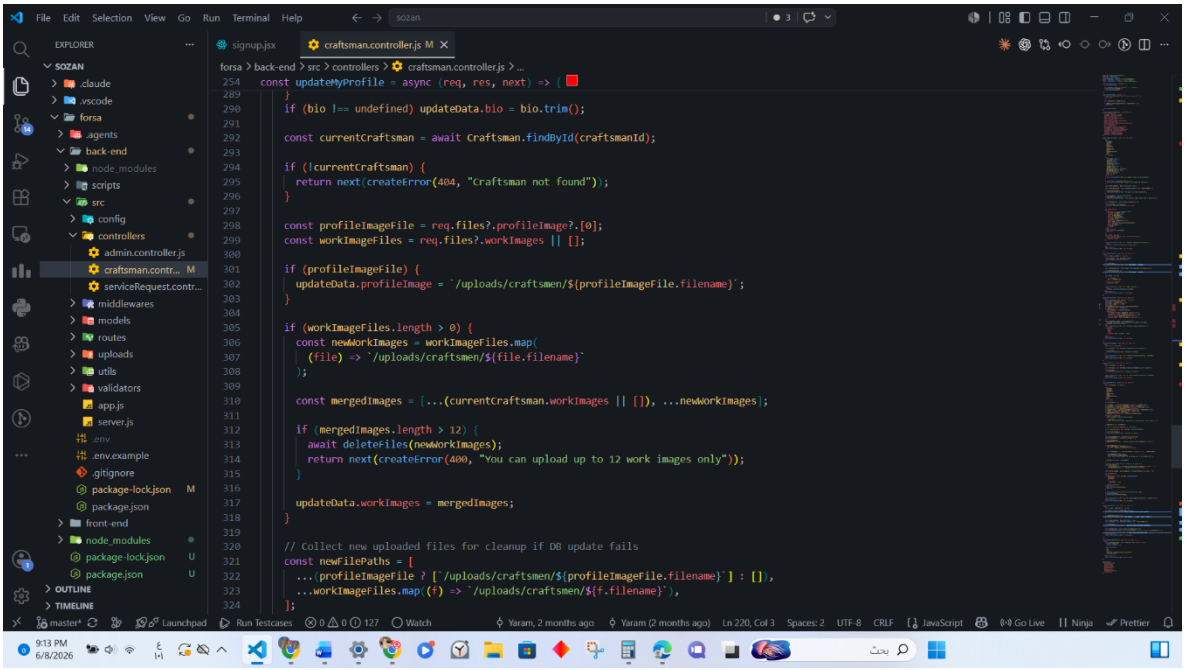
(Figure 6.7 — getAllCraftsmen function)

Update My Profile Function

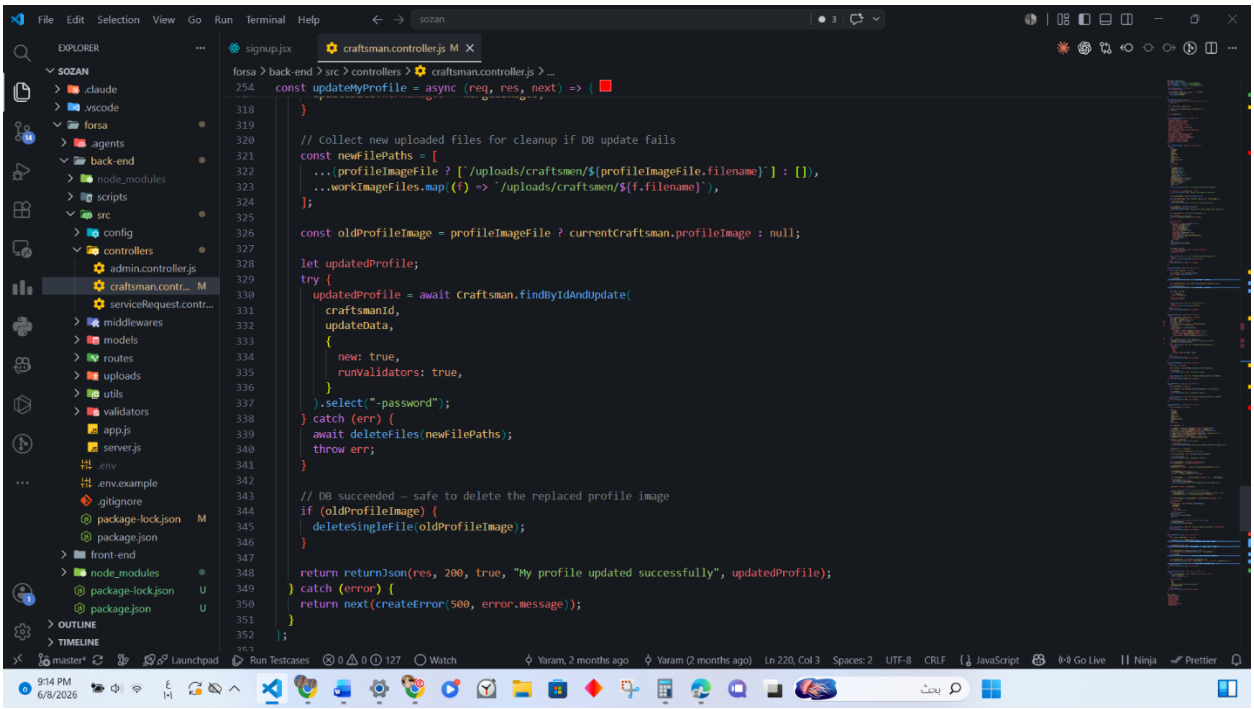
The **updateMyProfile** function allows an authenticated craftsman to update their profile. Only fields explicitly provided in the request body are added to the update object, preventing accidental overwriting of unchanged data. If a price is provided, it is validated to ensure it is a valid number greater than or equal to 1. The function first fetches the current craftsman record to access the existing profile image and work images. If a new profile image is uploaded, it is set in the update data; if new work images are uploaded, they are merged with the existing ones, enforcing a maximum of 12 images total. All newly uploaded file paths are tracked. The database update is performed using findByIdAndUpdate with runValidators: true inside a try/catch block — if the update fails, all newly uploaded files are deleted to prevent orphaned files. The old profile image is only deleted after the database update succeeds, ensuring it remains safe until the change is confirmed. The function returns the updated profile without the password field.



(Figure 6.8 — updateMyProfile: field update and validation)



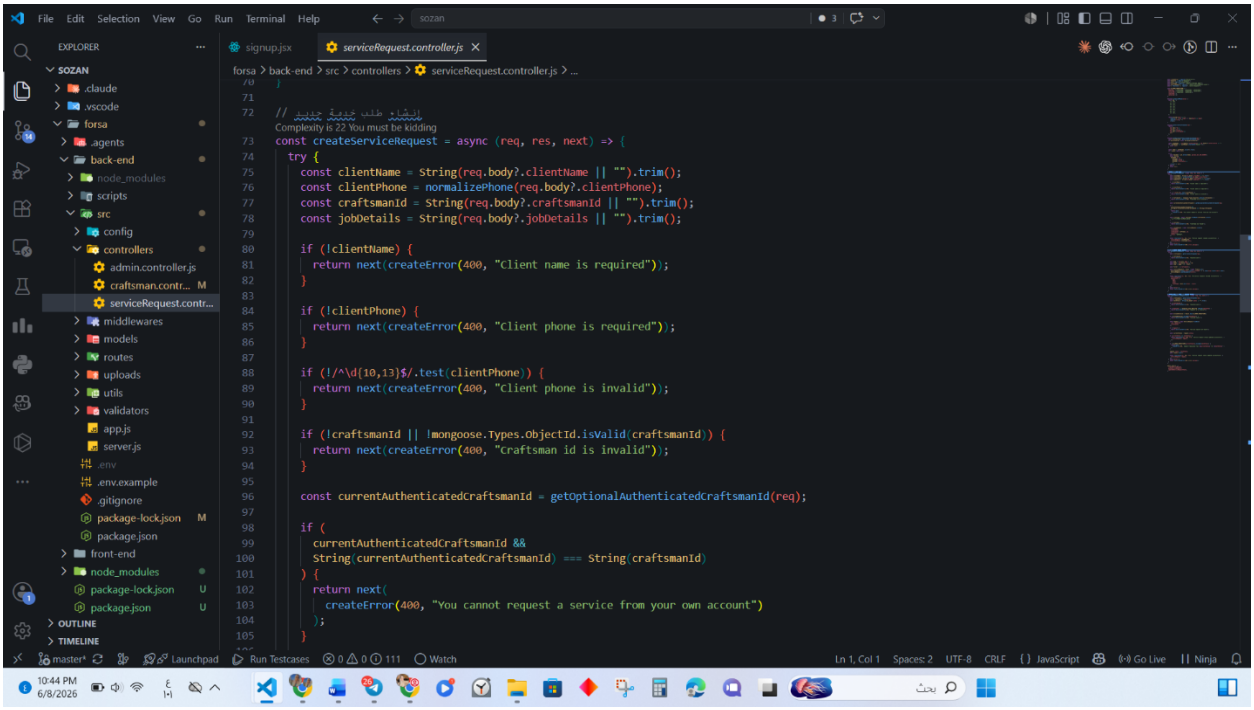
(Figure 6.9 — updateMyProfile: image merging and database update)



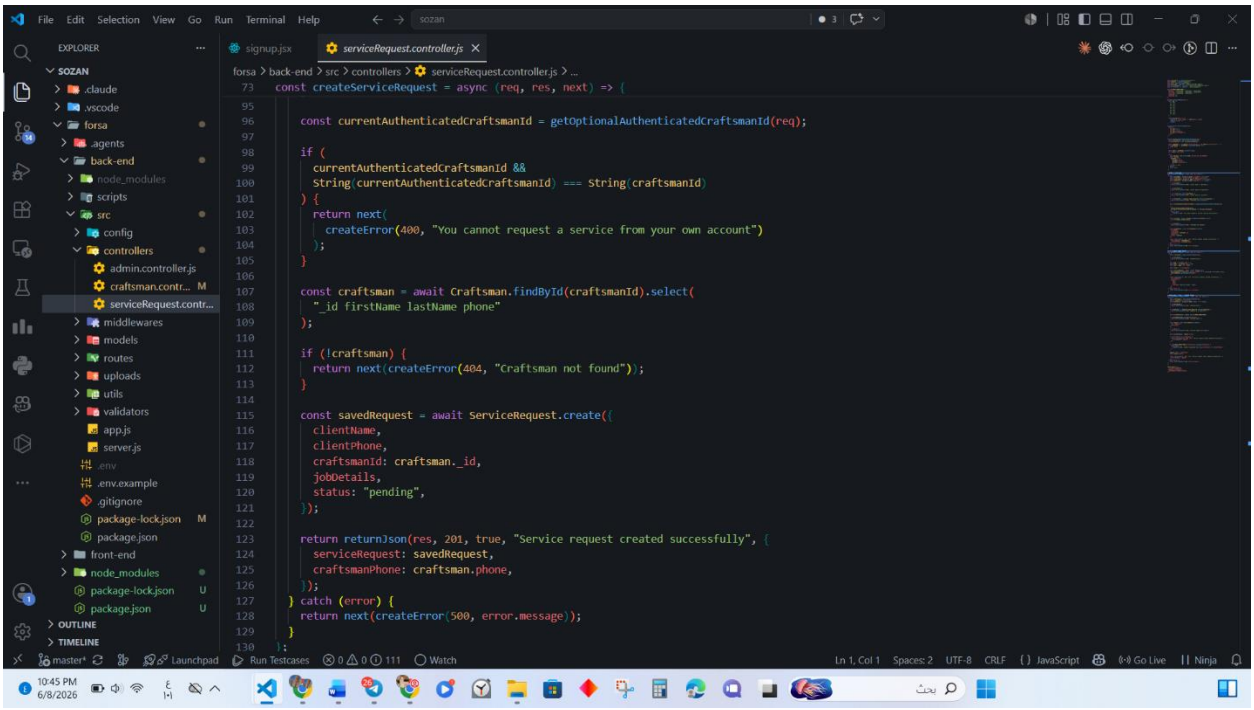
(Figure 6.10 — updateMyProfile: final save and response)

Create Service Request Function

The **createServiceRequest** function handles the submission of a new service request from a client to a registered craftsman and is exposed as an HTTP POST endpoint that does not require authentication. It extracts four fields from the request body: the client name, client phone, craftsman ID, and a free-text job details field describing the work the client requires. The phone value is first passed through a normalization utility that converts Eastern-Arabic numerals to their Western equivalents and strips whitespace, ensuring a uniform format before validation. The function then enforces sequential validation rules: the client name must be non-empty, the phone must match a 10-to-13 digit pattern, and the craftsman ID must be a valid MongoDB ObjectId. A self-request security check follows: if the caller carries a valid JWT token whose subject matches the target craftsman ID, the request is rejected to prevent a craftsman from generating requests against their own profile. After confirming the target craftsman exists, the function persists a new service request with an initial status of "pending" and returns the saved request together with the craftsman's contact phone number, enabling the client to reach them directly.



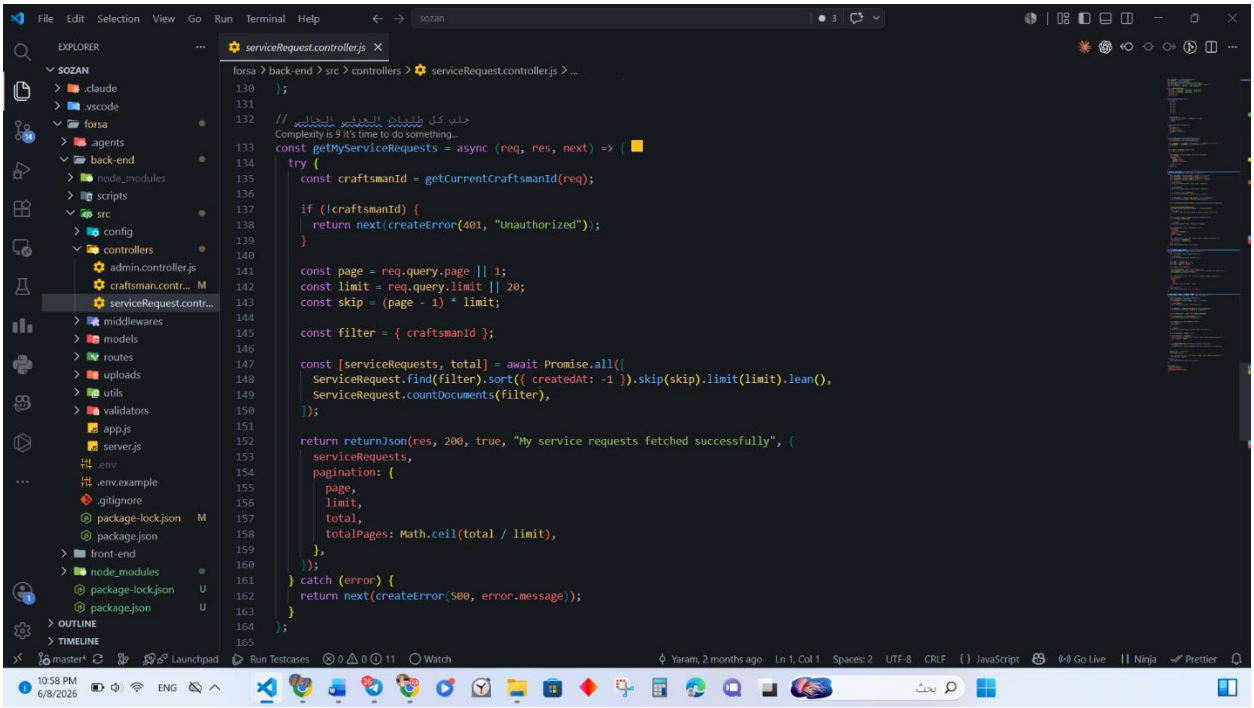
(Figure 6.11 — createServiceRequest: validation and self-request check)



(Figure 6.12 — createServiceRequest: craftsman check and response)

Get My Service Requests Function

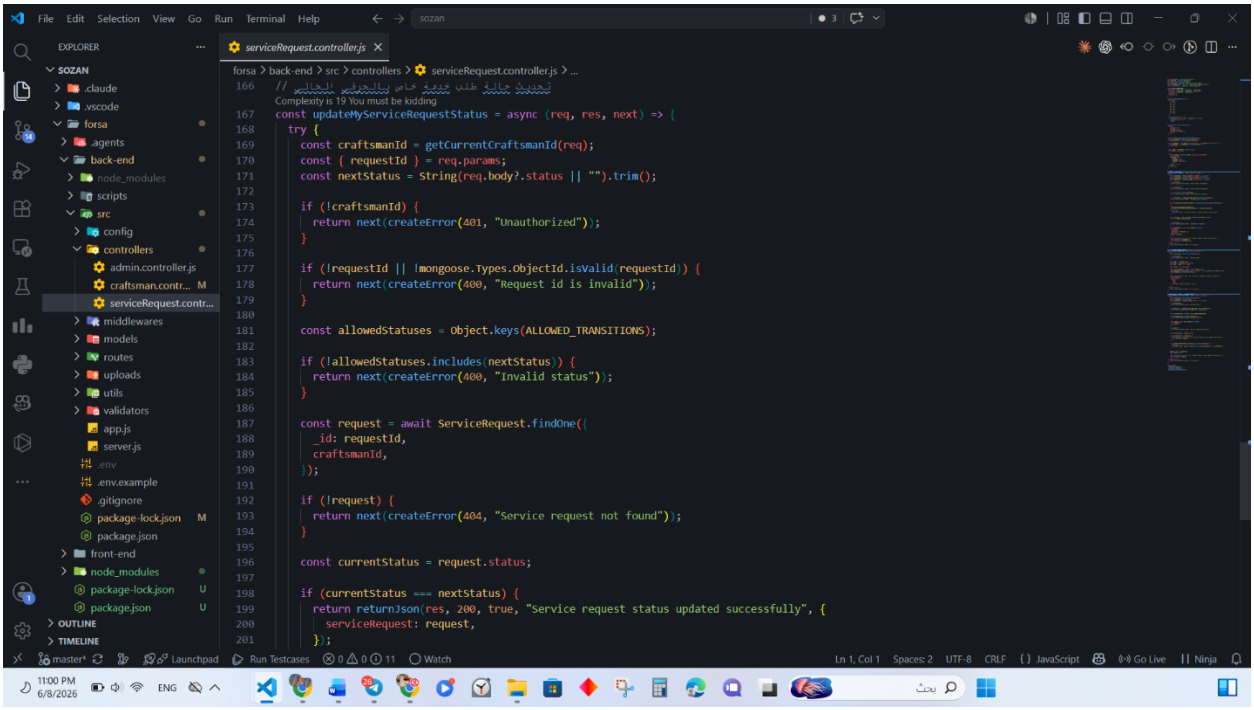
The **getMyServiceRequests** function is an authenticated endpoint that retrieves all service requests directed at the currently signed-in craftsman. Authentication is mandatory; the craftsman's identity is resolved from the request object populated by the authentication middleware, and the request is rejected if no identity is found. The function reads optional page and limit query parameters to support offset-based pagination. The query filters service requests strictly by the authenticated craftsman's identifier, sorts results in reverse-chronological order, and runs the paginated query alongside a count operation in parallel for efficiency. It returns the matching service requests along with pagination metadata including the current page, page size, total count, and total number of pages.



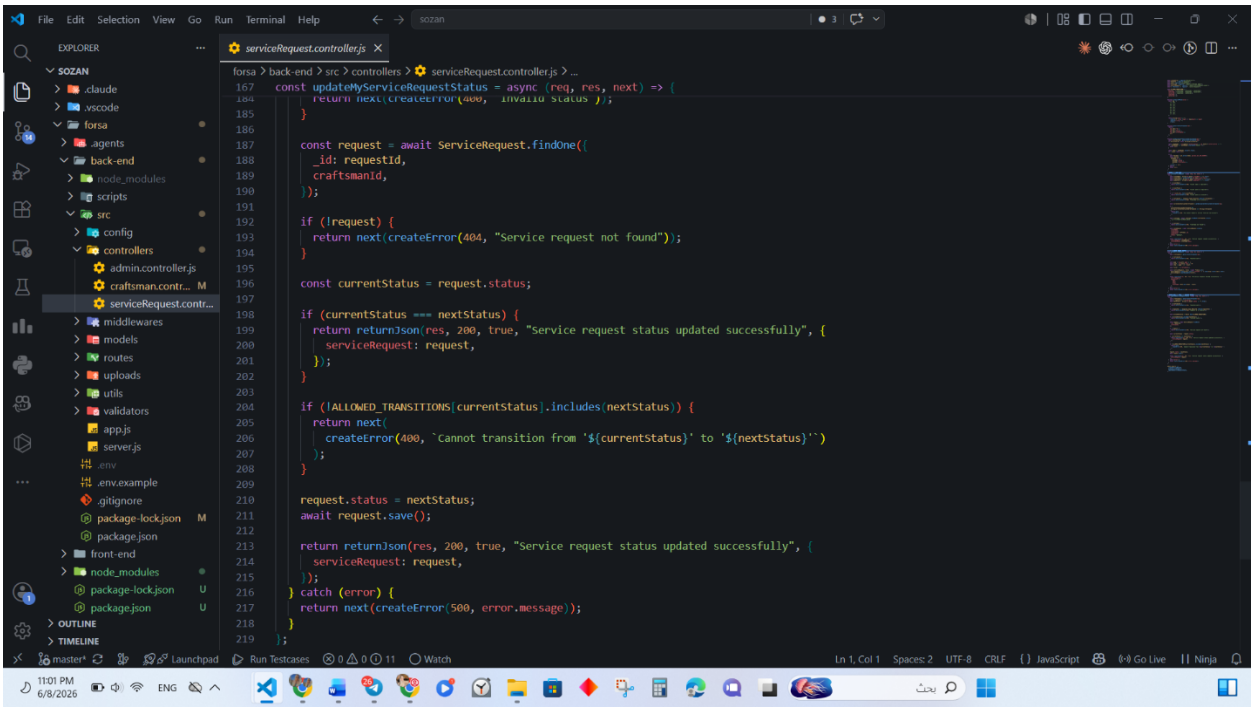
(Figure 6.13 — getMyServiceRequests function)

Update Service Request Status Function

The **updateMyServiceRequestStatus** function is an authenticated endpoint that allows a craftsman to advance the status of one of their own service requests through a strictly governed state machine. The function requires authentication and identifies the target request by its route parameter, validated as a valid ObjectId. Ownership is enforced at the database level by querying for a request that matches both the request ID and the authenticated craftsman's identifier simultaneously, ensuring a craftsman can never modify a request belonging to another. To support idempotent clients, if the requested status is identical to the current status, the function returns immediately without writing to the database. Otherwise the transition is validated against an allowed-transitions table: a pending request may move to confirmed, contacted, or cancelled; a confirmed request may move to contacted, completed, or cancelled; a contacted request may move only to completed or cancelled; while completed and cancelled are terminal states with no outgoing transitions. Any invalid transition is rejected, and a permitted transition updates the status and returns the updated request.



(Figure 6.14 — updateMyServiceRequestStatus: authentication and ownership check)

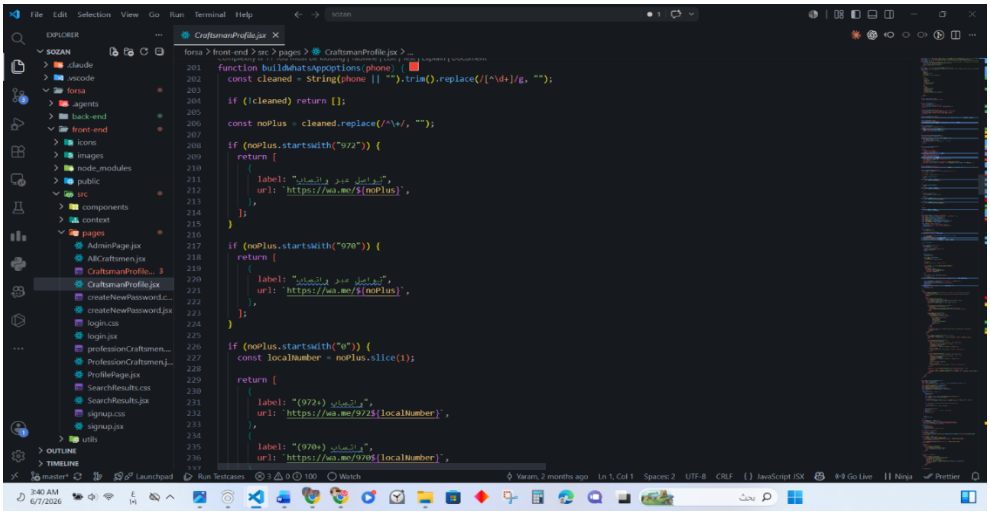


(Figure 6.15 — updateMyServiceRequestStatus: state machine and update)

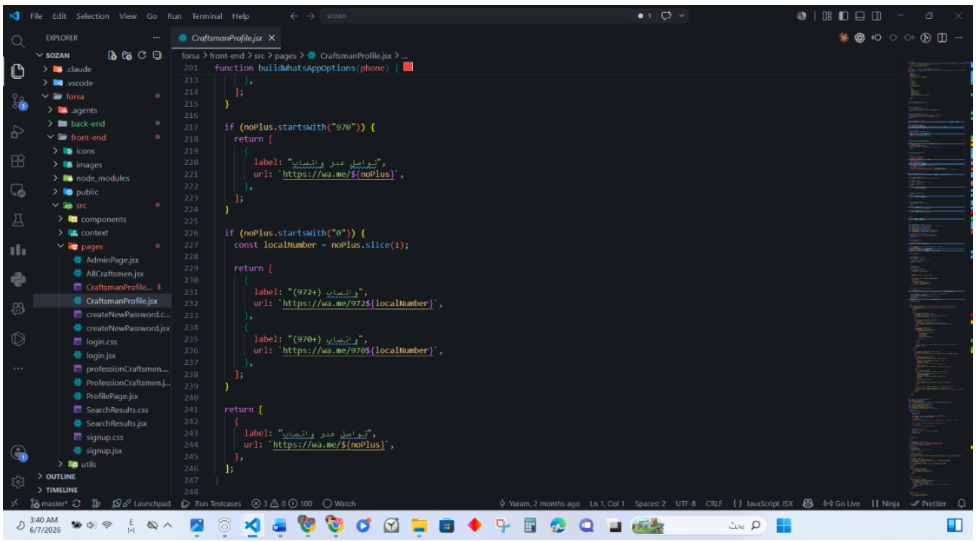
6.2.2 Frontend Implementation

Build WhatsApp Options Function

The buildWhatsAppOptions function generates one or more WhatsApp deep-link URLs based on the craftsman's phone number. Since Palestinian phone numbers can be valid under either the +972 or +970 country code, the function inspects the cleaned number prefix. If the number already contains a recognized country code, a single link is returned. If the number starts with a local 0, two separate WhatsApp links are generated — one for each country code — allowing the client to attempt contact through both. If the number does not match any of the recognized patterns, the function falls back to returning a single link using the number as provided. This ensures maximum reachability given the regional telecommunications ambiguity in Palestine



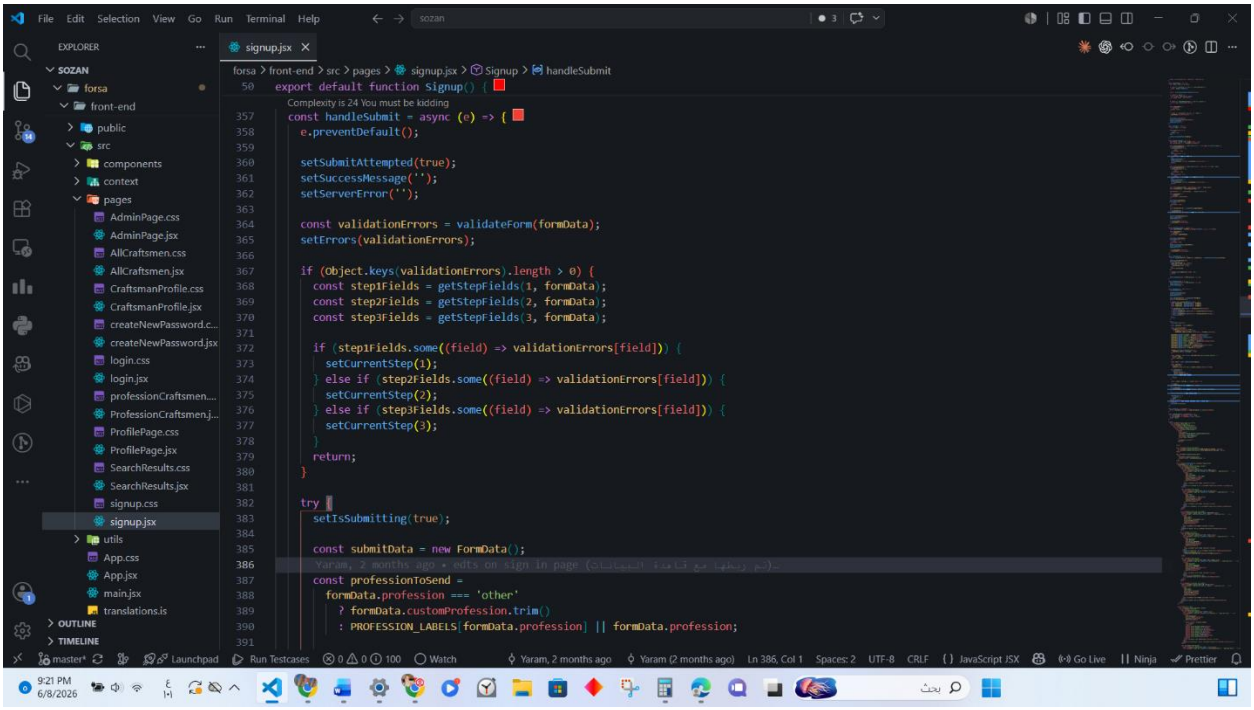
(Figure 6.16 — buildWhatsAppOptions: country code handling)



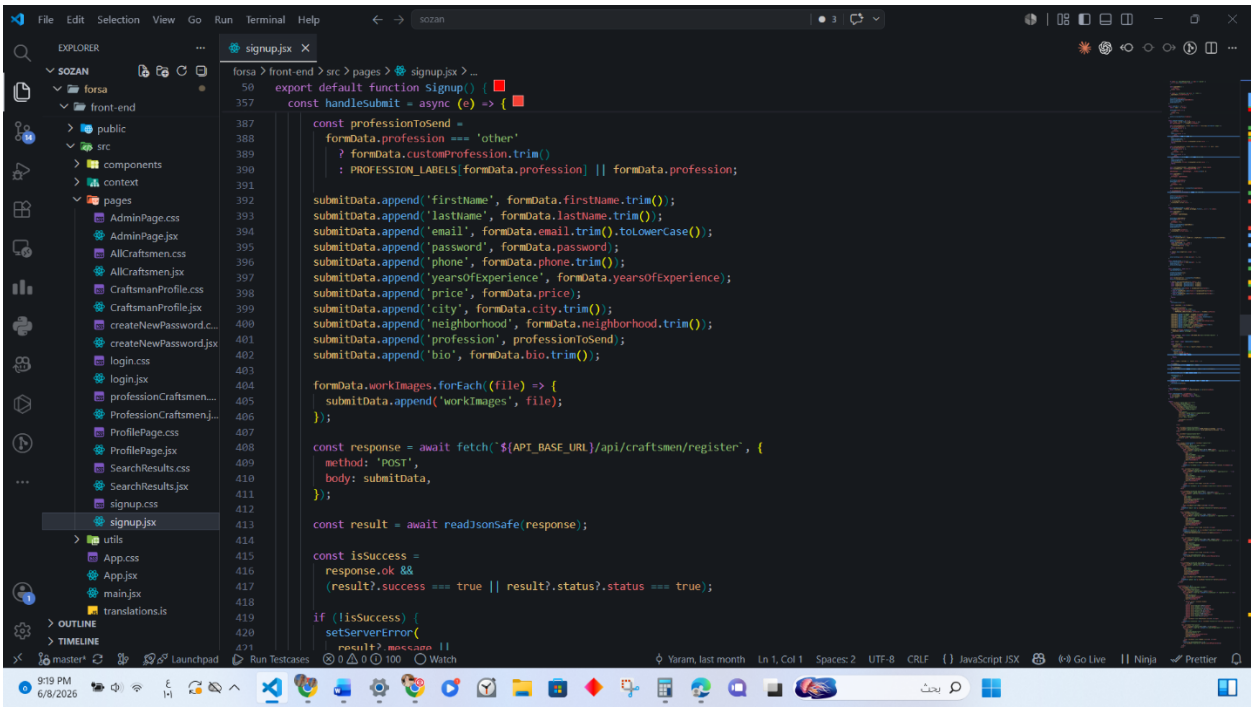
(Figure 6.17 — buildWhatsAppOptions: local number handling and fallback)

Handle Submit Function

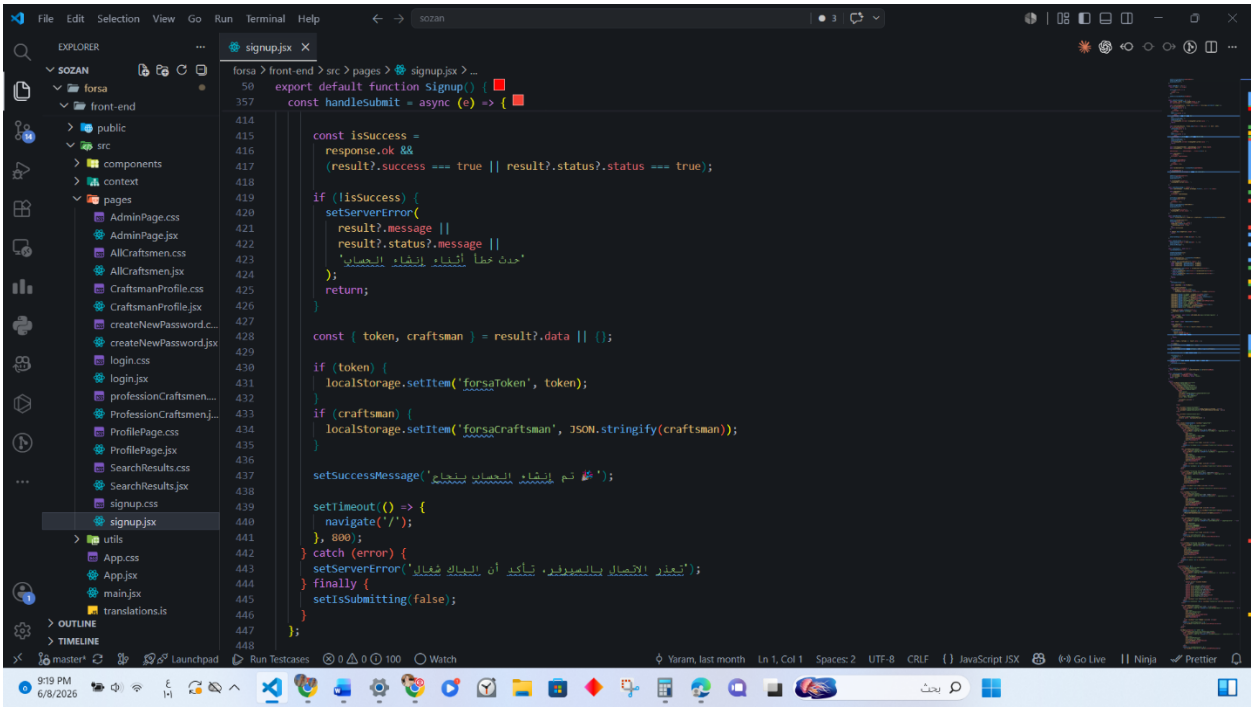
The **handleSubmit** function manages the final submission of the craftsman registration form. It first re-validates the entire form through a validation function that enforces several rules: all required fields must be present, the email must end with @gmail.com, the password must be at least 8 characters and contain letters, digits, and at least one special character, the two password fields must match, the phone number must start with 059 and be exactly 10 digits, and exactly three work images must be uploaded. If any errors exist, the user is navigated back to the specific step containing the first error. Once validation passes, the data is assembled into a FormData object — required because the request includes binary image files alongside text fields. The profession value is resolved from an internal key to its Arabic display label before submission, or the custom profession text is used if "Other" was selected. On a successful API response, the returned JWT token and craftsman object are stored in localStorage to establish a client-side session, and the user is redirected to the home page after a short delay.



(Figure 6.18— handleSubmit: form validation and step navigation)



(Figure 6.19 — handleSubmit: FormData assembly and API call)



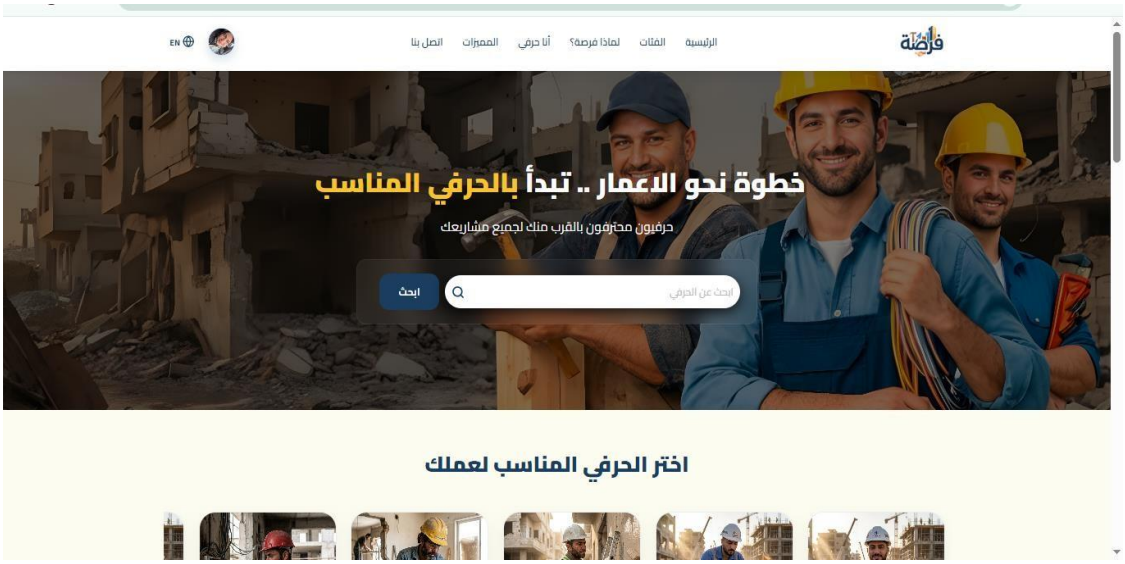
(Figure 6.20 — handleSubmit: session storage and redirect)

6.3 Interfaces

6.3.1 Web Application

Home Page

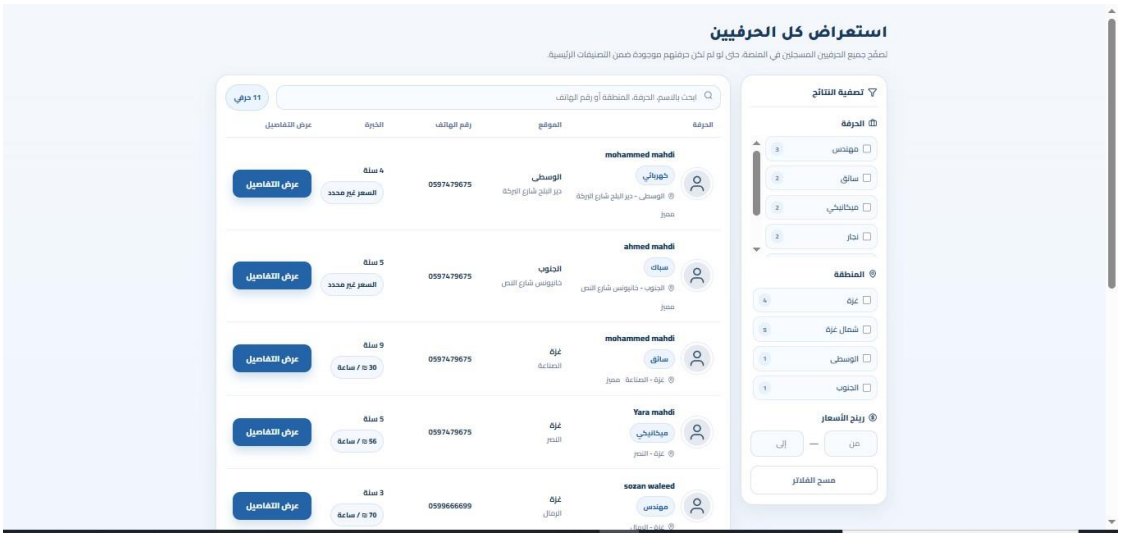
The Home Page serves as the main entry point of the Forsa platform. It features an animated hero section with a search bar that accepts craftsman profession names, displaying auto-complete suggestions as the user types. Below the hero section, the page displays eight profession category cards — engineer, plumber, painter, carpenter, electrician, technician, driver, and mechanic — that navigate to filtered results when clicked. A "Why Forsa" section highlights the platform's key values: safety and privacy, fast response, direct contact, and trusted specialists. A featured craftsmen carousel displays highlighted profiles, and a call-to-action section encourages craftsmen to register on the platform.



(Figure 6.21 — Home Page)

Browse All Craftsmen Page

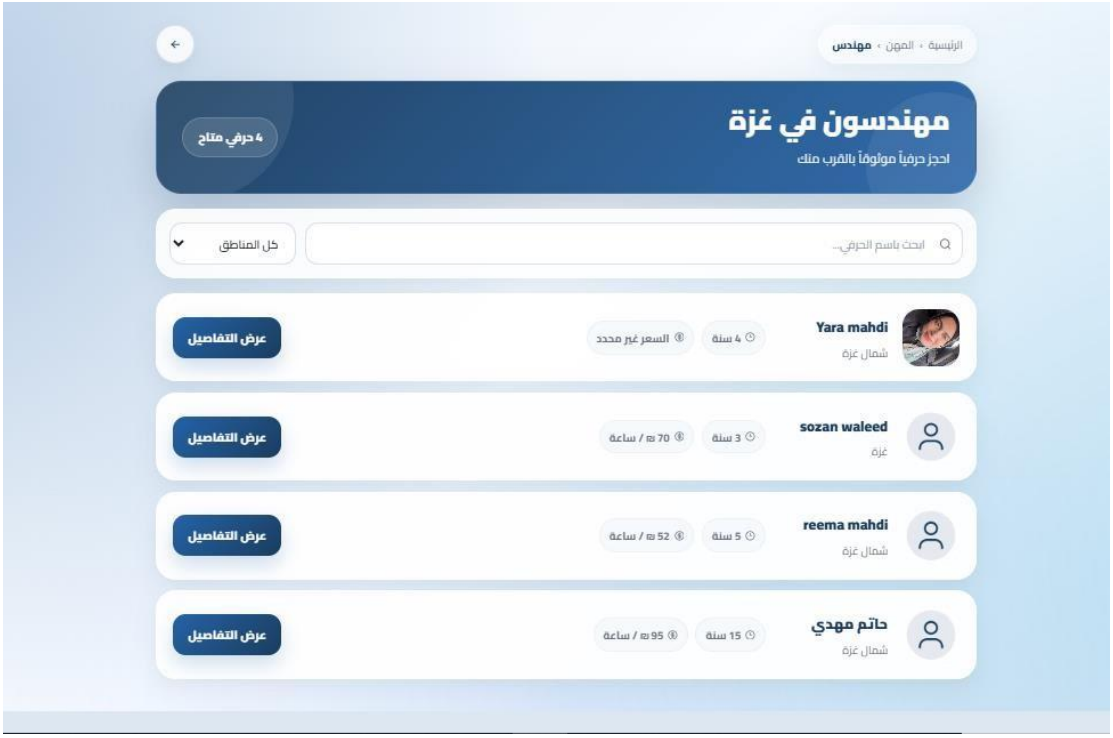
This page displays the complete list of registered craftsmen. A search bar at the top allows filtering across name, profession, city, and neighborhood. Each craftsman is displayed in a row showing their name, profession badge, location, phone number, years of experience, price, and a "View Details" button. A filter panel on the right allows filtering by profession, city region, and price range..



(Figure 6.22 — Browse All Craftsmen Page)

Craftsmen by Profession Page

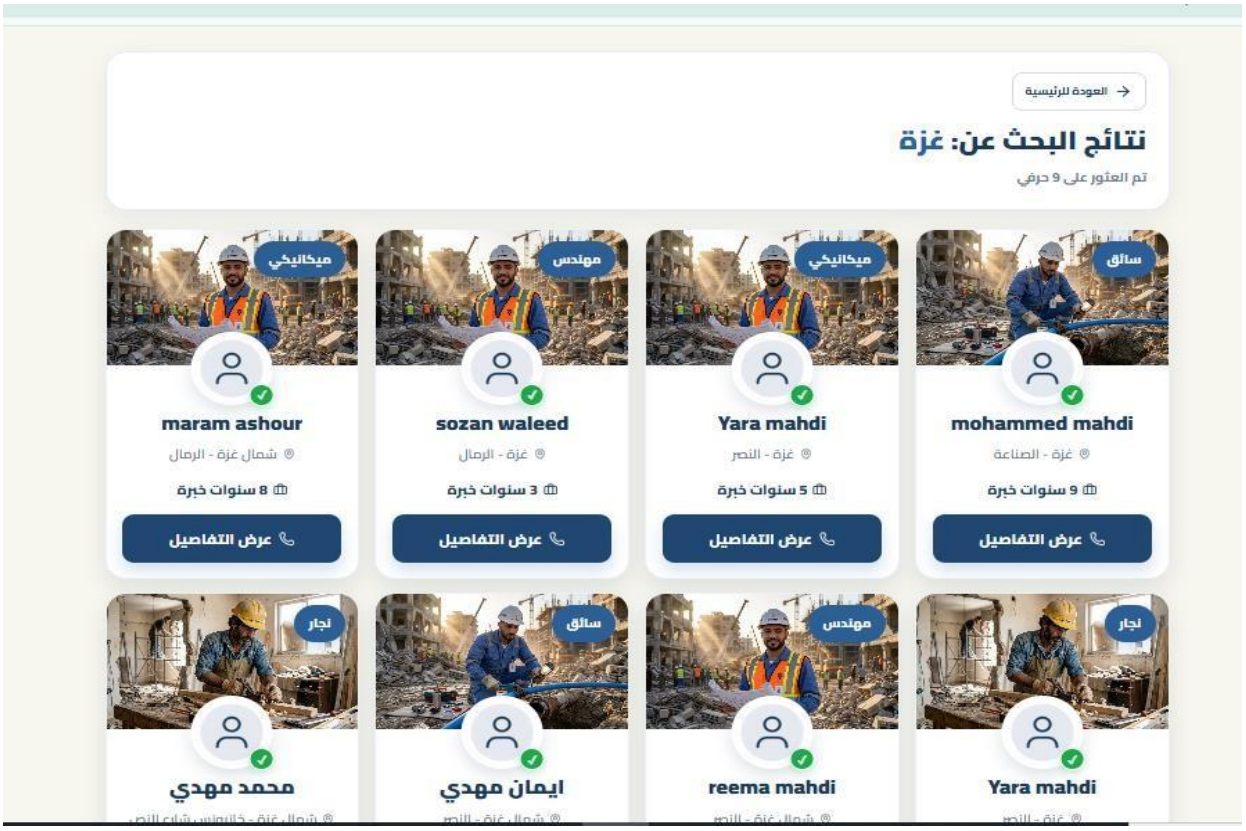
This page displays craftsmen filtered by a specific profession selected from the homepage categories. The page title reflects the selected profession and city, and a badge shows the total number of available craftsmen. A neighborhood filter dropdown and a name search bar are available to narrow results further. Each craftsman is shown with their name, location, years of experience, and hourly rate, along with a "View Details" button that navigates to their full profile.



(Figure 6.23 — Craftsmen by Profession Page)

Search Results Page

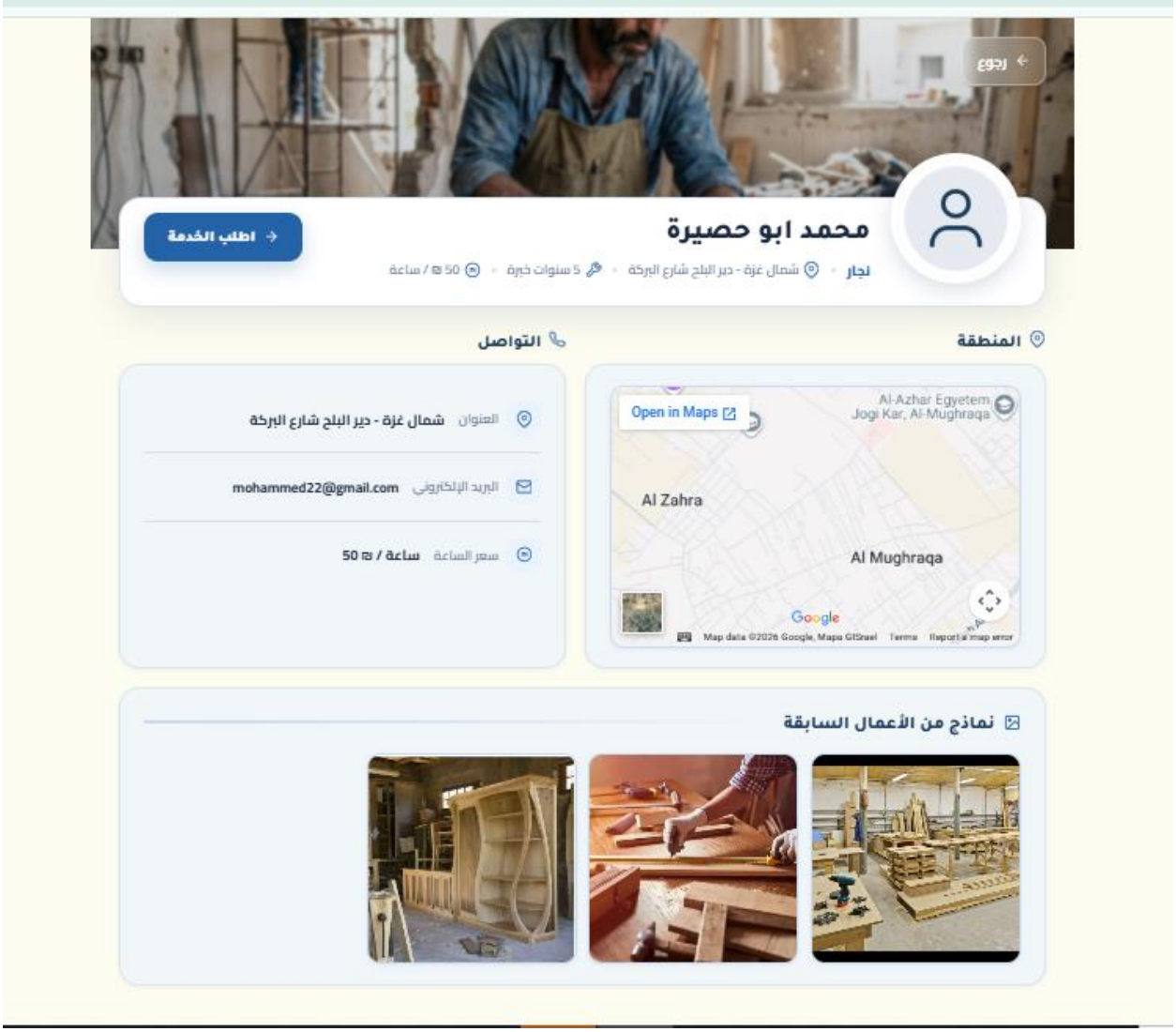
This page displays craftsmen matching a search query submitted from the homepage search bar. The search term is reflected in the page title, and the total number of results found is shown below it. Results are shown as cards with the craftsman's work image as the card background, displaying the craftsman's name, profession badge, location, years of experience, and a "View Details" button.



(Figure 6.24 — Search Results Page)

Craftsman Profile Page

This page displays the complete public profile of a selected craftsman. It includes the craftsman's name, profession, location, years of experience, hourly rate, and a short bio in a profile card at the top. Below that, a contact details section shows the address, email, and price, alongside an embedded Google Maps view of the craftsman's area. A work samples gallery displays the craftsman's uploaded images. A prominent "Request Service" button opens the service request modal.

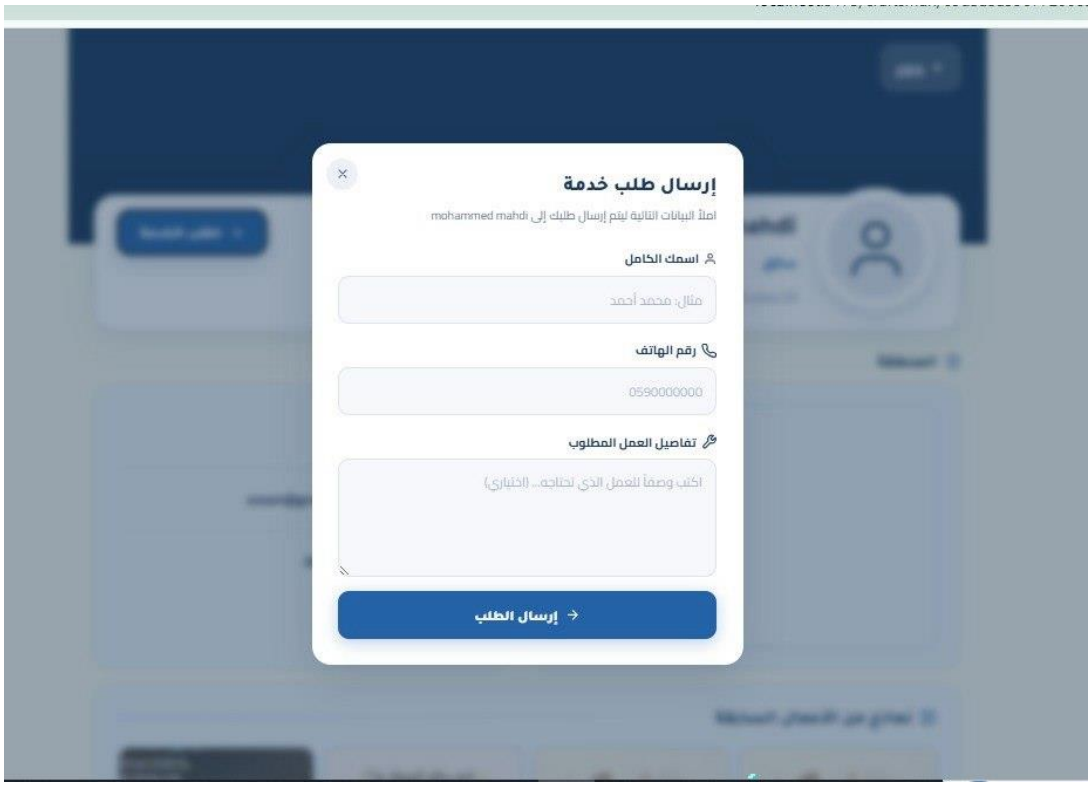


(Figure 6.25 — Craftsman Profile Page)

Service Request Modal

When a visitor clicks "Request Service" on a craftsman's profile page, a modal dialog appears. The user enters their full name, phone number, and optionally a description of the required service details. On submission, the request is sent to the backend API. Upon success, the modal transitions to a confirmation screen that displays WhatsApp deep-link buttons allowing direct .contact with the craftsman

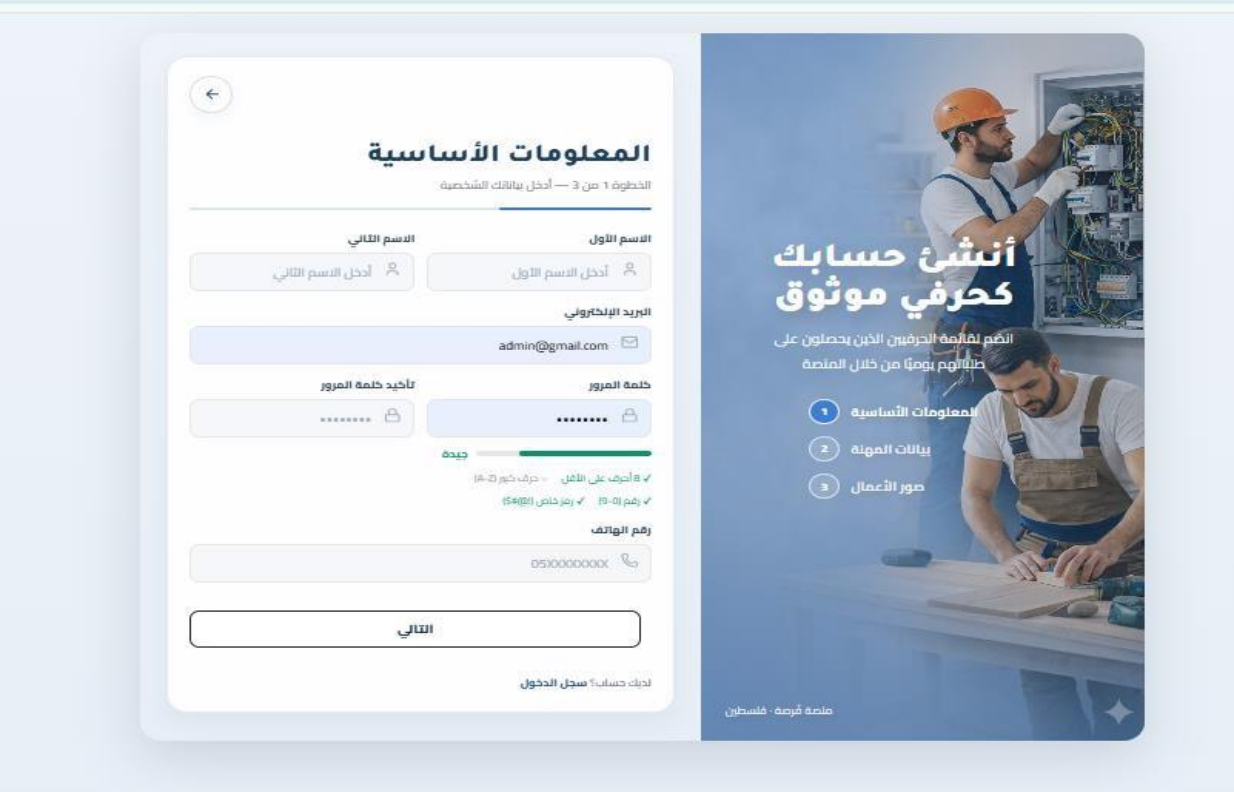
Note: The service details field was added to this form in the final version of the system.



(Figure 6.26 — Service Request Modal)

Craftsman Registration Page — Step 1 (Personal Information)

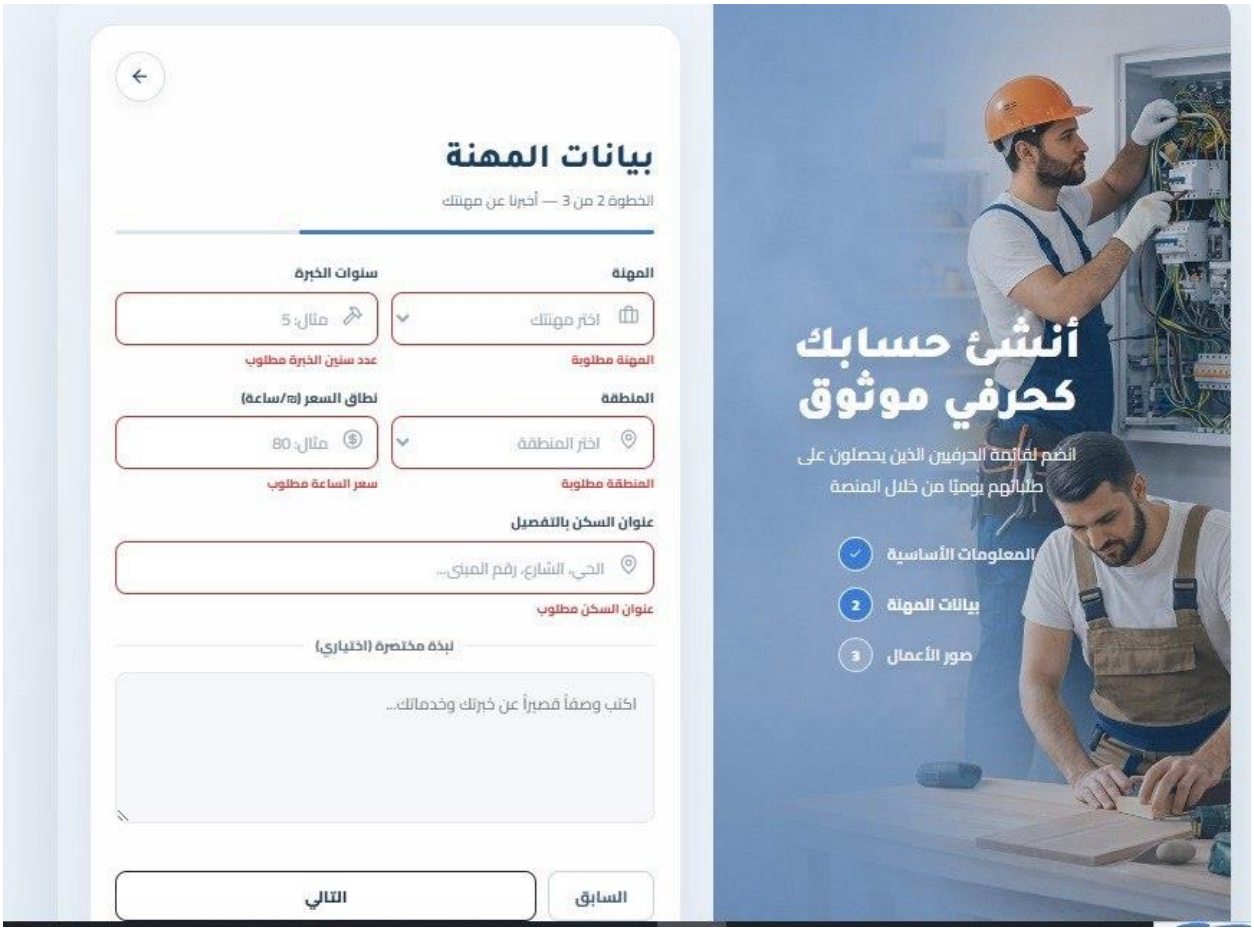
The registration page uses a three-step wizard layout with a progress indicator. Step 1 collects the craftsman's basic personal information: first name, last name, email address, password, password confirmation, and phone number. The password field includes a real-time strength indicator and a requirements checklist that updates as the user types. A sidebar on the right displays the three registration steps with the current active step highlighted.



(Figure 6.27 — Craftsman Registration Page — Step 1)

Craftsman Registration Page — Step 2 (Professional Information)

The second step of the registration wizard collects the craftsman's professional details. It includes the profession field, selected from a predefined list of eight crafts (engineer, plumber, carpenter, electrician, painter, technician, driver, mechanic) with an option to enter a custom profession if "Other" is chosen. The step also collects the years of experience, the city (selected from the four main regions of Gaza: Gaza, North Gaza, Central, and South), the hourly price in shekels, and a detailed neighborhood or street address. An optional bio field allows the craftsman to briefly describe their work. All required fields are validated before the user can proceed to the final step.



(Figure 6.28 — Craftsman Registration Page — Step 2)

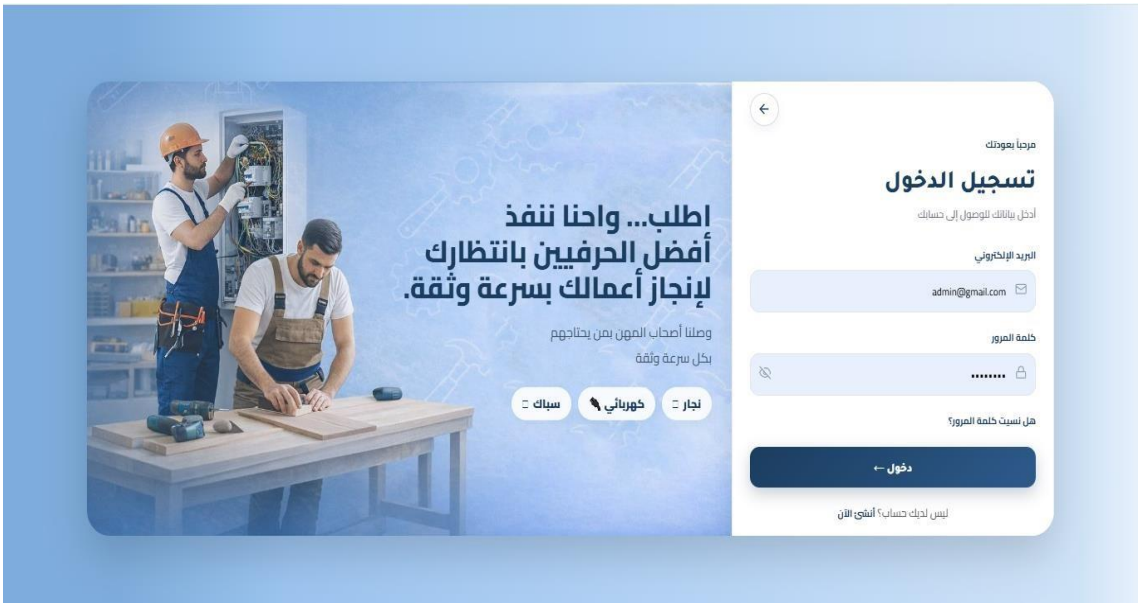
Craftsman Registration Page — Step 3 (Work Images)

The third and final step requires the craftsman to upload exactly three work sample images. A drag-and-drop upload zone is provided alongside preview slots that display each image as it is added, with the option to remove any image individually. Accepted formats are PNG and JPG with a maximum size of 5MB per image. A tips panel encourages uploading clear, high-quality photos to attract more service requests. Upon submission, all collected data from the three steps is sent to the backend API as a multipart request, and on success the craftsman is registered and redirected to the home page.

(Figure 6.29— Craftsman Registration Page — Step 3)

Login Page

The Login Page allows craftsmen and administrators to authenticate. The form collects email and password with inline validation. A password visibility toggle is provided. A "Forgot password?" link navigates to the password reset page. The system automatically determines whether the credentials belong to an admin or a craftsman and redirects accordingly.

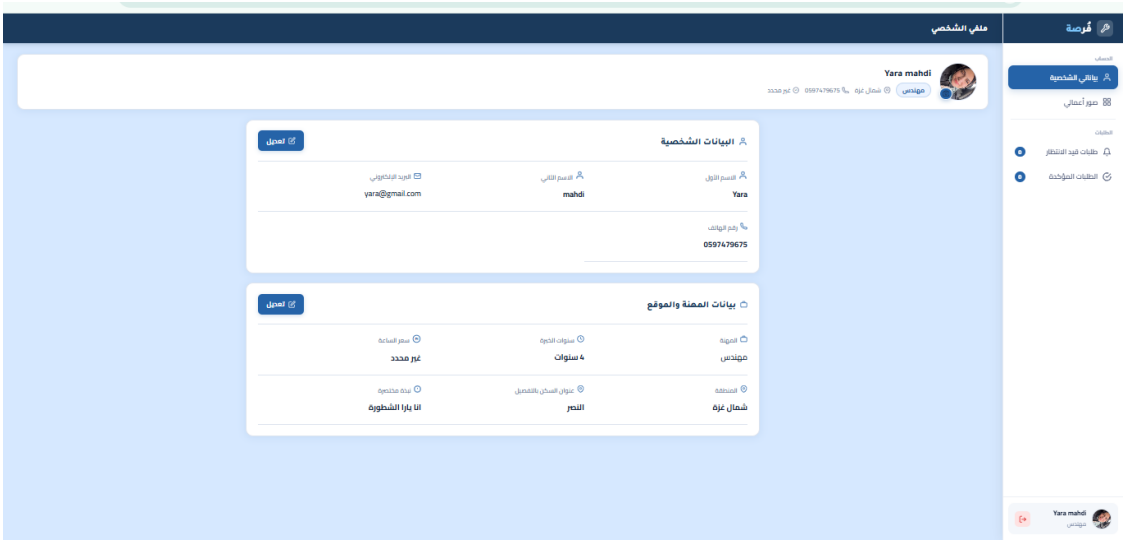


(Figure 6.30 — Login Page)

6.3.2 Craftsman Dashboard

Personal Profile Tab

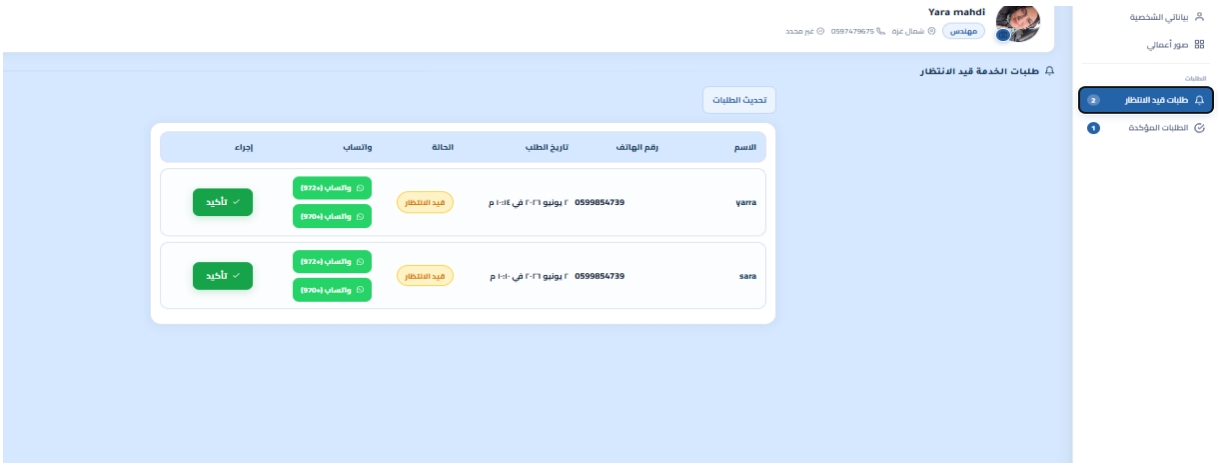
Authenticated craftsmen can access their personal dashboard from the navigation bar. The dashboard features a sidebar with navigation tabs for personal data, work images, and service requests. The personal profile tab displays the craftsman's full profile information including name, profession, contact details, location, years of experience, hourly price, and bio. Each section has an edit button that activates inline input fields, allowing the craftsman to update their information and save changes directly



(Figure 6.31 — Craftsman Dashboard — Personal Profile Tab)

Pending Service Requests Tab

This tab displays all incoming service requests submitted by clients. Each request shows the client's name, phone number, submission date, and current status. Two WhatsApp contact buttons are provided per request — one for the +972 prefix and one for the +970 prefix — to accommodate both dialing formats used in Palestine. A confirm button allows the craftsman to mark a request as confirmed. A refresh button at the top of the tab allows manual reloading of the request list.

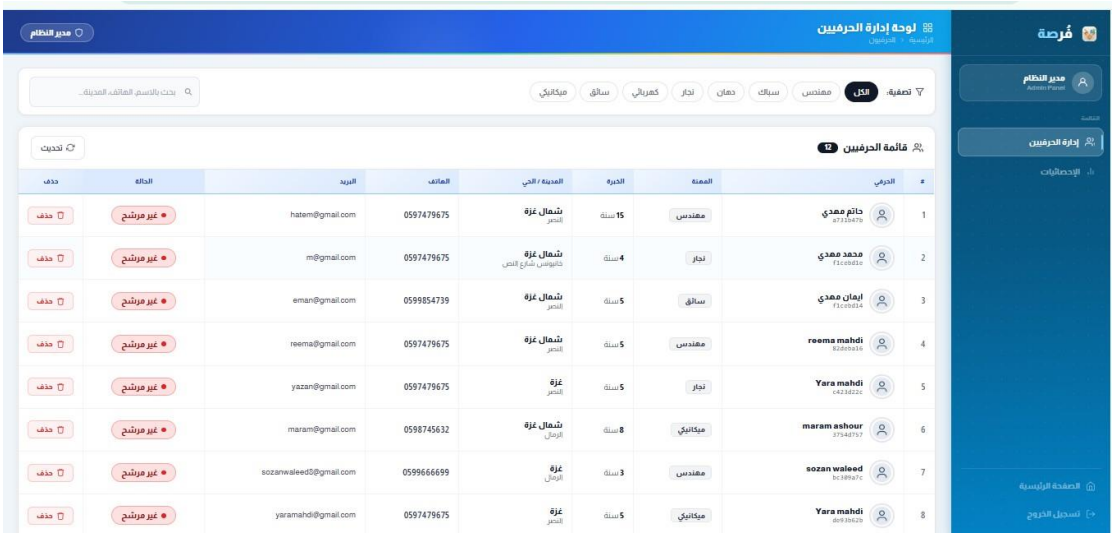


(Figure 6.32— Craftsman Dashboard — Pending Service Requests Tab)

6.3.3 Admin Panel

Craftsmen Management Page

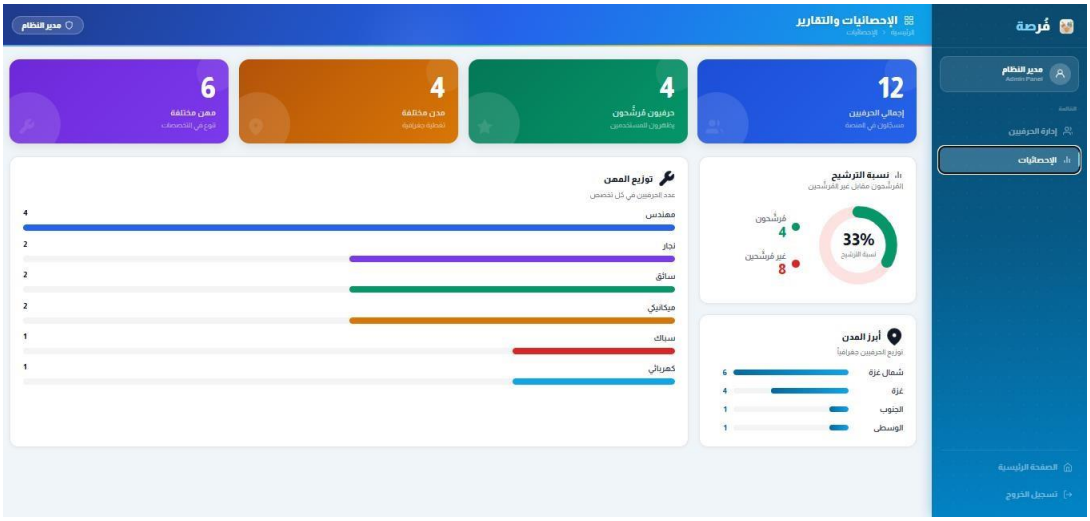
The admin panel provides a complete management interface for all registered craftsmen. The sidebar includes navigation links for craftsmen management, statistics, and the main site. The craftsmen management page displays a full table with each craftsman's name, profession, city and neighborhood, phone, email, years of experience, and featured status. A search bar allows filtering by name, phone, or email. Profession filter tabs at the top allow quick filtering by craft type. The admin can toggle the featured status of any craftsman or permanently delete their account.



(Figure 6.33— Admin Panel — Craftsmen Management Page)

Statistics and Reports Page

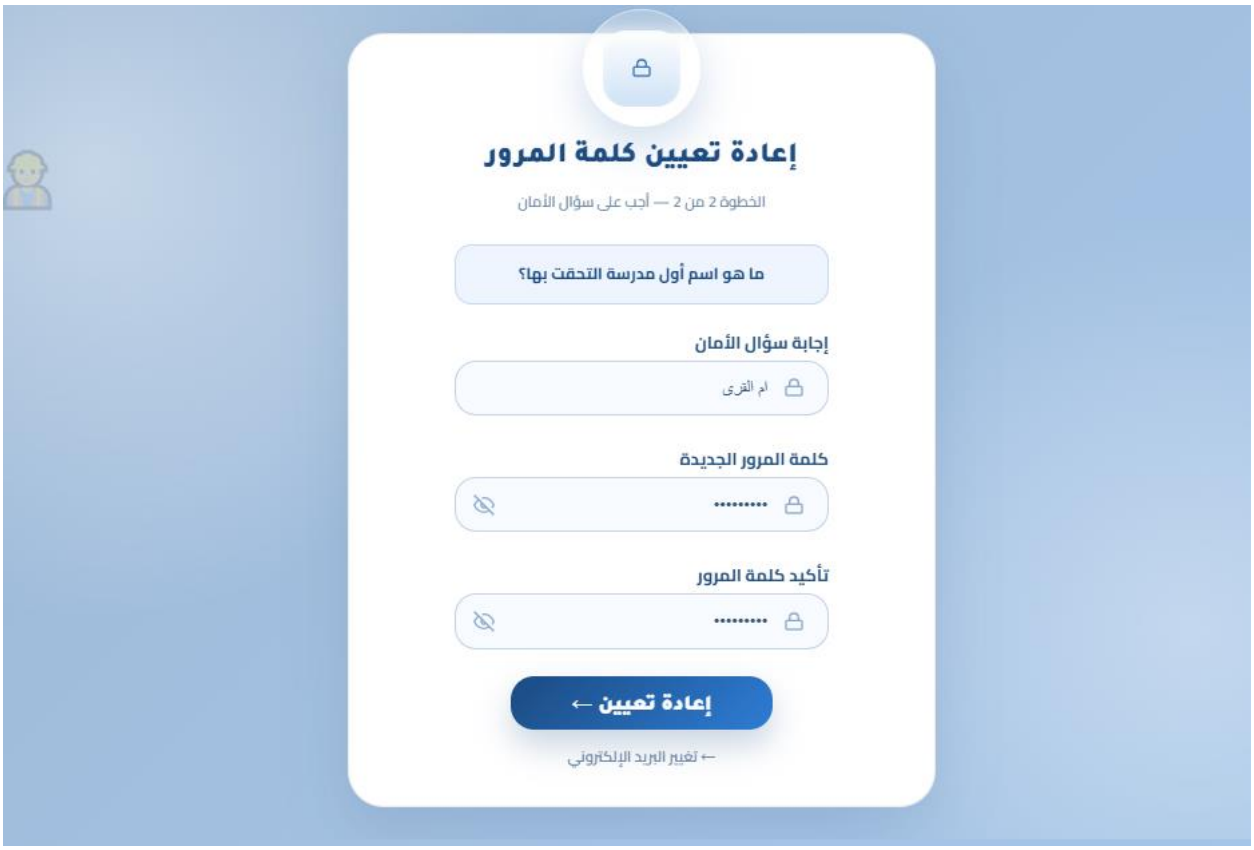
This page provides an overview of platform statistics through four summary cards: total craftsmen count, number of featured craftsmen, number of distinct cities covered, and number of distinct professions available. A profession distribution chart shows the breakdown of craftsmen by craft type, a featured percentage chart displays the ratio of featured to non-featured craftsmen, and a city distribution chart shows the geographic spread across Gaza regions.



(Figure 6.34 — Admin Panel — Statistics and Reports Page)

Creating New Password Page

This page allows craftsmen to reset their password. The user enters their registered email address, a new password, and a password confirmation. Both password fields include a visibility toggle button. The system verifies the email exists in the database and updates the stored password hash. A success message is displayed upon completion.



(Figure 6.35 — Create New Password Page)

Chapter 7
Testing and Quality Assurance

Chapter 7
Testing and Quality Assurance

In this chapter we will discuss all the units developed in the implementation phase that are integrated into a system after testing of each unit. Post integration, the entire system is tested for any faults and failures. Testing was performed in multiple ways as follows:

7.1 Unit testing

Unit testing is a type of software testing where individual units or components of a software are tested. The purpose is to validate that each unit of the software code performs as expected. Unit Testing is done during the development (coding phase) of an application by the developers. Unit Tests isolate a section of code and verify its correctness. A unit may be an individual function, method, procedure, module, or object.

In the Forsa platform, unit tests were applied to the core validation and data-fetching functions across all six modules: Login, Signup, Password Reset, Search, Craftsman Profile, and Profile Management.

7.1.1 Login module

Figure 7.1 shows the unit test for the login validation function that checks whether the email and password fields are empty before sending the request to the server.

```
93    // في صفحة الدخول نتحقق فقط من وجود القيم
94    const validate = () => {
95      const newErrors = {
96        email: "",
97        password: "",
98      };
99
100     if (!formData.email.trim()) {
101       newErrors.email = "البريد الإلكتروني مطلوب";
102     }
103
104     if (!formData.password.trim()) {
105       newErrors.password = "كلمة المرور مطلوبة";
106     }
107
108     return newErrors;
109   };
```

Figure 7.1 unit test for login validation function

Figure 7.2 shows the server response handler inside **`handleSubmit`**() that manages incorrect credentials and connection failures.

```
147     // Check server response success
148     const isSuccess =
149       response.ok &&
150       (result?.success === true || result?.status?.status === true);
151
152     if (!isSuccess) {
153       setServerError(
154         result?.message ||
155           result?.status?.message ||
156           "بيانات الدخول غير صحيحة"
157       );
158       return;
159     }
174       // Handle network/connection error
175     } catch (error) {
176       setServerError("تعذر الاتصال بالسيرفر");
```

Figure 7.2 unit test for login server response handler

7.1.2 Signup module

Figure 7.3 shows the unit test for the email and password strength validation inside **`validateForm()`**. The function checks email format, password length, character requirements, and password confirmation match.

```
110     if (!data.email.trim()) {
111       newErrors.email = 'البريد الإلكتروني مطلوب';
112     } else if (!data.email.toLowerCase().endsWith('@gmail.com')) {
113       newErrors.email = 'يجب أن يحتوي الإيميل على @gmail.com';
114     }
115
116     if (!data.password) {
117       newErrors.password = 'كلمة المرور مطلوبة';
118     } else if (data.password.length < 8) {
119       newErrors.password = 'كلمة المرور يجب أن تكون 8 أحرف على الأقل';
120     } else if (!/(?=.*[a-zA-Z])(?=.*\d)(?=.*[@$!%*?&])/.test(data.password)) {
121       newErrors.password = 'يجب أن تحتوي على أحرف وأرقام ورمز خاص';
122     }
123
124     if (!data.confirmPassword) {
125       newErrors.confirmPassword = 'تأكيد كلمة المرور مطلوب';
126     } else if (data.password !== data.confirmPassword) {
127       newErrors.confirmPassword = 'كلمتا المرور غير متطابقتين';
128     }
```

Figure 7.3 unit test for signup email and password validation

Figure 7.4 shows the unit test for the phone number, hourly price, and work images validation.

```
130     if (!data.phone.trim()) {
131       newErrors.phone = 'رقم الهاتف مطلوب';
132     } else if (!data.phone.startsWith('059') || data.phone.length !== 10) {
133       newErrors.phone = 'رقم الهاتف يجب أن يبدأ بـ 059 ويتكون من 10 أرقام';
134     }
```

```
140     if (!data.price.trim()) {
141       newErrors.price = 'سعر الساعة مطلوب';
142     } else if (!/^\d+$/.test(data.price) || Number(data.price) < 1) {
143       newErrors.price = 'سعر الساعة يجب أن يكون رقماً أكبر من أو يساوي 1';
144     }
```

```
162     if (data.workImages.length !== 3) {
163       newErrors.workImages = 'يجب إضافة 3 صور أعمال بالضبط';
164     }
```

Figure 7.4 unit test for phone, price, and work images validation

7.1.3 Password reset module

Figure 8.5 shows the unit test for the password reset validation function inside **`createNewPassword.jsx`.** It verifies the email format, new password strength, and confirmation match before submitting.

```
79  const validate = () => {
80    const newErrors = {
81      email: "",
82      newPassword: "",
83      confirmPassword: "",
84    };
85
86    if (!formData.email.trim()) {
87      newErrors.email = "البريد الإلكتروني مطلوب";
88    } else if (!/^[a-zA-Z0-9._%+-]+@gmail\.com$/.test(formData.email)) {
89      newErrors.email = "يجب أن يكون البريد من نوع @gmail.com";
90    }
91
92    if (!formData.newPassword.trim()) {
93      newErrors.newPassword = "كلمة المرور مطلوبة";
94    } else if (formData.newPassword.length < 8) {
95      newErrors.newPassword = "كلمة المرور يجب أن تكون 8 أحرف على الأقل";
96    } else if (
97      !/^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*?&_\-#^()+=]).{8,}$/.test(
98        formData.newPassword
99      )
100   ) {
101     newErrors.newPassword =
102       "كلمة المرور يجب أن تحتوي على حروف وأرقام ورمز خاص";
103   }
104
105   if (!formData.confirmPassword.trim()) {
106     newErrors.confirmPassword = "تأكيد كلمة المرور مطلوبة";
107   } else if (formData.confirmPassword !== formData.newPassword) {
108     newErrors.confirmPassword = "كلمتا المرور غير متطابقتين";
109   }
110
111   return newErrors;
112 };
```

Figure 7.5 unit test for password reset validation function

7.1.4 Search module

Figure 7.6 shows the unit test for the **`fetchResults()`** function in **`SearchResults.jsx`.** It dynamically builds the API URL based on the search query and handles empty results and connection errors.

```
454        const url = searchQuery
455          ? `${API_BASE_URL}/api/craftsmen?search=${encodeURIComponent(searchQuery)}`
456          : `${API_BASE_URL}/api/craftsmen`
457
458        const response = await fetch(url)
459        const payload = await readJsonSafe(response)
460
461        const isSuccess =
462          response.ok &&
463          (payload?.success === true || payload?.status?.status === true)
464
465        if (!isSuccess) {
466          throw new Error(
467            payload?.message ||
468            payload?.status?.message ||
469            'حدث خطأ أثناء جلب نتائج البحث'
470          )
471        }
472
473        const normalized = extractCraftsmen(payload).map(normalizeCraftsman)
474        setResults(normalized)
475      } catch (err) {
476        setResults([])
477        setError(err.message || 'تعذر الاتصال بالسيرفر')
```

Figure 7.6 unit test for search results fetch function

7.1.5 Profession craftsmen filter module

Figure 7.7 shows the unit test for the client-side filter logic inside **ProfessionCraftsmen.jsx**. It filters craftsmen by name search term and sorts results by experience or price

```
211  const filteredCraftsmen = useMemo(() => {
212    const query = normalizeText(searchTerm);
213
214    let result = visibleCraftsmen.filter((craftsman) => {
215      if (!query) return true;
216
217      const fullName = `${craftsman.firstName || ""} ${craftsman.lastName || ""}`;
218      return normalizeText(fullName).includes(query);
219    });
220
221    result = [...result].sort((a, b) => {
222      if (sortBy === "experience") {
223        return (
224          Number(b.yearsOfExperience || 0) - Number(a.yearsOfExperience || 0)
225        );
226      }
227
228      if (sortBy === "price") {
229        const aPrice = Number(a.price || Infinity);
230        const bPrice = Number(b.price || Infinity);
231        return aPrice - bPrice;
232      }
233
234      return getRatingValue(b) - getRatingValue(a);
235    });
236
237    return result;
238  }, [visibleCraftsmen, searchTerm, sortBy]);
```

Figure 7.7 unit test for profession page filter and sort function

7.1.6 Craftsman profile and service request module

Figure 7.8 shows the unit test for the **fetchCraftsman()** function that loads a craftsman's data by ID, and **Figure 7.9** shows the service request **validate()** function.

```
const response = await fetch(`${API_BASE_URL}/api/craftsmen/${id}`);
const payload = await readJsonSafe(response);
if (!response.ok) {
  throw new Error(payload?.status?.message || payload?.message || "تعذر جلب بيانات الحرفي");
}
const normalized = normalizeCraftsman(extractCraftsman(payload));
if (!normalized) {
  throw new Error("بيانات الحرفي غير مكتملة");
}
setCraftsman(normalized);
```

Figure 7.8 unit test for craftsman profile fetch function

Figure 7.9 shows the service request **validate()** function.

```
const validate = () => {
  const nextErrors = {};
  if (!form.clientName.trim()) {
    nextErrors.clientName = "الاسم مطلوب";
  }
  const cleanedPhone = form.clientPhone.replace(/\s+/g, "").trim();
  if (!cleanedPhone) {
    nextErrors.clientPhone = "رقم الهاتف مطلوب";
  } else if (!/^\d{10}$/.test(cleanedPhone)) {
    nextErrors.clientPhone = "رقم الهاتف يجب أن يكون 10 أرقام";
  }
  return nextErrors;
};
```

Figure 7.9 unit test for service request validation function

7.1.7 Craftsman profile management module

Figure 7.10 shows the unit test for the **`fetchMyProfile()`** function that loads the logged-in craftsman's own data using a Bearer token, and **Figure 7.11** shows the save profile handler that sends updated data via PATCH request.

```
const fetchMyServiceRequests = async () => {
  const token = getToken();
  if (!token) {
    navigate("/login");
    return;
  }
  try {
    setLoadingRequests(true);
    setRequestsError("");
    const response = await apiFetch(MY_REQUESTS_ENDPOINT, { method: "GET" });
    const result = await response.json();
    const isSuccess =
      response.ok &&
      (result?.success === true || result?.status?.status === true);
    if (!isSuccess) {
      throw new Error(
        result?.message ||
          result?.status?.message ||
          "تعذر جلب طلبات الخدمة"
      );
    }
    const normalized = extractRequests(result).map(normalizeRequest);
    setServiceRequests(normalized);
    setRequestsFetched(true);
  } catch (error) {
    setRequestsError(error.message || "تعذر الاتصال بالسيرفر");
```

Figure 7.10 unit test for craftsman profile page fetch function

Figure 7.11 shows the save profile handler that sends updated data via PATCH request.

```
const response = await fetch(PROFILE_ENDPOINT, {
  method: "PATCH",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify({
    firstName: draft.firstName.trim(),
    lastName:  draft.lastName.trim(),
    phone:     draft.phone.trim(),
    city:      draft.city.trim(),
    yearsOfExperience: Number(draft.yearsOfExperience),
    price:     Number(draft.price),
  }),
});
if (!isSuccess) {
  showToast(result?.message || "فشل حفظ التعديلات", "⚠");
  return;
}
showToast("تم حفظ التغييرات بنجاح");
```

Figure 7.11 unit test for save profile PATCH request handler

7.1.8 Admin panel module

Figure 7.12 shows the unit test for the admin panel's **`toggleFeatured()`** function that marks or unmarks a craftsman as featured, and the `filtered` logic that searches and filters craftsmen by profession.

```
  const res = await fetch(`${ADMIN_CRAFTSMEN_ENDPOINT}/${craftsman.id}/featured`, {
    method: "PATCH",
    headers: getHeaders(true),
    body: JSON.stringify({ featured: nextFeatured, isFeatured: nextFeatured }),
  });
  const payload = await readJsonSafe(res);
  if (!res.ok) throw new Error(payload?.status?.message || payload?.message ||
    "فشل تحديث حالة الترشيح");
const filtered = craftsmen
  .filter(item => profession === "الكل" || item.profession === profession)
  .filter(item =>
    [item.fullName, item.phone, item.email, item.city]
      .some(v => String(v).toLowerCase().includes(q))
);
```

Figure 7.12 unit test for admin toggle featured and filter functions

7.2 Black box testing

Black box testing involves testing a system with no prior knowledge of its internal workings. A tester provides input and observes the output generated by the system under test. This makes it possible to identify how the system responds to expected and unexpected user actions, its response time, usability issues, and reliability issues.

1. Website testing

Table 7.1 Website testing

ID	Feature	Expected result	Actual result	Status
BT_01	Sign in with correct credentials	The craftsman is signed in and redirected to the home page.	As expected	Pass
BT_02	Sign in with wrong password	Error message is displayed; user stays on the login page.	As expected	Pass
BT_03	Sign in with empty fields	Inline validation error messages appear under the empty fields.	As expected	Pass
BT_04	Register new craftsman	The craftsman completes all 3 steps; account is created and saved in the system.	As expected	Pass
BT_05	Register with weak password	Error message prompts user to use letters, numbers, and a special character.	As expected	Pass
BT_06	Register with less than 3 work images	Error message states that exactly 3 work images are required.	As expected	Pass
BT_07	Reset password	User provides email and new password; success message appears after submission.	As expected	Pass
BT_08	Search for craftsman by profession	Search results page shows all craftsmen matching the searched profession.	As expected	Pass
BT_09	Search with no matching results	System shows "لم يتم العثور على حرفي" message with a return-to-search button.	As expected	Pass
BT_10	Browse craftsmen by category	Clicking a profession category navigates to a filtered list of craftsmen in that profession.	As expected	Pass
BT_11	Filter craftsmen by city	Selecting a city from the filter shows only craftsmen from that city.	As expected	Pass
BT_12	Sort craftsmen by experience	Craftsmen are reordered with the most experienced appearing first.	As expected	Pass
BT_13	Sort craftsmen by price	Craftsmen are reordered from lowest to highest hourly price.	As expected	Pass
BT_14	View craftsman profile page	Full profile page displays: bio, contact details, location map, and work images.	As expected	Pass

ID	Feature	Expected result	Actual result	Status
BT_15	Send service request	User submits name and phone number; a WhatsApp link is provided to contact the craftsman.	As expected	Pass
BT_16	Service request with empty fields	Error messages appear under the empty name and phone fields.	As expected	Pass
BT_17	Switch language (AR / EN)	All visible text on the page switches between Arabic and English correctly.	As expected	Pass

2. Dashboard testing

Table 7.2 Craftsman dashboard testing

ID	Feature	Expected result	Actor	Actual result	Status
BT_18	View own profile page	The craftsman's dashboard loads with their current data pre-filled in the form.	Craftsman	As expected	Pass
BT_19	Edit and save profile info	Updated bio, phone, city, or price is saved and reflected immediately on the profile.	Craftsman	As expected	Pass
BT_20	Update profile image	New profile image under 2 MB is uploaded and displayed; files over 2 MB are rejected.	Craftsman	As expected	Pass
BT_21	View incoming service requests	List of all service requests submitted by users is displayed correctly.	Craftsman	As expected	Pass
BT_22	Redirect own public profile	When a logged-in craftsman visits their public profile URL, they are redirected to /profile automatically.	Craftsman	As expected	Pass
BT_23	Access dashboard without login	Unauthenticated access to /profile redirects the user to the login page.	Craftsman	As expected	Pass

3. Admin panel testing

Table 7.3 Admin panel testing

ID	Feature	Expected result	Actor	Actual result	Status
BT_24	Admin login	The admin logs in with admin credentials and is redirected to the /admin dashboard.	Admin	As expected	Pass
BT_25	View all craftsmen	Admin panel loads and displays all registered craftsmen with their details.	Admin	As expected	Pass
BT_26	Filter craftsmen by profession	Selecting a profession from the filter shows only craftsmen of that profession.	Admin	As expected	Pass
BT_27	Search craftsmen in admin panel	Typing a name, city, phone, or email finds matching craftsmen instantly.	Admin	As expected	Pass
BT_28	Toggle craftsman featured status	Marking a craftsman as featured updates their status; unmarking reverses it.	Admin	As expected	Pass

7.3 Acceptance test

Acceptance testing, a testing technique performed to determine whether or not the software system has met the requirement specifications. The main purpose of this test is to evaluate the system's compliance with the business requirements and verify if it is having met the required criteria for delivery to end users.

We presented the Forsa platform to a group of real users from the target audience — including individuals who need craftsmen services and craftsmen themselves — and received feedback on the developed system. They positively evaluated the overall experience and we took note of their comments to improve the platform further.

Here are some of the points that the users tested:

1. **Acceptance tests for login and password reset:**
   - Check whether login works with correct credentials.
   - Check that it does not work with wrong credentials.
   - Check that empty fields show appropriate error messages.
   - Check whether password reset successfully updates the craftsman's password.
2. **Acceptance tests for craftsman registration:**
   - Check that the 3-step form navigates correctly and saves progress between steps.
   - Check that the system rejects weak passwords and invalid email formats.
   - Check that exactly 3 work images must be uploaded before final submission.
   - Check that the craftsman's profile appears on the platform after successful registration.

3.Acceptance tests for searching and filtering craftsmen:

- Check that searching by profession returns relevant craftsmen.
- Check that browsing by category navigates to the correct profession page.
- Check that city filter and sort options work correctly.
- Check that language toggle switches all visible text between Arabic and English.

4.Acceptance tests for service request:

- Check that the craftsman profile page displays all details correctly.
- Check that the service request form validates name and phone before sending.
- Check that a WhatsApp link is provided after a successful service request.

5.Acceptance tests for craftsman dashboard:

- Check that the craftsman can edit and save profile information successfully.
- Check that updating the profile image works and the new image is displayed.
- Check that incoming service requests are listed correctly in the dashboard.

7.4 Performance testing

Performance testing is a non-functional software testing technique that determines how the stability, speed, scalability, and responsiveness of an application hold up under a given workload. It's a key step in ensuring software quality, but unfortunately, it is often seen as an afterthought, in isolation, and to begin once functional testing is completed, and in most cases, after the code is ready to release.

By testing and using the system functions, we estimated the server response speed for the following functions which are estimated from 1s to 8s:

- Searching for craftsmen by profession or name
- Loading the craftsman profile page with work images
- Craftsman registration with image upload
- Submitting a service request
- Craftsman profile page: loading profile data and service requests
- Admin panel loading and management operations

We have shown the system on many browsers and the performance of the system in all of them was excellent, such as:

- Google Chrome.
- Microsoft Edge.
- Firefox.

Chapter 8
Conclusion and Future works

8.1 ConclusionChapter 7

This project successfully achieved its main goal of building a web platform that connects craftsmen and skilled workers with people in need of their services within the Gaza Strip, in a way that makes it easy for users to reach a suitable craftsman without having to create an account. The platform allows visitors to browse craftsmen and filter them by area, type of craft, and budget, view each craftsman's details and previous work, and then contact him via WhatsApp after filling out a simple registration form. It also provides craftsmen with a dedicated dashboard to manage their personal profiles, add a portfolio of their work, and follow up on the service requests they receive.

The platform was implemented using the MERN technology stack: the front end was built with React, the server was developed using Node.js and Express, and a MongoDB database was used to store the data. The work included applying a set of security measures, such as authentication via JWT, hashing passwords and security-question answers using bcrypt, rate limiting to protect against abuse, in addition to protection against injection attacks. The platform was also distinguished by its support for both Arabic and English, a responsive design that works across different screen sizes, and an admin dashboard that enables the monitoring of craftsmen and requests.

Despite the completion of the platform's core functionalities, the team faced some challenges, most notably the frequent internet outages and the limited time available, which led us to focus first on building the essential and necessary features and to defer the rest to future work. Among the most prominent features that were not completed is the craftsman rating system, which was postponed due to time constraints. In addition, the platform has not yet been tested with real users in an actual environment. This experience allowed us to deepen our understanding of full-stack development, working with RESTful APIs, and the principles of security in web applications, while also developing our teamwork and time-management skills under pressure
.

8.2 Future Work

Despite the completion of the core functionalities of the Forsa platform, there are several improvements and additions that could be developed in the future to enhance the platform's value and expand the scope of its services, most notably:

1- Adding a Posts Section: Developing a section that allows users to publish service requests for crafts not currently available on the platform (e.g., "I need a well driller" or "I need a wood installer for tents"), so that craftsmen and platform visitors can view these posts and interact with them through comments and sharing. This widens the range of services offered and increases user engagement.

2-Completing the Ratings and Reviews System: Finalizing the existing rating structure so that a customer can rate a craftsman after contacting him, with the ability to nominate highly

rated craftsmen to appear within the featured craftsmen list, which enhances trust and credibility on the platform.

3-Account Verification: Adding a mechanism to verify craftsmen's accounts in order to confirm their identities and the credibility of their data, thereby raising the level of security and trust between the parties.

4-Adding an Internal Chat System: Building a direct messaging system within the platform between the craftsman and the customer, eliminating the need for external communication via WhatsApp and allowing conversations to be tracked and managed within the platform itself.

5-Integrating an Electronic Payment System: Adding an electronic payment service within the platform to complete booking and payment operations, with the possibility of collecting a percentage of the service value as a revenue model for the platform, which transforms it from a mere directory of craftsmen into a fully integrated services platform.

References

References

[1] European Union, United Nations & World Bank Group (2026) Final Gaza Rapid Damage and Needs Assessment. Available at: https://www.un.org/unispal/document/final-gaza-press-release-20apr26/ (Accessed: July 2026).

[2] UNOSAT (2025) Gaza Strip Comprehensive Damage Assessment. Available at: https://www.un.org/unispal/document/unosat-gaza-strip-damage-assessment-31oct25/ (Accessed: July 2026).

[3] UNDP (2025) Clearing most of the rubble in the Gaza Strip is possible in seven years. Available at: https://www.undp.org/stories/clearing-most-rubble-gaza-strip-possible-seven-years-under-right-conditions (Accessed: July 2026).

[4] UNRWA (2025) Situation Report #192 on the Gaza Strip and West Bank. Available at: https://www.unrwa.org/resources/reports/unrwa-situation-report-192-situation-gaza-strip-and-west-bank-including-east-jerusalem (Accessed: July 2026).

[5] Market Research Future (2025) Home Improvement Service Market Report. Available at: https://www.marketresearchfuture.com (Accessed: July 2026).

[6] TMS Outsource (2025) Apps Like TaskRabbit: Gig Economy Market Data. Available at: https://tms-outsource.com/blog/posts/apps-like-taskrabbit/ (Accessed: July 2026).

[7] Contrary Research (2024) Thumbtack Company Profile. Available at: https://research.contrary.com/company/thumbtack (Accessed: July 2026).

[8] The National (2015) Justmop.com: the UAE start-up looking to scrub up the GCC's cleaning industry. Available at: https://www.thenationalnews.com/business/economy/justmop-com-the-uae-start-up-looking-to-scrub-up-the-gcc-s-cleaning-industry-1.989415 (Accessed: July 2026).

[9] International Labour Organization (2021) World Employment and Social Outlook: The Role of Digital Labour Platforms in Transforming the World of Work. Available at: https://ilabour.oii.ox.ac.uk/ilo-report-2021/ (Accessed: July 2026).

[10] World Bank Group (2019) Jobs in West Bank and Gaza: Enhancing Job Opportunities for Palestinians. Available at: https://documents1.worldbank.org/curated/en/523241562095688030/pdf/West-Bank-and-Gaza-Jobs-in-West-Bank-and-Gaza-Project-Enhancing-Job-Opportunities-for-Palestinians.pdf (Accessed: July 2026).

[11] SCRUMstudy (n.d.) Agile Scrum Adaptation to Change. Available at: https://www.scrumstudy.com/article/agile-scrum-adaptation-to-change (Accessed: July 2026).

[12] MongoDB – The Developer Data Platform. MongoDB Inc. Available at: https://www.mongodb.com/ (Accessed: December 2025).

[13] Express – Fast, Unopinionated, Minimalist Web Framework for Node.js. OpenJS Foundation. Available at: https://expressjs.com/ (Accessed: December 2025).

[14] React – A JavaScript Library for Building User Interfaces. Meta Open Source. Available at: https://react.dev/ (Accessed: December 2025).

[15] Node.js – JavaScript Runtime Built on Chrome's V8 Engine. OpenJS Foundation. Available at: https://nodejs.org/ (Accessed: December 2025).

[16] IJRASET (2023) MERN Stack: Technologies Used for Web Development. Available at: https://www.ijraset.com/research-paper/mern-stack-technologies-used-for-web-development (Accessed: July 2026).

[17] Palestinian Central Bureau of Statistics & Palestine Monetary Authority (2026) The Performance of the Palestinian Economy for 2025, and Economic Forecasts for 2026. Available at: https://pcbs.gov.ps/post.aspx?ItemID=6123&lang=en (Accessed: July 2026).

[18] Palestinian Central Bureau of Statistics (2022) Features of the Informal Sector in Palestine 2022. Available at: https://pcbs.gov.ps/post.aspx?ItemID=4565&lang=en (Accessed: July 2026).

[19] Electronic Frontier Foundation (2025) Connectivity is a Lifeline, Not a Luxury: Telecom Blackouts in Gaza. Available at: https://www.eff.org/deeplinks/2025/06/connectivity-lifeline-not-luxury-telecom-blackouts-gaza-threaten-lives-and-digital (Accessed: July 2026).

[20] Meir Amit Intelligence and Terrorism Information Center (2022) The Use of Social Networks by the Palestinian Public. Available at: https://www.terrorism-info.org.il/en/the-use-of-social-networks-by-the-palestinian-public-facts-and-assessments/ (Accessed: July 2026).

[21] TaskRabbit – Same Day Home Services. IKEA. Available at: https://www.taskrabbit.com/ (Accessed: December 2025).

[22] Thumbtack – Home Services Marketplace. Available at: https://www.thumbtack.com/ (Accessed: July 2026).

[23] Souq Al-Harafyeen – Craftsmen Directory Platform. Available at: https://souqalharafyeen.com/ (Accessed: July 2026).

[24] Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

[25] Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: https://agilemanifesto.org/ (Accessed: December 2025).

[26] bcrypt – A Library to Help Hash Passwords. npm. Available at: https://www.npmjs.com/package/bcryptjs (Accessed: December 2025).

[27] JSON Web Tokens (JWT) – Introduction and Standards. Available at: https://jwt.io/ (Accessed: December 2025).

[28] Vite – Next Generation Frontend Tooling. Available at: https://vitejs.dev/ (Accessed: December 2025).

[29] Vercel – Develop, Preview, Ship Frontend Applications. Vercel Inc. Available at: https://vercel.com/ (Accessed: December 2025).

[30] Render – Cloud Application Hosting for Developers. Available at: https://render.com/ (Accessed: December 2025).

[31] MongoDB Atlas – Multi-Cloud Developer Data Platform. MongoDB Inc. Available at: https://www.mongodb.com/atlas (Accessed: December 2025).

[32] Mongoose – Elegant MongoDB Object Modeling for Node.js. Available at: https://mongoosejs.com/ (Accessed: December 2025).

[33] Helmet – Help Secure Express Apps by Setting HTTP Response Headers. Available at: https://helmetjs.github.io/ (Accessed: December 2025).

Appendix A
User Interfaces

This appendix presents the complete set of user interface screens of the Forsa platform, including the home page, the “Who Are We” page, the account creation and login pages, the craftsman profile page, the browse-craftsmen page, the admin management panel, the statistics and reports dashboard, and the password reset page. The high-fidelity screenshots are inserted in the placeholders provided above, presented in the order in which users encounter them during a typical session.

Appendix B: Additional Sequence Diagrams

Chapter 5 presented the most important data-flow sequence diagrams (login, search, and service-request submission). For completeness, the remaining flows of the system are summarized below and may be expanded into full sequence diagrams in future documentation:

1. Craftsman Registration Flow – the craftsman submits registration data together with portfolio images; the backend validates the input through express-validator, hashes the password and the security answer with bcrypt, stores the new document, and returns a JWT token.

2. Profile Update Flow – an authenticated craftsman edits profile fields or uploads new images; the backend verifies the JWT, validates the changes, enforces the 12-image limit, and persists the update.

3. Update Service-Request Status Flow – the craftsman changes a request status; the backend validates the transition against the state machine before saving.

4. Toggle Featured Status Flow (Admin) – the admin enables or disables a craftsman’s featured flag; the backend verifies the admin JWT and updates the record.

5. Cascade Delete Flow (Admin) – the admin deletes a craftsman; the backend removes the craftsman and all associated service requests inside a single MongoDB transaction, then deletes the related image files from disk.

Appendix C: Security Questions for Password Reset

To verify a craftsman’s identity during password reset without relying on external email or SMS services, the platform uses a predefined security-question mechanism. During registration, the craftsman selects one of the following three questions and provides an answer, which is normalized (lower-cased and trimmed) and stored as a bcrypt hash:

• What is the name of the first school you attended?

• What is the name of your best childhood friend?

• What is the name of the street where you spent your childhood?

During password reset, the craftsman enters the email, the system returns the associated security question, and the submitted answer is compared against the stored hash using bcrypt.compare before allowing the password to be changed.